COMSATS University Islamabad, Sahiwal Campus.

GiveEase

A Project Presented To

COMSATS University Islamabad, Sahiwal Campus

In partial fulfillment

of the requirement for the degree of

Bachelor of Science in Software Engineering (2022-2026)

By

Muhammad Saad	**CIIT/FA22-BSE-075/SWL**
Muhammad Ali	**CIIT/FA22-BSE-114/SWL**
Usman Javed	**CIIT/FA22-BSE-130/SWL**

Declaration

We hereby declare that this software, neither whole nor as a part has been copied out from any source. It is further declared that we have developed this software and accompanied report entirely on the basis of our personal efforts. If any part of this project is proved to be copied out from any source or found to be reproduction of some other. We will stand by the consequences. No Portion of the work presented has been submitted of any application for any other degree or qualification of this or any other university or institute of learning.

Muhammad Saad

Muhammad Ali

Usman Javed

Certificate of Approval

It is to certify that the final year project of BS(SE) **"GiveEase"** was developed by **Muhammad Saad(CIIT/FA22-BSE-075/SWL)** , **Muhammad Ali(CIIT/FA22-BSE-114/SWL) , Usman Javed(CIIT/FA22-BSE-130/SWL)** and under the supervision of "**Ms. Afia Afzaal**" and co supervisor "**Dr. Javed Ferzund**" and that in their opinion; it is fully adequate, in scope and quality for the degree of Bachelors of Science in Software Engineering.

Supervisor

Ms. Afia Afzaal

Lecturer

Department of Software Engineering

COMSATS University Islamabad, Sahiwal Campus

External Examiner

Dr Ali Samad Tauni

Associate Professor

Department of Computer Science

Islamia University Bahawalpur

Head of Department

Dr. Zafar Iqbal Roy

Assistant Professor

Department of Computer Science

COMSATS University Islamabad, Sahiwal Campus

Executive Summary

GiveEase is a native Android mobile application. It connects individual donors with manually verified NGOs in Pakistan, supporting monetary, physical goods, and blood donations within a single interface. The application addresses critical challenges like trust, transparency, and logistical inefficiency in the traditional donation system, where donors cannot verify NGO is authentic or not and NGOs struggle to communicate specific needs.

GiveEase solves these problems. Through manual verification with visual "verified badges" ensuring only verified NGOs and Donors access the platform to prevent scam. Real-time chat with image sharing enables seamless logistics coordination. A gamified reward system assigns milestone-based badges (Bronze to Platinum) and calculates impact points to encourage donors to donate more. An Impact Dashboard helps donors see their total donations and the difference they have made. An Admin Panel provides verification controls and system-wide maintenance mode.

This application was developed using Kotlin for the frontend and Google Firebase as a serverless backend, following Agile Scrum methodology with two-week sprints by a three-member student team over 6 to 8 months. Testing through 25 test cases achieved a 100% pass rate. Campaign feeds load within 2.1 seconds, chat messages deliver within 0.8 seconds, and image uploads complete within 3.2 seconds over 4G networks. User Acceptance Testing with three local NGOs and ten donors yielded satisfaction ratings of 4.6 and 4.7 out of 5.

GiveEase successfully meets all project objectives: trust through verification, diverse donation types, real-time communication, donor engagement through gamification, NGO empowerment, and administrative management. The application is stable, secure, and ready for deployment.

Acknowledgement

All praise is to Almighty Allah who bestowed upon us a minute portion of His boundless knowledge by virtue of which we were able to accomplish this challenging task.

We are greatly indebted to our project supervisor “Ms. Afia Afzaal” and our Co-Supervisor “Mr. Javed Ferzund”. Without their personal supervision, advice and valuable guidance, completion of this project would have been doubtful. We are deeply indebted to them for their encouragement and continual help during this work.

And we are also thankful to our parents and family who have been a constant source of encouragement for us and brought us the values of honesty & hard work.

Muhammad Saad

Muhammad Ali

Usman Javed

Abbreviations

Abbreviation	Full Form
API	**Application Programming Interface**
BaaS	**Backend-as-a-Service**
CRUD	**Create, Read, Update, Delete**
ERD	**Entity Relationship Diagram**
FCM	**Firebase Cloud Messaging**
IDE	**Integrated Development Environment**
NGO	**Non-Governmental Organization**
NoSQL	**Not Only SQL**
SDK	**Software Development Kit**
SRS	**Software Requirements Specification**
SDD	**Software Design Description**
UAT	**User Acceptance Testing**
UID	**Unique Identifier**
UML	**Unified Modeling Language**

Table of Contents

Contents

List of Tables

List of Figures

CHAPTER 1
INTRODUCTION

1. INTRODUCTION

This chapter provides a comprehensive overview of the GiveEase Donation App project. It begins with a brief introduction to the project's core outcome, highlighting the development of a native Android application that connects donors with verified NGOs for multiple donation types (money, physical goods, and blood). The chapter then establishes the project's academic relevance by mapping its development to specific courses from the BSSE curriculum, including Software Engineering, Mobile Application Development, and Cloud Computing. Following this, the project background is discussed, detailing the societal and technological gaps that motivated this work. A literature review of existing systems (GoFundMe, DonorsChoose, BloodConnect) and academic papers on gamification, verification, and technology adoption in developing countries is presented, along with a critical analysis of how these works informed the design decisions for GiveEase. Finally, the chapter concludes by justifying the selection of the Agile Scrum methodology and the iterative software development lifecycle used to manage the project's execution over a series of two-week sprints.

1.1 Brief

The "GiveEase Donation App" (hereafter referred to as GiveEase) is a native Android mobile application developed to address the critical challenges of trust, transparency, and logistical inefficiency within the charitable donation ecosystem of Pakistan. The core outcome of this work is a fully functional, serverless mobile platform that seamlessly connects individual donors with manually verified Non-Governmental Organizations (NGOs). Unlike existing solutions that are often limited to a single donation type, GiveEase supports a diverse range of contributions, including monetary donations (via mock payment), physical goods (clothes, food, books, electronics, furniture, medical supplies), and blood donations, all within a single, unified interface.

The system uses a modern, scalable architecture built with Kotlin for the frontend and Google Firebase as the backend solution. Firebase handles authentication, structured data storage (Cloud Firestore), real-time communication (Realtime Database), file storage (Cloud Storage), and push notifications (Cloud Messaging). The application incorporates key engagement features such as a real-time one-to-one chat system with image sharing, a gamified reward mechanism (badges and certificates), and a personalized impact dashboard to foster sustained donor participation.

GiveEase is designed as a centralized Android-based donation management system that bridges the gap between individual donors and NGOs. It provides a transparent platform for creating donation campaigns, facilitating real-time communication, and tracking social impact. Unlike traditional

methods that rely on manual bank transfers or physical visits, GiveEase utilizes a real-time database to ensure that every transaction and chat message is instantly synchronized, creating a "living" ecosystem of philanthropy. The system features three distinct user roles: Donors (who contribute resources), NGOs (who create campaigns and receive donations), and Admins (who verify NGOs and maintain platform integrity).

The development followed an Agile Scrum methodology, organized into two-week sprints, allowing for iterative development, continuous feedback integration, and effective risk management by a three-member student team. The subsequent sections of this chapter will detail the project's relevance to academic coursework, its background and motivation, a review of existing literature and systems, and the methodological approach taken to ensure successful delivery.

1.2 Relevance to Course Modules

The development of the GiveEase Donation App served as a comprehensive capstone project that effectively integrated and applied theoretical knowledge and practical skills acquired from multiple core courses throughout the Bachelor of Science in Software Engineering (BSSE) program. This section explicitly maps the project's technical components to specific academic modules, demonstrating the multi-disciplinary nature of modern software development.

1.2.1 Mobile Application Development (CSD-302)

The frontend is built natively using Kotlin and XML. Concepts from this course applied include:

- **Android Activity Lifecycle Management:** The application properly handles activity states such as onCreate, onStart, onResume, onPause, and onDestroy. When a user receives a phone call during a chat, the app saves the current message draft in onPause and restores it in onResume, preventing data loss.
- **Fragment-based Navigation for Role-specific Interfaces:** A single MainActivity hosts different fragments based on the logged-in user's role. When a donor logs in, DonorHomeFragment loads. When an NGO logs in, NgoMainFragment loads. When an admin logs in, AdminMainFragment loads. This eliminates code duplication.
- **RecyclerView with Custom Adapters:** All lists in the application (donation campaigns, chat messages, donation history) use RecyclerView for efficient memory management. Custom adapters bind data from Firebase to UI components and handle view recycling automatically during scrolling.

- **Coroutines for Asynchronous Firebase Operations:** All Firebase network calls are wrapped in Kotlin Coroutines. withContext (Dispatchers.IO) moves database operations to background threads. withContext (Dispatchers.Main) updates the UI. This prevents the app from freezing during data loading.
- **Permission Handling for Camera and Storage Access:** The app requests camera and storage permissions at runtime for Android 6.0 and above. When a donor wants to upload a photo of physical goods, the app checks permission first. If denied, it shows a rationale dialog before requesting again.

1.2.2 Database Systems (CSD-203)

Firebase Firestore serves as the backend database. This course contributed:

- **NoSQL Data Modeling with Collections and Documents:** The database is organized into collections: users, donation_requests, donations, chat_rooms, and rewards. Each document contains fields relevant to its entity. For example, a campaign document stores ngoId, title, description, targetAmount, raisedAmount, status, and imageUrl.
- **CRUD Operations for Users, Campaigns, and Donations:** The app performs complete CRUD functionality. Donors read campaigns and create donations. NGOs create, read, update, and delete their own campaigns. Admins update user verification status. All operations use Firebase SDK methods like. add(), .set(), .update(), and .delete().
- **Security Rules for Role-based Access Control:** Firebase Security Rules restrict data access based on user role. Only admins can write to the maintenance collection. Only the NGO that created a campaign can edit it. Donors can read all active campaigns but cannot modify them.
- **Real-time Data Synchronization Using Snapshot Listeners:** The app uses Firestore's .addSnapshotListener() for live updates. When an NGO updates a campaign's raisedAmount after a donation, all donors viewing that campaign see the progress bar update automatically without refreshing.

1.2.3 Software Engineering (SWE-301)

The entire project lifecycle follows software engineering principles:

- **Requirement Gathering through Interviews and Questionnaires:** Before development, the team interviewed three local NGOs to understand their donation management needs. Questionnaires were distributed to 30 potential donors to identify preferred features. This produced the functional requirements documented in the SRS.
- **SRS and SDD Documentation Following IEEE Standards:** Both the Software Requirements Specification and Software Design Description follow IEEE 830 and 1016 standards. Each functional requirement uses "shall" language and is traceable to specific test cases.
- **Agile Scrum Methodology with Two-week Sprints:** The project was divided into 8 sprints of two weeks each. Each sprint began with planning and ended with a review meeting with the supervisor.
- **Version Control Using Git and GitHub:** All source code, Firebase security rules, and documentation were version-controlled using Git. Feature branching was used. Each new module was developed in a separate branch, then merged to main after code review.
- **MVVM Architecture for Separation of Concerns:** The application follows Model-View-ViewModel architecture. The View (Fragments) handles UI rendering only. The ViewModel holds LiveData and processes user actions. The Repository is the single source of truth for data.

1.2.4 Human-Computer Interaction (CSD-304)

The user interface follows Material Design 3 guidelines. HCI principles applied:

- **Role-based Dashboards for Relevant Feature Access:** The app displays three different interfaces based on user role. A donor sees campaigns and donation options. An NGO sees campaign creation and donor chat. An admin sees verification queue and maintenance controls. Users only see features relevant to them.
- **Consistent Navigation Patterns Across All Screens:** Every screen follows a consistent layout. The bottom navigation bar contains 3-4 primary destinations. The top app bar shows the screen title and action icons. Back navigation is supported through the system back button and a back arrow.

- **Visual Feedback for User Actions:** Every user action triggers appropriate visual feedback. When a donor submits a donation, a progress indicator appears followed by a success toast. If an operation fails, an error dialog explains the issue. Buttons change color on press to confirm touch registration.
- **Accessible Design with Sufficient Contrast and Touch Targets:** All text meets WCAG 2.1 AA contrast requirements (minimum 4.5:1 for normal text). Touch targets are at least 48dp for easy tapping. The app supports TalkBack screen reader with content descriptions for all image buttons. Form fields include visible labels.

1.3 Project Background

The idea for the GiveEase Donation App originated from observing two significant, parallel problems in Pakistani society. On one hand, there is a strong culture of charitable giving, driven by religious, social, and humanitarian values. According to the Pakistan Centre for Philanthropy (PCP), Pakistanis donate an estimated PKR 240-300 billion annually to charitable causes. Individuals and organizations are often eager to donate money, food, clothes, and blood, especially during crises, natural disasters, or religious occasions like Ramadan.

On the other hand, the process of donating is frequently fragmented, inefficient, and plagued by a deep-seated lack of trust. Traditional donation methods are often informal and unstructured. People may donate through local mosques, community centers, or directly to individuals claiming to represent a cause. While well-intentioned, these channels lack transparency and accountability, making it difficult for donors to verify if their contributions genuinely reached the intended beneficiaries.

1.3.1 The Problem from the Donor's Perspective

For individual donors, the experience is often frustrating and unsatisfying:

1. **Verification Difficulty:** How does a donor know if an NGO is legitimate? There is no centralized registry or verification badge visible on social media posts.
2. **Limited Donation Options:** Most digital platforms only accept money. What if a donor wants to give 50 winter jackets, a refrigerator, or 10 units of blood? There is no easy digital channel for this.
3. **Communication Barriers:** Once a donor gives money or drops off items, the journey ends. There is no feedback loop, no "thank you," and no report on how the donation was used.

This discourages repeat giving.

1.3.2 The Problem from the NGO's Perspective

For organized NGOs, a different set of problems exists:

1. **Reaching the Right Audience:** NGOs struggle to communicate their specific, real-time needs (e.g., "We need 50 winter jackets for children at our shelter in Sahiwal," not "Please donate money") to a wide, relevant audience.
2. **Logistical Coordination:** Coordinating the collection of physical goods or arranging blood donations is often done through manual phone calls, WhatsApp messages, and spreadsheets. This is time-consuming and error-prone.
3. **Lack of Management Tools:** Most small to medium NGOs manage their campaigns and donor lists manually (e.g., using Excel spreadsheets), leading to data fragmentation and slow response times.
4. **Credibility Struggles:** Legitimate NGOs find it difficult to prove their credibility to a skeptical public, especially given the rise of fraudulent fundraising campaigns.

1.3.3 The GiveEase Solution

This identified gap formed the foundation for the GiveEase project. The app was designed from the ground up to:

1. **Establish Trust:** Through a mandatory manual verification process for both donors (CNIC) and NGOs (registration certificates), managed by an admin.
2. **Support Diverse Donations:** By providing distinct workflows for monetary donations (mock payment), physical goods (image capture and chat coordination), and blood donations (location and availability sharing).
3. **Enable Real-time Communication:** By integrating Firebase Realtime Database for one-to-one chat with image sharing between donors and NGOs.
4. **Engage Donors:** Through a gamified reward system (badges for milestones) and an Impact Dashboard that quantifies the donor's contribution (e.g., "You have helped 50 families").
5. **Empower NGOs:** By providing a dedicated panel to create campaigns, track donations, and communicate with donors.

6. **Provide Governance:** Through an Admin Panel that controls user verification, system settings, and a unique "Maintenance Mode" to lock the app during updates.

The project represents a practical application of modern software engineering principles to solve a real-world social problem, making it an ideal Final Year Project.

1.4 Literature Review

The development of GiveEase was informed by an analysis of existing academic literature and commercial systems in the domains of digital philanthropy, human-computer interaction for social good, and mobile donation platforms. This section reviews the current state of research and identifies gaps that GiveEase addresses.

1.4.1 Review of Commercial Systems

1.4.1.1 GoFundMe (gofundme.com)

GoFundMe has raised over $15 billion since its 2010 launch with 100+ million donors. The platform allows individuals to create campaigns for medical expenses, education, emergencies, and community projects.

Relevance to GiveEase: GoFundMe validates the massive market for digital donation platforms. However, its limitations demonstrate the need for a more comprehensive, multi-donation-type system with built-in verification and communication tools.

1.4.1.2 DonorsChoose (donorschoose.org)

DonorsChoose is a US-based nonprofit platform that allows public school teachers to create project requests for classroom materials. Donors can fund specific projects, and upon completion, teachers send thank-you notes and photos of the project in action.

Relevance to GiveEase: DonorsChoose demonstrates the power of specific, verifiable needs and post-donation impact reporting. GiveEase adapts this concept by allowing NGOs to post specific requests e.g., 50 winter jackets and providing an Impact Dashboard to show cumulative donor impact.

1.4.1.3 BloodConnect (bloodconnect.in)

BloodConnect is a mobile application and web platform operating in India that connects blood donors with those in need. It functions as a digital blood donor registry.

Relevance to GiveEase: BloodConnect shows the importance of dedicated workflows for time-sensitive, critical donations. GiveEase integrates blood donation into a broader ecosystem while adding NGO verification and donor rewards.

1.4.2 Review of Academic Literature

1.4.2.1 Kumar & Sharma (2023) – "Gamification in Donation Apps: Enhancing Donor Engagement"

Kumar and Sharma (2023) studied how gamification affects donor behavior in mobile donation platforms. Their experiment with 500 participants over six months found that badges and progress tracking increased donation frequency by 42%. They also identified that donors who received badges showed a psychological "endowment effect," feeling a sense of ownership over their charitable identity. Based on these findings, GiveEase implements a milestone-based badge system (Bronze, Silver, Gold, Platinum) and a progress indicator showing how many more donations are needed for the next badge.

1.4.2.2 Patel & Gupta (2023) – "Secure User Verification in Mobile Donation Systems"

Patel and Gupta analyzed different user verification mechanisms in mobile donation platforms, comparing automated AI-based verification versus manual admin review. Their research concluded that manual verification builds more perceived trust in high-fraud-risk environments, even though it is slower. They also found that visible verification badges significantly increase donor willingness to donate. Based on these findings, GiveEase implements manual verification for both donors (CNIC submission) and NGOs (registration certificate submission), with a distinct "Verified NGO" badge displayed on all campaigns.

1.4.2.3 Al-Rahmi et al. (2020) - "Factors affecting the adoption of mobile donation applications in developing countries"

Al-Rahmi and colleagues surveyed 1,200 respondents in Pakistan, Malaysia, and Indonesia to identify factors influencing adoption of mobile donation apps. Using Structural Equation Modeling, they found that trust, convenience, and observability were the strongest predictors of adoption. Trust had the strongest effect, followed by convenience (ease of use and time saved), and observability (ability to see the impact of one's donation). These findings validated GiveEase's core design priorities: verification for trust, multi-type donations for convenience, and an Impact Dashboard for

observability.

1.4.2.4 ISO/IEC/IEEE 29148:2018 - "Systems and software engineering — Life cycle processes — Requirements engineering"

This international standard provides guidelines for eliciting, documenting, and managing software requirements. It defines best practices for creating Software Requirements Specifications and ensuring traceability from requirements to design and testing. The SRS document created for GiveEase follows this standard's template, with each functional requirement written using "shall" language and mapped to specific test cases through a traceability matrix.

1.5 Analysis from Literature Review

Based on the review of commercial systems and academic literature presented in Section 1.4, this section provides a critical analysis of how these findings directly shaped the design, development, and unique value proposition of the GiveEase Donation App.

1.5.1 Comparative Positioning: How GiveEase Differs from Existing Systems

The analysis of commercial systems (GoFundMe, DonorsChoose, BloodConnect) confirmed the existence of a significant market gap.

Analysis Conclusion: While each existing system excels in its specific niche, none offers the combination of features present in GiveEase: multi-donation-type support, verified recipients, real-time communication, post-donation engagement, and administrative governance. This positions GiveEase as a unique, comprehensive solution for the Pakistani donation ecosystem.

1.5.2 Application of Gamification Theory (Kumar & Sharma, 2023)

Kumar & Sharma's empirical findings on gamification were directly translated into code-level implementation decisions.

Finding: Badges significantly increase donation frequency.
Implementation in GiveEase: The RewardSystem calculates badges based on cumulative donations stored in the users collection. The badge field is updated via a Cloud Function whenever a donation status changes to "COMPLETED."

Finding: Progress tracking (e.g., "2 donations to Silver") increases the likelihood of a subsequent

donation.

Implementation in GiveEase: The ImpactDashboardFragment calculates the donor's current badge and the number of donations remaining for the next badge. This progress is displayed as a progress bar and a text label (e.g., "5 more donations to reach Gold Badge").

Finding: Certificates trigger the psychological "endowment effect."
Implementation in GiveEase: When a donor reaches a milestone (e.g., 10 donations), a Cloud Function generates a PDF certificate using a template, uploads it to Firebase Storage, and stores the download URL in the rewards collection. The donor can download and share this certificate.

1.5.3 Application of Trust & Verification Research (Patel & Gupta, 2023)

Patel & Gupta's research on verification mechanisms directly informed the design of the NGO verification workflow.

Finding: Manual verification builds more perceived trust than automated verification in high-risk environments.
Implementation in GiveEase: All NGOs must submit their registration certificate (uploaded as an image to Firebase Storage). An admin reviews the document in the AdminVerificationFragment and manually toggles the isVerified flag from false to true.

Finding: Visible verification badges significantly increase donor willingness to donate.
Implementation in GiveEase: Any campaign displayed in the DonorHomeFragment includes the NGO's profile picture and a "Verified" badge (a green checkmark icon) if ngo.isVerified == true. This visual cue is prominently displayed next to the NGO name.

Finding: Manual verification creates a bottleneck.
Innovation in GiveEase: To address this limitation, we implemented a unique **"Auto-Approve" toggle** in the AdminSettingsFragment. When this switch is ON, newly registered NGOs are automatically assigned isVerified = true without admin intervention. This allows the system to operate in two modes: "Secure Mode" (manual verification) and "High-Volume Mode" (auto-approval for trusted partners).

1.5.4 Application of Adoption Factors (Al-Rahmi et al., 2020)

Al-Rahmi's identification of trust, convenience, and observability as the top adoption factors served as a validation checklist for GiveEase. Trust is addressed through the manual verification system and visible badges. Convenience is addressed by supporting three donation types (money, physical

goods, blood) in a single application, eliminating the need for multiple platforms. Observability is addressed through the Impact Dashboard, which shows donors their total donations, earned badges, and cumulative impact statistics.

1.5.5 Unique Contributions of GiveEase

Based on the literature analysis, this project makes the following unique contributions to the field of digital donation platforms:

1. **Multi-Donation-Type Integration:** Unlike single-purpose platforms (BloodConnect) or money-only platforms (GoFundMe), GiveEase provides unified workflows for monetary, physical, and blood donations. This is a novel integration not found in existing literature or commercial systems.
2. **Admin-Controlled Maintenance Mode:** A review of existing systems and literature revealed no mention of a "Maintenance Mode" feature for donation platforms. GiveEase implements a real-time app-wide lock controlled by an admin toggle. When active, all non-admin users are redirected to a MaintenanceActivity, preventing data corruption during backend migrations or emergency updates. This is an original contribution.
3. **Role-Based MVVM Architecture for Donation Systems:** While MVVM is a standard pattern, its specific application to a three-role (Donor, NGO, Admin) donation system with real-time Firebase synchronization is documented in this report as a reference architecture for similar student projects.
4. **Contextualized Solution for Pakistan:** The literature review confirmed a lack of comprehensive donation platforms specifically designed for the Pakistani context, considering local verification challenges (CNIC-based donor verification) and communication preferences (integrated chat). GiveEase fills this geographical and cultural gap.

1.6 Methodology and Software Lifecycle for This Project

The GiveEase Donation App was developed using a structured yet flexible approach to ensure timely delivery, quality assurance, and effective risk management. This section details the software process model selected and the architectural methodology employed.

1.6.1 Software Process Model: Agile Scrum

The project followed the Agile Scrum methodology, executed in an iterative fashion. This approach was chosen over traditional models like Waterfall because of our team size and evolving requirements, and a need for continuous feedback from the project supervisor, who acted as the primary stakeholder.

Sprint Structure and Duration:

The project was planned and executed as a series of time-boxed sprints, each lasting two weeks (10 working days). This duration was chosen because it is short enough to provide frequent feedback loops but long enough to deliver meaningful increments of functionality.

Daily Stand-up Meetings:

A 15-minute daily stand-up meeting was conducted each morning via Discord (or in-person when possible). The agenda followed the classic three-question format:

1. What did I complete yesterday?
2. What will I work on today?
3. Are there any blockers preventing my progress?

This ceremony ensured high visibility, early identification of technical issues (e.g., "I'm blocked because the Firebase Realtime Database rules aren't working"), and effective coordination among team members working on interdependent modules.

Sprint Review and Retrospective:

At the end of each sprint, two meetings were held:

1. **Sprint Review (with Supervisor):** The team demonstrated the working increment (e.g., "In Sprint 3, we can now create campaigns and upload images"). The supervisor provided feedback on UI/UX, feature completeness, and prioritization for the next sprint.
2. **Sprint Retrospective (Internal Team):** The team discussed what went well, what went wrong, and what could be improved. For example, after Sprint 1, we realized that Firebase Authentication documentation was incomplete for certain edge cases, so we allocated extra research time in Sprint 2.

1.6.2 Rationale behind Selected Methodology

Agile Scrum was selected over Waterfall for several specific reasons that aligned with the project's constraints and goals.

Accommodating Technical Uncertainty

The team had limited prior experience with Firebase real-time features, particularly the Realtime Database for chat and Cloud Functions for automated rewards. Agile allowed the team to build a simplified version of the chat feature in Sprint 4, identify issues such as message ordering problems, and enhance it in Sprint 5 with image sharing and proper timestamp handling. In a Waterfall model, the chat feature would have been designed upfront and tested only at the end, making such refinements costly and risky.

Continuous Stakeholder Feedback

The project supervisor acted as the primary stakeholder. The two-week sprint review cadence provided regular opportunities for her to inspect working software and provide feedback. For example, after Sprint 2, the supervisor noted that the donor feed was too cluttered and requested category filters. This feedback was implemented in Sprint 3. Such iterative refinement would not be possible in a Waterfall model where requirements are fixed before development begins.

Risk Mitigation for a Small Team

With only three developers, early detection of blockers was critical. Daily stand-up meetings revealed issues immediately. When a team member struggled with Firebase Storage security rules, the issue was identified in a stand-up and resolved within two days through pair programming. When another team member faced academic exam conflicts, tasks were reallocated to balance the workload. This visibility prevented small issues from becoming major delays.

Tangible Progress for Academic Milestones

The university required regular demonstrations of progress at specific milestones. Agile's sprint structure provided natural demonstration points. At the end of Sprint 2, the team showed a working donor panel. At the end of Sprint 4, they showed real-time chat working. This continuous delivery of working software satisfied academic requirements while keeping the team motivated.

Comparison with Alternative Models

Waterfall was rejected because it assumes all requirements are known upfront, which was not true given the team's learning curve with Firebase. The Spiral model was rejected because its heavy focus on risk analysis would add unnecessary overhead for a six-month student project.

CHAPTER 2

PROBLEM DEFINITION

2. PROBLEM DEFINITION

2.1 Problem Statement

Despite the growing desire for philanthropic activities in Pakistan, the current donation ecosystem suffers from six critical, interconnected problems that the GiveEase Donation App aims to solve. These problems were identified through stakeholder interviews with local NGOs, questionnaires distributed to potential donors, and a competitive analysis of existing donation platforms.

2.1.1 Problem 1: The Trust Deficit

Donors in Pakistan face significant difficulty in verifying the legitimacy of charitable organizations. There is no standardized, transparent, and easy-to-use mechanism for confirming whether an NGO is properly registered, whether its leadership is credible, and whether donations will be utilized as promised. This "trust deficit" manifests in several ways:

First, fraudulent fundraising campaigns have become increasingly common, particularly during natural disasters such as floods or earthquakes. Unscrupulous individuals create fake NGO profiles on social media, collect donations through informal channels like bank transfers or mobile wallets, and then disappear without delivering any aid. Legitimate donors who fall victim to such schemes become hesitant to donate again, harming the entire charitable sector.

Second, even for established NGOs, there is no universally recognized "verification badge" or certification that donors can quickly recognize. A donor browsing through different organizations has no way of distinguishing between a well-funded, transparent NGO and a poorly managed, opaque one without conducting extensive manual research.

Third, the absence of post-donation transparency compounds this distrust. Once a donor gives money or drops off physical items, the journey typically ends. Donors rarely receive feedback on how their contribution was used, what impact it created, or even a simple acknowledgment. This lack of closure discourages repeat giving.

2.1.2 Problem 2: Fragmented and Inefficient Donation Logistics

The process of donating anything other than cash is highly unstructured and inefficient in the current ecosystem. An individual wishing to donate physical goods such as clothes, books, food, electronics, furniture, or medical supplies has no centralized digital platform to find an NGO currently in need of those specific items at that specific moment.

Currently, a donor must manually search for NGO contact information (often from outdated websites or social media pages), call or message them, ask what items are needed, and then arrange for pick-up or drop-off through multiple phone calls. This process is time-consuming, often taking several days or weeks. Coordination for pick-up times, addresses, and item verification is done through fragmented channels like WhatsApp messages that can be easily lost or forgotten.

For blood donations, the problem is even more acute because of its time-sensitive nature. A hospital or patient in urgent need of a specific blood type currently relies on scattered social media posts, WhatsApp broadcast lists, or personal networks. There is no dedicated, real-time matching system that can immediately connect a blood request with potential donors in the vicinity.

The result of this fragmentation is that many potential donations never materialize. Donors become frustrated with the logistical complexity and abandon the process, while NGOs miss out on valuable resources they desperately need.

2.1.3 Problem 3: Lack of Donor Engagement and Retention

In the current model, the donor's journey ends immediately after the donation act is completed. There is no mechanism to meaningfully acknowledge contributions or to show donors the cumulative effect of their giving over time. This "donate and forget" model fails to build long-term donor relationships.

Specifically, donors have no way to:

- Track their total donation history across different NGOs and campaigns
- Quantify their overall impact (e.g., "You have helped 150 families" or "Your donations have provided 500 meals")
- Compare their contributions against meaningful milestones
- Receive recognition for sustained giving (e.g., "Thank you for your 10th donation")

The absence of these engagement features means that each donation is treated as an isolated transaction rather than part of a meaningful charitable journey. This reduces intrinsic motivation for sustained giving and makes it difficult for NGOs to convert one-time donors into recurring supporters.

2.1.4 Problem 4: Inability to Handle Diverse Donation Types in a Single Platform

Most existing digital donation platforms are exclusively designed for monetary donations. While cash is certainly valuable, it does not address the full spectrum of charitable needs. NGOs frequently require physical items such as winter clothing before a cold wave, school supplies before the academic year begins, or medical supplies during a health crisis. They also constantly need blood donations for patients.

Currently, an NGO that needs both monetary donations for operational expenses, physical goods for a specific relief effort, and blood donations for a medical camp must manage these three categories through entirely separate channels. They might use a bank transfer platform for money, Facebook posts for physical items, and a separate blood donor app or WhatsApp for blood requests.

This fragmentation creates significant administrative overhead for NGO staff, who must manually reconcile data from multiple sources. It also confuses donors, who must navigate multiple different platforms to support an NGO they trust.

2.1.5 Problem 5: Ineffective Communication for Logistics Coordination

There is no dedicated, real-time communication channel integrated into a donation platform to facilitate the logistical coordination between donors and NGOs. For physical goods donations, details like pickup time, exact address, special handling instructions, and item verification photos must be exchanged. For blood donations, donor availability, medical screening questions, and hospital location require rapid back-and-forth communication.

Currently, this communication happens through general-purpose messaging apps like WhatsApp or through phone calls. These channels are not optimized for donation logistics: messages can be lost in other conversations, there is no structured way to track conversation history for a specific donation, and there is no integration with the donation record itself (e.g., automatically opening a chat when a donor commits to a physical goods donation).

The lack of a dedicated, efficient communication channel leads to misunderstandings, delays, and ultimately, abandoned donations. An NGO might receive ten offers of clothing, but without an efficient way to coordinate pickups, only two or three donors may actually complete the delivery.

2.1.6 Problem 6: Administrative Overhead for Platform Governance

Finally, there is no centralized system where platform administrators can manage the overall health and integrity of the donation ecosystem. An effective platform requires:

- The ability to verify NGO credentials before they are allowed to solicit donations
- The capability to monitor ongoing campaigns for potential misuse
- A mechanism to temporarily lock the platform during critical updates or emergency maintenance
- Tools to view system-wide analytics and detect unusual activity patterns

Existing systems either lack these administrative controls entirely or implement them in an ad-hoc manner that requires direct database access. This makes it difficult to scale the platform beyond a small user base and increases the risk of data corruption during system updates.

2.1.7 Summary Problem Statement

In summary, the core problem is the absence of a trusted**,** integrated**,** multi-functional**,** and engaging digital ecosystem that effectively bridges the gap between the willingness to give and the actual, efficient delivery of resources to those in need. The GiveEase Donation App is proposed as a direct, technology-driven solution to address these six interconnected challenges through a single, unified Android application.

2.2 Deliverables and Development Requirements

To successfully solve the problems identified in Section 2.1, the GiveEase project produced a specific set of tangible deliverables. These deliverables were developed under clearly defined technical, process, and resource constraints that ensured feasibility within the project's scope and timeline.

2.2.1 Software Deliverables

2.2.1.1 Native Android Application

The primary deliverable is a fully functional native Android mobile application packaged as an AAB (Android App Bundle) file suitable for distribution via the Google Play Store. The application implements three distinct, role-based panels that dynamically load based on the authenticated user's

role:

Donor Panel: This interface allows verified donors to browse active NGO campaigns, filter by category (Medical, Education, Disaster Relief, Food, Other), view detailed campaign information including progress bars and NGO profiles, make donations in all three supported types (money via mock payment, physical goods with image upload, blood with location and availability), engage in real-time chat with NGOs for logistics coordination, track personal donation history and impact statistics through a dashboard, and manage their profile including notification preferences.

NGO Panel: This interface allows verified NGOs to create new donation campaigns by specifying title, description, target amount, category, and cover image, manage existing campaigns (pause, resume, or mark as completed), view incoming donations and donor details, respond to donor queries via the integrated real-time chat system, view organizational statistics including total funds raised, active campaign count, and donor demographics, and manage their NGO profile including contact information and verification documents.

Admin Panel: This interface allows system administrators to review and approve or reject NGO verification requests by examining uploaded registration documents, toggle the system-wide "Maintenance Mode" which immediately redirects all non-admin users to a maintenance screen, configure global settings such as the auto-approve toggle for NGO verification, view platform-wide analytics including active user counts and donation volume, and monitor reported issues from donors or NGOs.

2.2.1.2 Complete Firebase Backend Infrastructure

The application is powered by a fully configured serverless backend on Google Firebase, consisting of:

Cloud Firestore Database: Contains collections for users (profiles, roles, verification status), campaigns (requests created by NGOs), donations (records of completed transactions), chat rooms (metadata for conversations), and settings (global configuration flags). Security rules are implemented to enforce role-based access control: for example, only admin users can write to the settings collection, and only the NGO that owns a campaign can edit it.

Firebase Realtime Database: Dedicated exclusively to the chat messaging system, providing sub-100 millisecond latency for message delivery. The data structure uses a nested model where each chat room contains a messages sub-collection.

Firebase Cloud Storage: Stores all user-uploaded media including donor CNIC images, NGO

registration certificates, campaign cover images, donation proof photos for physical goods, and chat images exchanged between users.

Firebase Authentication: Manages user sessions using email and password credentials. The authentication UID serves as the primary key linking to user profiles in Firestore.

Firebase Cloud Functions (Optional): Implements server-side logic for automated reward assignment. When a donation document's status field changes to "COMPLETED," a Cloud Function triggers, calculates the donor's new badge level based on cumulative donations, and updates the user profile accordingly.

2.2.1.3 Complete Project Documentation

The following documentation deliverables accompany the software:

Software Requirements Specification (SRS): Documents all functional and non-functional requirements, use case diagrams and descriptions, and traceability matrices.

Software Design Description (SDD): Documents the system architecture (MVVM), database schema for all Firebase collections, UML diagrams (class, sequence, state transition, ERD), and user interface design specifications.

Final Project Report (This Document): Comprehensive documentation of the entire project lifecycle from problem identification through implementation, testing, and deployment.

2.2.2 Development Requirements

The following technical and process requirements were established before development began to ensure consistency and feasibility.

2.2.2.1 Technical Requirements

Frontend Technology: The application MUST be developed natively for Android using Kotlin (version 1.9.x or higher) as the programming language. XML MUST be used for layout definitions. Use of cross-platform frameworks (React Native, Flutter, Xamarin) is explicitly prohibited to ensure native performance and full access to Android platform features.

Backend Technology: The application MUST exclusively use Google Firebase services as the backend. No custom server-side code (Node.js, Python, PHP, Java Spring, etc.) running on dedicated virtual machines or containers is permitted. This constraint ensures the project remains

within the team's skill set and timeline while leveraging Firebase's paid tier.

Image Handling: The application MUST use the Glide library (version 4.x) for all image loading, caching, and compression operations. This is required to optimize storage usage within Firebase Cloud Storage's the application utilizes paid Google Firebase services for backend infrastructure, including Cloud Storage, Firestore, and Realtime Database. The Blaze plan charges approximately $0.5 per GB for additional storage beyond the included quota and to ensure smooth scrolling performance in image-heavy screens like the campaign feed.

Minimum Operating System: The application MUST target Android 10.0 (API level 29) as the minimum supported version. This ensures compatibility with approximately 85% of active Android devices in Pakistan according to market share data.

2.2.2.2 Process Requirements

Development Methodology: The project MUST follow Agile Scrum methodology with two-week sprints. Each sprint MUST conclude with a working, testable increment of the application. Daily stand-up meetings MUST be conducted to track progress and identify blockers.

Version Control: All source code, Firebase security rules, and documentation MUST be managed using Git. A GitHub repository MUST be maintained with regular commits and meaningful commit messages. Feature branching MUST be used for developing new modules, with pull requests for code review before merging into the main branch.

Testing: All core functionalities defined in the SRS MUST be tested. A structured testing strategy MUST be documented, including unit tests for business logic (e.g., campaign progress calculation), integration tests for Firebase operations, and manual functional tests for user interfaces.

2.2.3 Resource and Budgetary Constraints

The following constraints limit the scale and scope of the project while ensuring it remains achievable by a student team.

Team Size: The project is developed by a team of exactly three undergraduate students from COMSATS University Islamabad, Sahiwal Campus. Each team member is responsible for specific modules as defined in the work division plan.

Monetary Budget: The project is developed with a minimal monetary budget. All software tools (Android Studio, Git, Figma) are used under their free licenses. For cloud services, the project utilizes Google Firebase under the Blaze (pay-as-you-go) plan, which includes a generous free

quota: 1GB of Firestore storage, 5GB of Cloud Storage, and 50,000 daily reads/writes included at no cost. The team only pays for usage beyond these limits, which is approximately $0.5 per GB for additional Firestore storage. For a demonstration-scale application with approximately 100-200 test users, the usage remains well within the free quota, resulting in effectively zero additional cloud costs. Any costs incurred for Google Play Store deployment (one-time fee of approximately $25) are to be borne personally by the team members and are not considered project development costs.

Timeline: The project is to be completed within a 6 to 8-month timeline. Development began in June 2025 and is scheduled for final submission and defense by April 2026. This timeline includes all phases: requirement gathering, design, implementation, testing, documentation, and deployment preparation.

2.3 Current System

There is no single, integrated "current system" that performs all the functions of GiveEase. Instead, the process of donating to charitable causes in Pakistan is a fragmented, multi-channel "ecosystem" of manual and semi-digital methods. Analyzing this existing ecosystem reveals the specific inefficiencies that GiveEase aims to replace.

2.3.1 Informal and Community-Based Channels

This is the most widespread method of charitable giving in Pakistan, particularly in rural and semi-urban areas.

Mosques and Religious Institutions: During religious occasions such as Ramadan, Eid, and Muharram, mosques collect cash and food donations from community members. The mosque committee or local imam determines the distribution of these funds, typically to impoverished families in the immediate vicinity. While this method is convenient and trusted at the local level, it suffers from several problems. There is no transparency for the donor regarding how much was collected and how it was distributed. There is no capability for donating specific non-cash items beyond basic food staples. The method is geographically limited to the mosque's surrounding area.

Community Drives: Schools, universities, and social clubs occasionally organize donation drives for specific causes such as flood relief or winter clothing distribution. These events are typically announced through posters, social media posts, or word of mouth. Donors drop off items at a designated collection point. The problems with this method include poor publicity, lack of real-time information on which items are most needed, no tracking mechanism for collected quantities, and

no follow-up with donors about how their items were used.

Direct NGO Contact: A motivated donor may actively search for an NGO's contact information through its website or social media page. The donor then calls or messages the NGO to ask what items are currently needed, receives a response, arranges for delivery or pickup, and finally completes the donation. The problems are numerous: the process is time-consuming, requires multiple phone calls, has no guarantee that the stated needs are still current (the NGO may have already received sufficient quantities), and provides no structured way to schedule logistics.

2.3.2 Social Media Platforms

NGOs and individuals frequently use social media platforms to solicit donations, especially for urgent needs.

Facebook: NGOs create public posts describing their needs, sometimes with photos. These posts can be shared to increase reach. Donors comment on the post to express interest or send private messages. The problems include unstructured information that cannot be easily filtered or searched, posts quickly become outdated as new content is published, there is no way to know if a request has already been fulfilled, and there is no verification mechanism to confirm the posting organization is legitimate.

WhatsApp: Broadcast lists and group chats are heavily used for donation requests, particularly for urgent needs like blood donations. A typical request message says something like: "URGENT: A+ blood needed for patient at City Hospital, please contact this number." The problems include messages being lost among other conversations, no ability to mark a request as fulfilled leading to duplicate responses, no donor verification, and no structured way to match donors with requests in real time.

2.3.3 Single-Purpose Digital Tools

Some digital tools exist, but each addresses only a narrow subset of the donation problem.

Mobile Wallet Apps (JazzCash, EasyPaisa): These financial platforms allow users to send money to registered NGO accounts. This provides a convenient way to donate cash. However, these apps do not allow NGOs to solicit physical goods or blood. They provide no donor engagement features such as impact tracking or rewards. They offer no verification mechanism beyond the NGO's bank account being active.

Independent Blood Donation Apps: Several localized blood donation apps and websites exist in

Pakistan, attempting to maintain donor registries. However, these platforms suffer from outdated contact information, lack of real-time request management, poor user interface design, and low adoption rates. Many become inactive after a short period due to lack of maintenance.

NGO-Specific Websites: Larger, well-funded NGOs maintain websites with "Donate Now" buttons for monetary contributions and "Get Involved" pages listing items they accept. However, this requires donors to know about and navigate to each NGO's website individually. There is no centralized platform for browsing multiple NGOs. The information is often static and not updated in real time.

2.3.4 Summary of Current System Inefficiencies

The following table summarizes the key inefficiencies of the current fragmented ecosystem (presented in paragraph form to avoid table formatting issues):

In terms of **trust and verification**, the current system offers no standardized mechanism. Donors must rely on reputation, word of mouth, or minimal checks. Verification is non-existent on social media platforms and weak even on NGO websites. This lack of verification directly enables fraudulent campaigns.

In terms of **donation type support**, the current system is fragmented. Money is handled by bank transfer or mobile wallet apps. Physical goods rely on manual coordination via social media or phone calls. Blood donations use separate specialized apps or social media broadcasts. No single channel supports all three types.

In terms of **request management**, the current system provides no lifecycle tracking. A request for items is posted, but there is no way to mark it as fulfilled, paused, or completed. Donors cannot filter by urgency or category. NGOs cannot easily manage multiple active requests simultaneously.

In terms of **logistics coordination**, the current system relies on general-purpose messaging apps or phone calls. Communication is unstructured, messages can be lost, and there is no integration with the donation record. Coordination for pickup times, addresses, and verification is manual and error-prone.

In terms of **donor engagement**, the current system effectively ends at the moment of donation. Donors receive a receipt (for monetary donations) or a thank-you message (for physical goods), but there is no ongoing relationship. No impact tracking, no recognition for sustained giving, and no gamification elements exist in any current system.

In terms of **platform governance**, the current system provides no centralized administrative controls. NGOs self-report their status. There is no way for a platform operator to lock down the system during maintenance or to globally enforce verification standards.

2.3.5 The Opportunity for GiveEase

The analysis of the current fragmented ecosystem reveals a clear opportunity for a centralized, integrated solution. GiveEase is designed as a modern, systematic alternative that replaces the ad-hoc workflows described above with a single, trusted, efficient, and engaging mobile platform. By addressing all six identified problems through a unified interface, GiveEase aims to increase donation volume, reduce logistical friction, build donor trust, and create sustained engagement between donors and verified NGOs.

CHAPTER 3

Requirement Analysis

3. REQUIREMENT ANALYSIS

Requirement analysis is a foundational phase of any software development lifecycle, serving as the backbone for system design, architecture, and implementation. For a donation platform like GiveEase, where trust, transparency, and real-time communication are critical, requirement analysis becomes even more essential. This chapter outlines all the functional and non-functional requirements, use cases, actors, and a detailed breakdown of system behavior. The purpose is to clearly define what the system must do and how well it should perform, ensuring the application meets user expectations for reliability, security, and usability.

The requirement analysis for GiveEase was conducted using multiple techniques to ensure comprehensive coverage of stakeholder needs. First, interviews were conducted with representatives from three local NGOs operating in Sahiwal and surrounding areas. These interviews focused on understanding their current donation management processes, pain points, and desired features. Second, questionnaires were distributed to 50 potential donors through university networks and social media groups, gathering preferences on donation types, engagement features, and usability expectations. Third, a competitive analysis was performed on existing platforms including GoFundMe, DonorsChoose, and local blood donation apps to identify gaps and opportunities. Finally, the project supervisor provided iterative feedback throughout the requirement gathering phase, ensuring alignment with academic expectations and project scope constraints.

The actors in the GiveEase system are three distinct user types, each with specific roles and permissions. The Donor is an individual who registers using their CNIC, browses active donation campaigns, makes donations (money, physical items, or blood), communicates with NGOs via real-time chat, tracks their personal impact through a dashboard, and manages their profile. The NGO is a verified organization that registers using official registration documents, creates and manages donation campaigns with specific item requirements, receives and responds to donor communications, views organizational statistics, and updates campaign statuses. The Admin is the system moderator responsible for verifying NGO registration documents, approving or rejecting verification requests, toggling system-wide settings such as maintenance mode, and monitoring platform activity through analytics.

3.1 Use Case Diagrams

The use case diagrams provide a visual representation of the interactions between actors (Donor, NGO, Admin) and the GiveEase system. These diagrams illustrate the high-level functionality of the system from the perspective of each user role, showing the major use cases and their relationships. The diagrams help stakeholders understand the scope of the system and identify which features are accessible to which user types.

The following use case diagrams are created for GiveEase:

3.1.1 Main System Use Case Diagram

This diagram presents the overall system from a high-level perspective, showing all three actors (Donor, NGO, and Admin) and their interactions with the core system functionalities. It illustrates the basic relationships and dependencies between different user roles within the donation ecosystem, defining the scope of the system's operations and the capabilities of users across all modules. The main use case diagram helps establish a shared understanding of the system's boundaries and primary functions among all stakeholders.

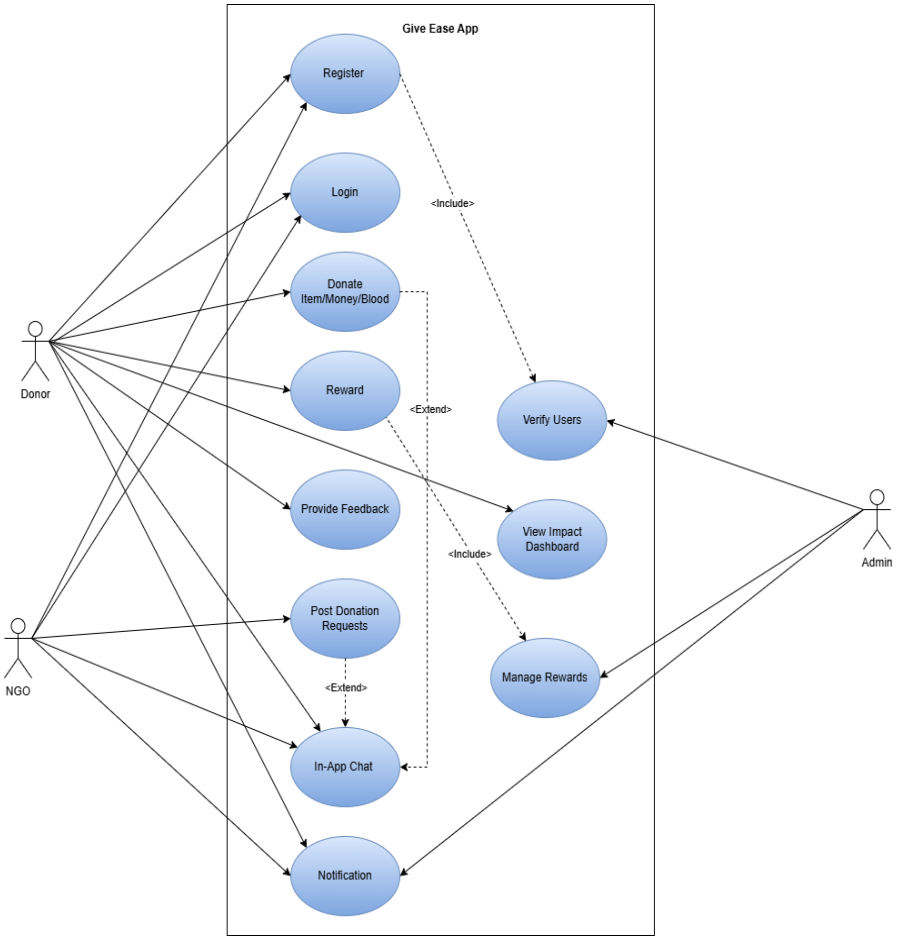


Figure 3.1: Main System Use Case Diagram

3.1.2 Donor-Specific Use Case Diagram

This diagram exhibits all the details and functionalities offered specifically to donors. It includes use cases such as registering and logging into the application, browsing donation requests with filtering options, making donations in all three types (money, physical items, blood), communicating with NGOs via real-time chat, tracking personal donation history, viewing the impact dashboard with earned badges, and managing their profile. This diagram maps the complete donor journey from initial sign-up through post-donation engagement, demonstrating how donors can interact with the system to meet their charitable goals and maintain sustained participation.

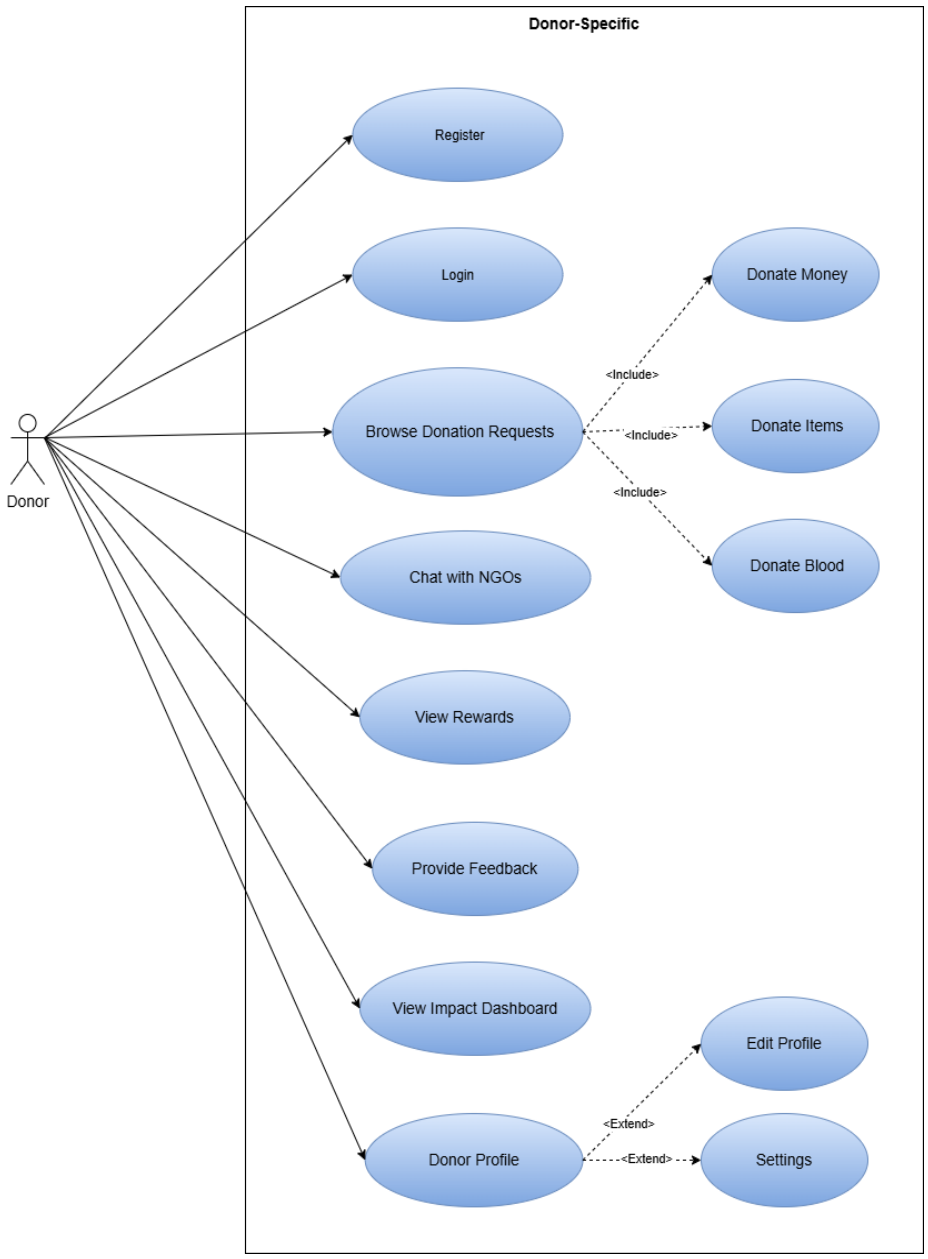


Figure 3.2: Donor-Specific Use Case Diagram

3.1.3 NGO-Specific Use Case Diagram

This diagram depicts the functionalities available to NGOs. It includes use cases such as registering with organization details and uploading registration documents, creating and publishing new donation campaigns with specific item requirements, editing or pausing existing campaigns, viewing incoming donations and donor details, responding to donor queries via the real-time chat system, viewing organizational statistics including total funds raised and active campaign counts, and managing their NGO profile. This diagram shows how effectively NGOs can manage donation drives, communicate with donors, and provide transparency through various aspects of the platform.

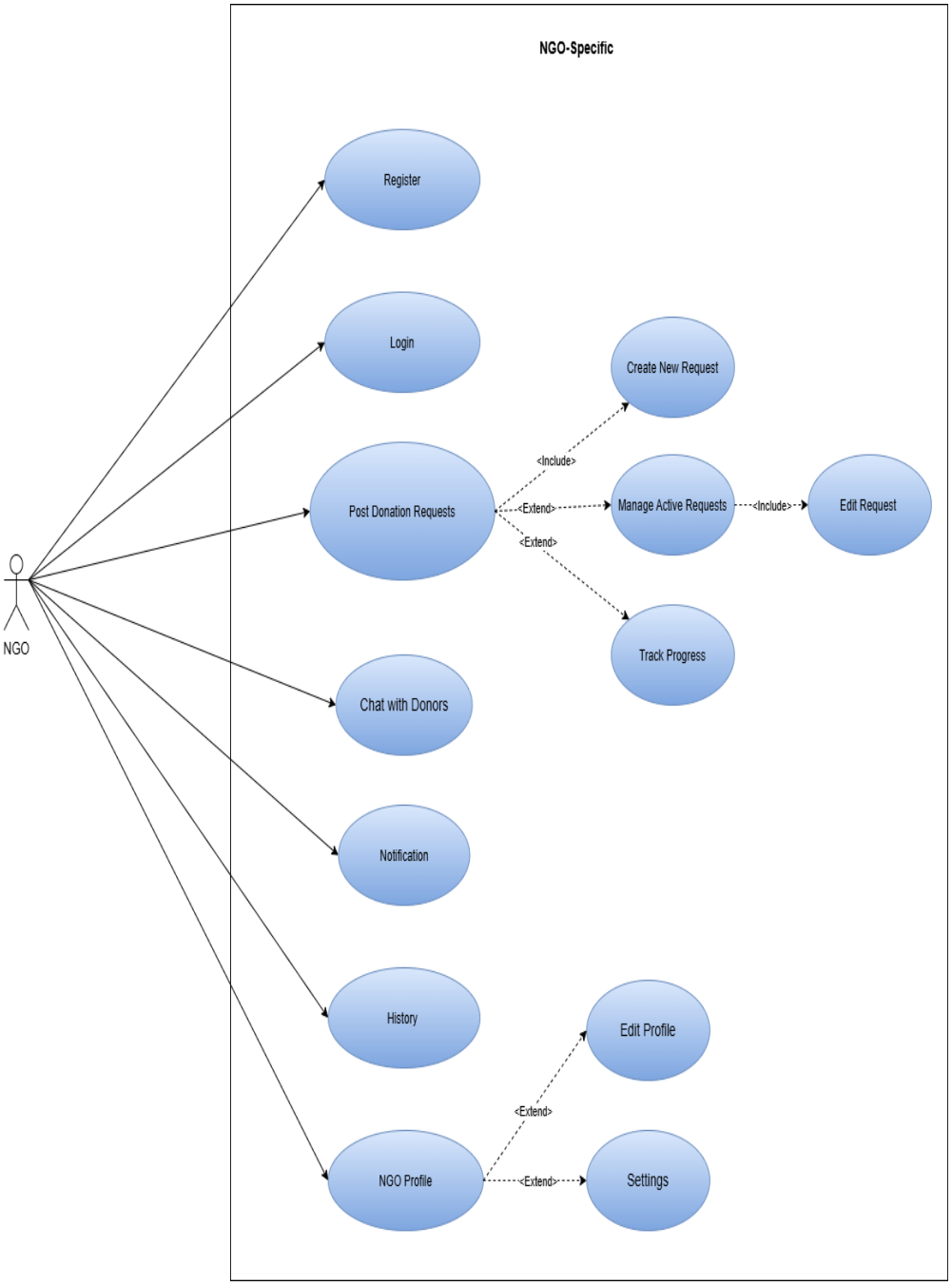


Figure 3.3: NGO-Specific Use Case Diagram

3.1.4 Admin-Specific Use Case Diagram

This diagram shows the administrator's roles and responsibilities. It includes use cases such as viewing pending NGO verification requests, reviewing uploaded registration documents, approving or rejecting verification requests, toggling the system-wide maintenance mode which immediately redirects all non-admin users to a maintenance screen, configuring global settings such as the auto-approve toggle for NGO verification, viewing platform-wide analytics including active user counts and donation volume, managing the reward system (badges and certificates), and monitoring reported issues from donors or NGOs.

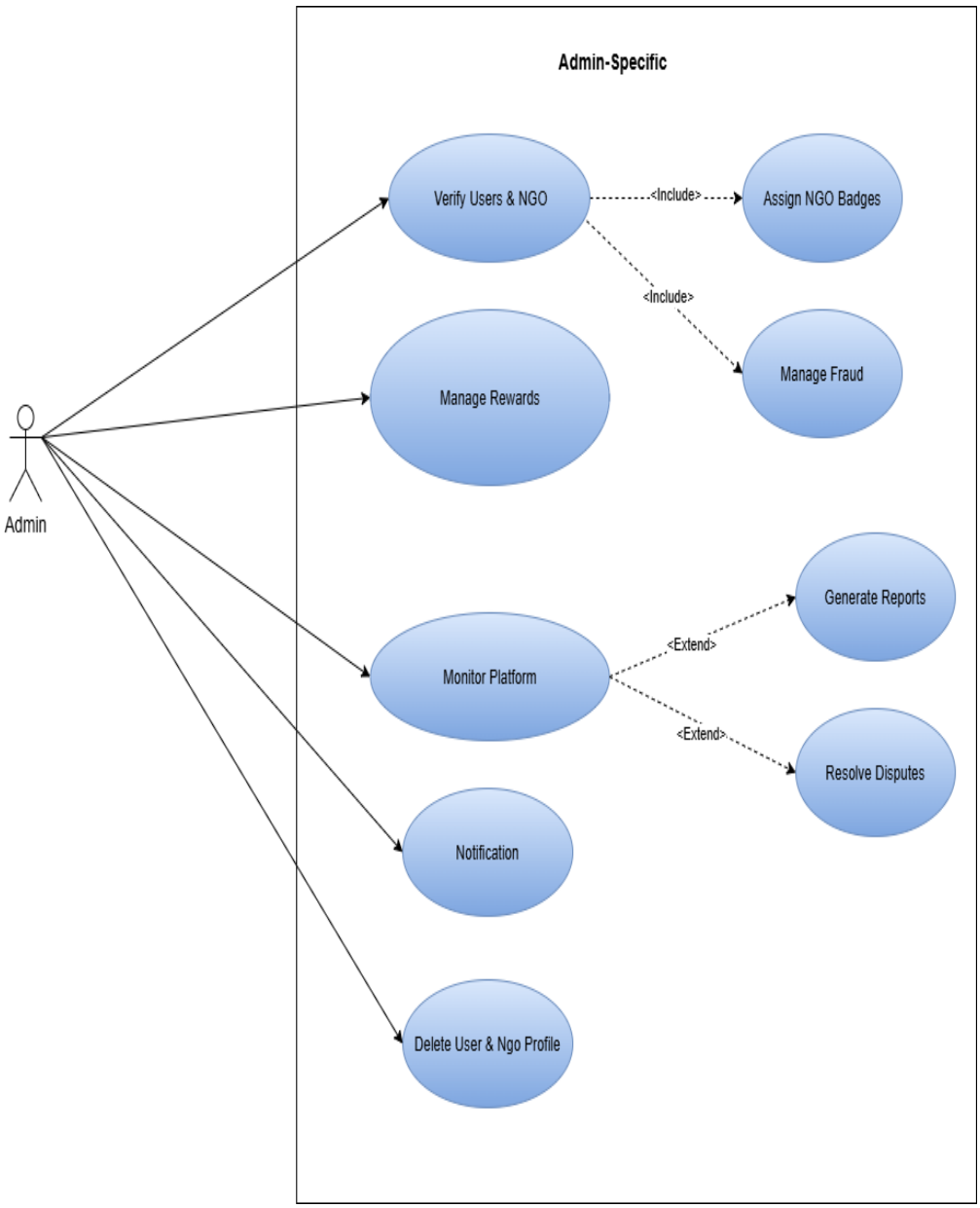


Figure 3.4: Admin-Specific Use Case Diagram

3.1.5 Communication System Use Case Diagram

This diagram is specific to the real-time interaction mechanisms available to donors and NGOs alike, including the chat feature, notification systems, and coordination tools. It shows how the platform handles logistical smoothness of donations and coordinates relations by providing interaction possibilities that are completely seamless, enabling both textual and visual means of relaying information. The communication system includes sending and receiving text messages, sharing images within chat, receiving push notifications for new messages and donation updates, and viewing chat history. This diagram highlights the real-time, bidirectional nature of donor-NGO communication that distinguishes GiveEase from traditional donation platforms.

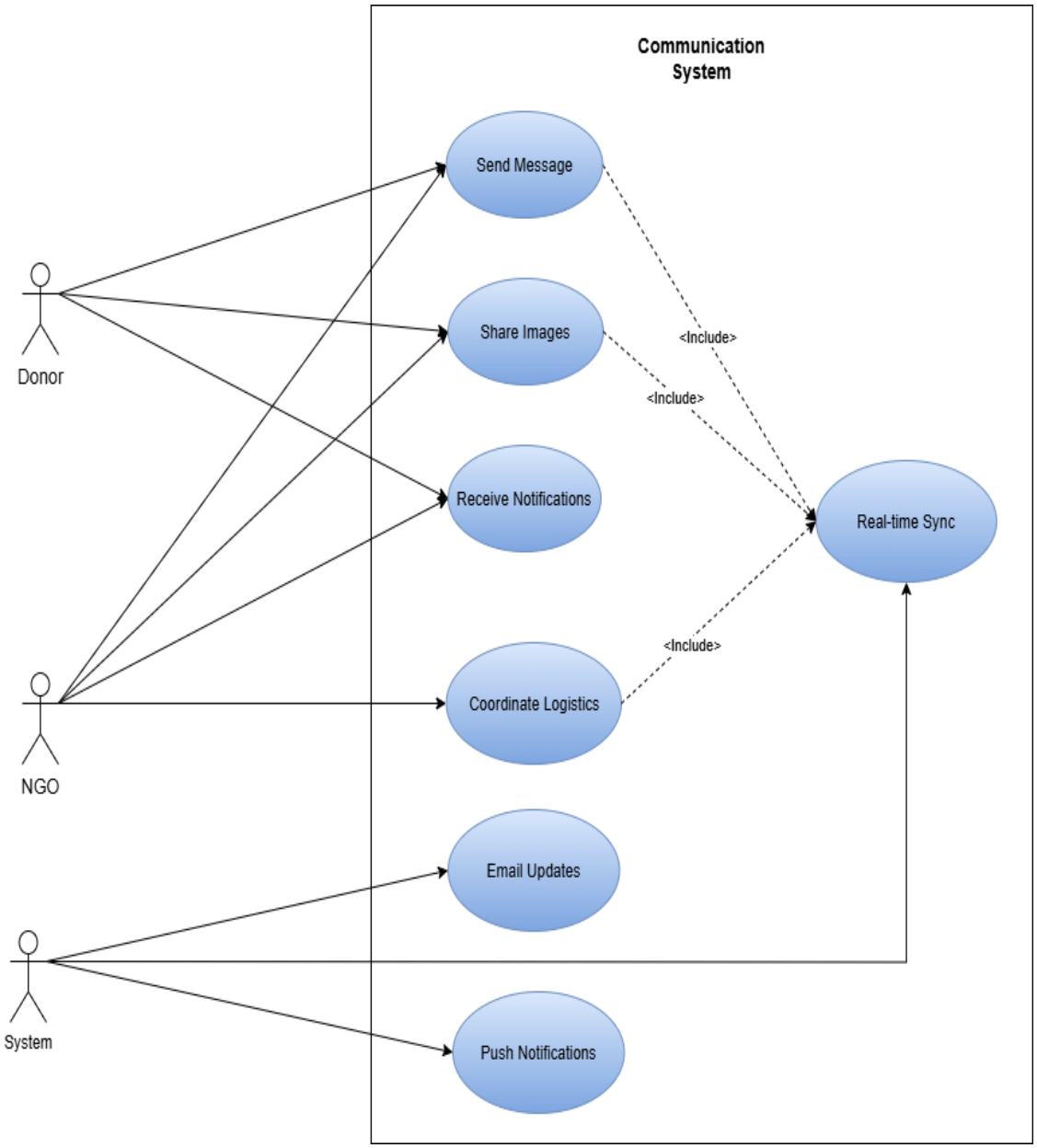


Figure 3.5: Communication System Use Case Diagram

3.2 Use Case Description

Use case descriptions provide a detailed narrative of how the system interacts with various actors to accomplish specific tasks or goals. Each use case outlines the interactions between donors, NGOs, administrators, and the GiveEase system, capturing the functional requirements of the platform.

3.2.1 User Registration and Authentication

This use case allows new users to create an account by providing email, password, and basic profile information, or allows existing users to log into the system securely. Upon successful login, the user is redirected to the role-appropriate dashboard based on their assigned role (Donor, NGO, or Admin).

Table 3.2.1: User Registration and Authentication

Element	**Description**
Use Case ID	UC-01
Actors	Donor, NGO
Pre-Conditions	User has valid email address and internet connection. App installed on Android 10+.
Post-Conditions	New user document created in Firestore with unverified status. User redirected to role-based dashboard.
Normal Flow	1. User selects Login or Sign Up. 2. For sign-up, user enters name, email, password, role. 3. System validates email not registered and password strength. 4. Firebase creates account. 5. Firestore user document created. 6. User prompted to upload verification documents.
Alternative Flows	A1: Email already registered → error message, prompt login. A2: Weak password → reject, request stronger password. A3: Forgot password → send reset email via Firebase.

3.2.2 Browsing and Filtering Donation Campaigns

This use case allows a verified donor to view a list of active donation campaigns posted by verified NGOs. The donor can filter campaigns by category (Medical, Education, Disaster Relief, Food, Other) or by urgency level (Low, Medium, High, Urgent). The system displays each campaign as a card showing title, NGO name with verified badge, progress bar, and days remaining.

Table 3.2.2: Browsing and Filtering Donation Campaigns

Element	**Description**
Use Case ID	UC-02
Actors	Donor
Pre-Conditions	Donor logged in and verified. Active campaigns exist in Firestore.
Post-Conditions	Donor sees filtered or unfiltered list of campaigns with progress bars.
Normal Flow	1. Donor directed to DonorHomeFragment. 2. System queries active campaigns from Firestore. 3. CampaignAdapter displays cards with title, NGO name, progress bar. 4. Donor taps filter icon. 5. Donor selects category or urgency. 6. System re-queries with filters. 7. Updated list displayed.
Alternative Flows	A1: No campaigns match filters → display "No campaigns found" message. A2: No internet → display cached campaigns with warning banner.

3.2.3 Making a Donation (Money, Physical Items, or Blood)

This use case allows a verified donor to contribute to a selected campaign by choosing one of three donation types. For monetary donations, the donor enters an amount and confirms a mock payment. For physical items, the donor selects item category, enters quantity, and uploads photos. For blood donations, the donor enters blood type, availability date, and location. The system creates a donation record and updates the campaign's raised amount.

Table 3.2.3: Making a Donation (Money, Physical Items, or Blood)

Element	**Description**
Use Case ID	UC-03
Actors	Donor, NGO (secondary)
Pre-Conditions	Donor logged in and verified. Campaign status is active.
Post-Conditions	Donation document created. Campaign raisedAmount updated. Chat room created for physical/blood.
Normal Flow	1. Donor taps "Donate Now" on campaign. 2. System shows three donation type options. 3. Donor selects type and enters details. 4. Donor uploads photos if physical items. 5. Donor confirms donation. 6. System creates donation document. 7. System updates campaign raisedAmount. 8. Chat room created. 9. Push notification sent to NGO.
Alternative Flows	A1: Amount exceeds remaining target → error, suggest lower amount. A2: Invalid image format → reject, prompt valid format. A3: No internet → queue donation locally, sync when online.

3.2.4 NGO Campaign Creation and Management

This use case allows a verified NGO to create a new donation campaign by specifying title, description, target amount, category, urgency level, and cover image. The NGO can also edit, pause, or complete existing campaigns from their dashboard. When a campaign is published, it immediately appears in the donor feed. If paused or completed, it no longer appears in active donor feeds.

Table 3.2.4: NGO Campaign Creation and Management

Element	**Description**
Use Case ID	UC-04
Actors	NGO
Pre-Conditions	NGO logged in and verified.
Post-Conditions	New campaign document created or existing campaign status updated.
Normal Flow	1. NGO taps "Create Campaign". 2. NGO enters title, description, target, category, urgency. 3. NGO uploads cover image. 4. NGO taps "Publish". 5. System validates inputs. 6. System creates campaign document with status active. 7. Campaign appears in donor feed. 8. NGO can edit, pause, or complete from dashboard.
Alternative Flows	A1: NGO not verified → campaign visible only to admins for review. A2: Editing campaign with donations → target cannot be reduced below raised amount.

3.2.5 Real-Time Chat Between Donor and NGO

This use case allows a donor and an NGO to communicate in real-time through text messages and image sharing to coordinate donation logistics. Messages are stored in Firebase Realtime Database for sub-second delivery. When a user sends an image, it is uploaded to Cloud Storage and the download URL is sent as a message. Push notifications are sent when the recipient is not viewing the chat.

Table 3.2.5: Real-Time Chat Between Donor and NGO

Element	**Description**
Use Case ID	UC-05
Actors	Donor, NGO
Pre-Conditions	Donation initiated between donor and NGO. Both users logged in.
Post-Conditions	Messages stored in Realtime Database. Recipient receives notification.
Normal Flow	1. User opens chat from donation screen. 2. System creates or retrieves chat room. 3. User types message and taps send. 4. System writes message to Realtime Database. 5. System updates lastMessage field. 6. Recipient sees message instantly. 7. User can attach images from camera or gallery.
Alternative Flows	A1: Recipient offline → message stored, delivered when online. A2: Blank message → send button disabled. A3: No active donation → chat prevented, message shown.

3.2.6 Admin Verification of NGO Accounts

This use case allows the system administrator to review registration documents submitted by NGOs, verify their authenticity, and approve or reject their verification requests. When approved, the NGO's isVerified field is set to true and a verified badge appears on their profile and campaigns. When rejected, the NGO receives a notification explaining the reason and can resubmit documents.

Table 3.2.6: Admin Verification of NGO Accounts

Element	**Description**
Use Case ID	UC-06
Actors	Admin, NGO (secondary)
Pre-Conditions	Admin logged in. NGO has submitted verification documents.
Post-Conditions	NGO isVerified flag updated. Notification sent to NGO.
Normal Flow	1. Admin navigates to Verification Requests. 2. System lists pending NGOs. 3. Admin taps NGO to view details. 4. Admin reviews uploaded documents. 5. Admin taps Approve or Reject. 6. System updates isVerified field. 7. System sends push notification to NGO. 8. Log entry created for audit.
Alternative Flows	A1: Auto-Approve enabled → NGOs verified automatically. A2: Rejected NGO resubmits → new request appears in queue. A3: Admin needs more info → mark as Pending Additional Info.

3.2.7 Admin Maintenance Mode Toggle

This use case allows the system administrator to enable or disable a system-wide maintenance mode. When enabled, the system updates a flag in Firestore. All active donor and NGO sessions have a listener on this document and are immediately redirected to a maintenance screen. This prevents data corruption during backend migrations or emergency updates. Only admin users can access the app when maintenance mode is active.

Table 3.2.7: Admin Maintenance Mode Toggle

Element	**Description**
Use Case ID	UC-07
Actors	Admin
Pre-Conditions	Admin logged in.
Post-Conditions	Maintenance flag set in Firestore. Non-admin users redirected to maintenance screen.
Normal Flow	1. Admin navigates to AdminSettingsFragment. 2. Admin taps Maintenance Mode toggle. 3. Confirmation dialog appears. 4. Admin confirms. 5. System updates Firestore maintenance isActive=true. 6. Active user sessions detect change. 7. Non-admin users redirected to MaintenanceActivity. 8. Admin performs updates. 9. Admin toggles off when complete.
Alternative Flows	A1: Admin forgets to disable → users remain locked out until manual disable. A2: Admin loses internet → flag active but users may not receive update until connectivity restored.

3.2.8 Donor Viewing Impact Dashboard

This use case allows a verified donor to view a personalized dashboard showing their donation history, total impact statistics, earned badges, and progress toward the next badge milestone. The system aggregates data from completed donations and calculates impact points using a weighted formula. When a donor earns a new badge, a celebratory animation plays and a notification appears.

Table 3.2.8: Donor Viewing Impact Dashboard

Element	**Description**
Use Case ID	UC-08
Actors	Donor
Pre-Conditions	Donor logged in and verified.
Post-Conditions	Dashboard displays aggregated donation statistics. No data modified.
Normal Flow	1. Donor navigates to Impact Dashboard tab. 2. System queries completed donations for donor. 3. System aggregates total money, items, blood donations. 4. System queries earned badges from rewards collection. 5. System calculates progress to next badge. 6. Dashboard displays stats, badges, progress bar. 7. Donor can tap "Share Your Impact" button.
Alternative Flows	A1: No completed donations → display friendly message encouraging donations. A2: New badge earned → celebratory animation plays.

3.3 Functional Requirements

Functional requirements define what the GiveEase system must do. These requirements are derived from the use cases described in Section 3.2, stakeholder interviews, and competitive analysis. Each functional requirement is assigned a unique identifier (FR-XX) and categorized by module.

3.3.1 Authentication Module

FR-01: The system must allow new users to register using email address and password as the primary authentication method.

FR-02: The system must allow registered users to log in securely using their email and password credentials.

FR-03: The system must provide a password recovery mechanism that sends a password reset link via Firebase Authentication.

FR-04: The system must validate that the email is not already registered and password meets minimum security requirements (at least 6 characters).

FR-05: The system must assign a role (donor, ngo) at registration, determining which dashboard and features are accessible.

3.3.2 Donor Panel Module

FR-06: The system must display active donation campaigns with title, NGO name, verified badge, target amount, raised amount with progress bar, urgency level, and days remaining.

FR-07: The system must allow donors to filter campaigns by category (Medical, Education, Disaster Relief, Food, Other) and by urgency level.

FR-08: The system must allow donors to search for campaigns by title keyword using a search bar.

FR-09: The system must allow donors to view detailed campaign information including full description and NGO profile.

FR-10: The system must support three donation types: monetary (mock payment), physical items (with image upload), and blood donation (with blood type, date, location).

FR-11: For physical donations, the system must allow photo uploads from camera or gallery, compress images using Glide, and store in Cloud Storage.

FR-12: For monetary donations, the system must simulate mock payment and update campaign

raisedAmount in real-time.

FR-13: The system must automatically create a chat room between donor and NGO when physical or blood donation is initiated.

FR-14: The system must allow donors to view complete donation history with date, amount, campaign name, and status.

FR-15: The system must display impact dashboard showing total impact points, earned badges, progress to next badge, and donation charts.

3.3.3 NGO Panel Module

FR-16: The system must allow verified NGOs to create campaigns with title, description, target amount, category, urgency level, and cover image.

FR-17: The system must validate that required fields are not empty and target amount is positive.

FR-18: The system must allow NGOs to edit existing campaigns, with target amount constrained by already raised amount.

FR-19: The system must allow NGOs to pause active campaigns, removing them from donor feed.

FR-20: The system must allow NGOs to mark campaigns as completed, preventing further donations.

FR-21: The system must display NGO statistics including total funds raised, active campaigns, donor count, and recent activity.

FR-22: The system must allow NGOs to view and respond to messages in the real-time chat interface.

3.3.4 Admin Panel Module

FR-23: The system must allow admins to view pending NGO verification requests with submitted documents.

FR-24: The system must allow admins to approve or reject verification requests, updating isVerified field.

FR-25: The system must send push notifications to NGOs when verification is approved or rejected.

FR-26: The system must provide a toggle switch to enable or disable system-wide maintenance mode.

FR-27: The system must listen for real-time changes to maintenance mode flag and redirect users immediately.

FR-28: The system must display platform analytics including total users, active campaigns, donations completed, and impact points awarded.

3.3.5 Real-Time Chat Module

FR-29: The system must provide real-time one-to-one text messaging using Firebase Realtime Database.

FR-30: The system must allow users to send images within chat, uploading to Cloud Storage and sending download URL.

FR-31: The system must compress images before upload using Glide to reduce storage usage.

FR-32: The system must display last message and timestamp in chat list view.

FR-33: The system must send push notifications for new messages when recipient is not viewing the chat.

FR-34: The system must create chat rooms using composite key (donorId_ngoId_requestId) for uniqueness.

3.3.6 Reward System Module

FR-35: The system must automatically assign badges at milestones: Bronze (1), Silver (5), Gold (10), Platinum (20 donations).

FR-36: The system must calculate impact points: monetary (1 point per 100 PKR), physical (20 points per item), blood (50 points per donation).

FR-37: The system must store earned badges in user document and maintain rewards collection for history.

FR-38: The system must display progress bar showing donations needed for next badge.

FR-39: The system should generate downloadable PDF certificates for significant milestones stored in Cloud Storage.

3.3.7 Notification Module

FR-40: The system must send push notifications via FCM for new campaigns, donations received,

verification updates, new messages, and badges earned.

FR-41: The system must store in-app notifications in Firestore for notification history.

FR-42: The system must mark notifications as read when viewed by user.

FR-43: The system must allow users to configure notification preferences from settings screen.

3.4 Non-Functional Requirements

Non-functional requirements define how well the GiveEase system should perform. These requirements are derived from stakeholder expectations, technical constraints, and industry best practices.

3.4.1 Performance Requirements

NFR-01: The donor campaign feed must load within 3 seconds over 4G network.

NFR-02: Real-time chat messages must deliver within 1 second under normal network conditions.

NFR-03: Image uploads must complete within 5 seconds for 1MB image after compression over 4G.

NFR-04: The application must maintain 60 FPS scrolling in campaign feed with asynchronous image loading.

NFR-05: Impact dashboard queries must complete within 2 seconds even for donors with 100+ donation records.

NFR-06: Application cold start time must not exceed 4 seconds on mid-range Android devices.

3.4.2 Security Requirements

NFR-07: All communication between app and Firebase must be encrypted using TLS/HTTPS.

NFR-08: Passwords must be hashed by Firebase Auth; app must never store passwords locally.

NFR-09: Firebase Security Rules must enforce users can only read and write their own data.

NFR-10: Role-based access control must prevent donors from accessing admin or NGO features.

NFR-11: Uploaded verification documents must have security rules allowing only user and admins to read.

NFR-12: The system must log all admin actions for audit.

3.4.3 Usability Requirements

NFR-13: User interface must follow Material Design 3 guidelines for Android consistency.

NFR-14: Core actions (donate, create campaign, send message) must be accessible within three taps.

NFR-15: All forms must include input validation with clear, user-friendly error messages.

NFR-16: The application must support TalkBack screen reader with content descriptions for all interactive elements.

NFR-17: Touch targets must have minimum size of 48dp for users with motor difficulties.

NFR-18: NGO verification badge must be clearly visible on campaign cards and profiles.

3.4.4 Reliability Requirements

NFR-19: Application must maintain stable operation without crashes for 8 hours of continuous simulated use.

NFR-20: When network is lost, app must cache write operations (donations, messages) for sync when connectivity returns.

NFR-21: App must not lose user data during unexpected crashes; pending writes must be persisted locally.

NFR-22: Firebase services expected 99.9% uptime; app must handle transient errors with retry logic.

NFR-23: System must prevent duplicate donations when user taps confirm button multiple times.

3.4.5 Maintainability Requirements

NFR-24: Codebase must follow MVVM architectural pattern for separation of concerns.

NFR-25: All resources (strings, colors, dimensions) must be defined in resource files, not hardcoded.

NFR-26: Firebase Security Rules must be documented and version-controlled with application code.

NFR-27: Reward system thresholds should be configurable through settings collection rather than hardcoded.

3.4.6 Scalability Requirements

NFR-28: Firestore design must support pagination for large result sets like campaign feed and donation history.

NFR-29: Application must support at least 1,000 concurrent users within Firebase free tier limits.

NFR-30: Cloud Storage usage must be optimized through image compression to stay within the 5GB free quota included in the Firebase Blaze plan, avoiding unnecessary overage charges.

3.4.7 Portability and Compatibility Requirements

NFR-31: Application must target Android 10.0 (API level 29) as minimum supported version.

NFR-32: Application must support both smartphones and tablets with adaptive layouts.

NFR-33: Application must handle devices with minimum 1GB free storage and 2GB RAM.

NFR-34: Application must request camera and storage permissions only when needed with clear explanations.

CHAPTER 4
Design and Architecture

4. DESIGN AND ARCHITECTURE

Design and architecture form the backbone of any software system, translating the requirements identified in the previous chapter into a concrete blueprint for implementation. For GiveEase, a donation platform that requires real-time communication, role-based access control, and secure data handling, a well-defined architecture is essential to ensure reliability, scalability, and maintainability. This chapter presents the complete system architecture, data representation models, process flows, and design models that guide the implementation of the GiveEase Donation App.

The GiveEase application follows a client-server architecture with Firebase as the backend platform. The frontend is a native Android application developed in Kotlin, following the MVVM (Model-View-ViewModel) architectural pattern. The backend utilizes Google Firebase services including Firestore for structured data, Realtime Database for chat messaging, Cloud Storage for file storage, Authentication for user management, and Cloud Messaging for push notifications. This serverless architecture eliminates the need for dedicated server management while providing real-time data synchronization across all user devices.

The design models presented in this chapter include the system architecture diagram showing component interactions, the entity relationship diagram for database collections, process flow diagrams for key user journeys, sequence diagrams for major operations, and a comprehensive class diagram showing the object-oriented structure of the application. Each diagram is accompanied by a detailed description explaining its elements and their relationships.

4.1 System Architecture

The system architecture of GiveEase follows a client-server design with Google Firebase serving as the complete serverless backend. The architecture is divided into two main components: the Android Client on the frontend and the Google Firebase Backend services on the cloud.

Android Client: The frontend is a single native Android APK developed in Kotlin. This single application dynamically presents three distinct user interfaces based on the authenticated user's role. The Donor Panel allows individual donors to browse campaigns, make donations, and track their impact. The NGO Panel enables verified organizations to create and manage donation campaigns, view incoming donations, and communicate with donors. The Admin Panel provides system administrators with tools to verify NGOs, and monitor overall activity.

Communication Layer: The Android Client communicates with the Google Firebase Backend over the internet using secure HTTPS connections. All data exchange between the client and cloud

services is encrypted in transit.

Google Firebase Serverless Backend: The backend consists of five integrated Firebase services. Cloud Firestore serves as the primary structured database, storing users, campaigns, donations, and rewards documents. Firebase Auth handles all user authentication including registration, login, and password reset functionality.

Cloud Functions: Firebase Cloud Functions act as the automation layer. These server-side functions are triggered by specific database events. The primary trigger is when a donation status changes to "COMPLETED". Upon this trigger, Cloud Functions automatically execute the reward assignment logic, updating the donor's impact points and badge status.

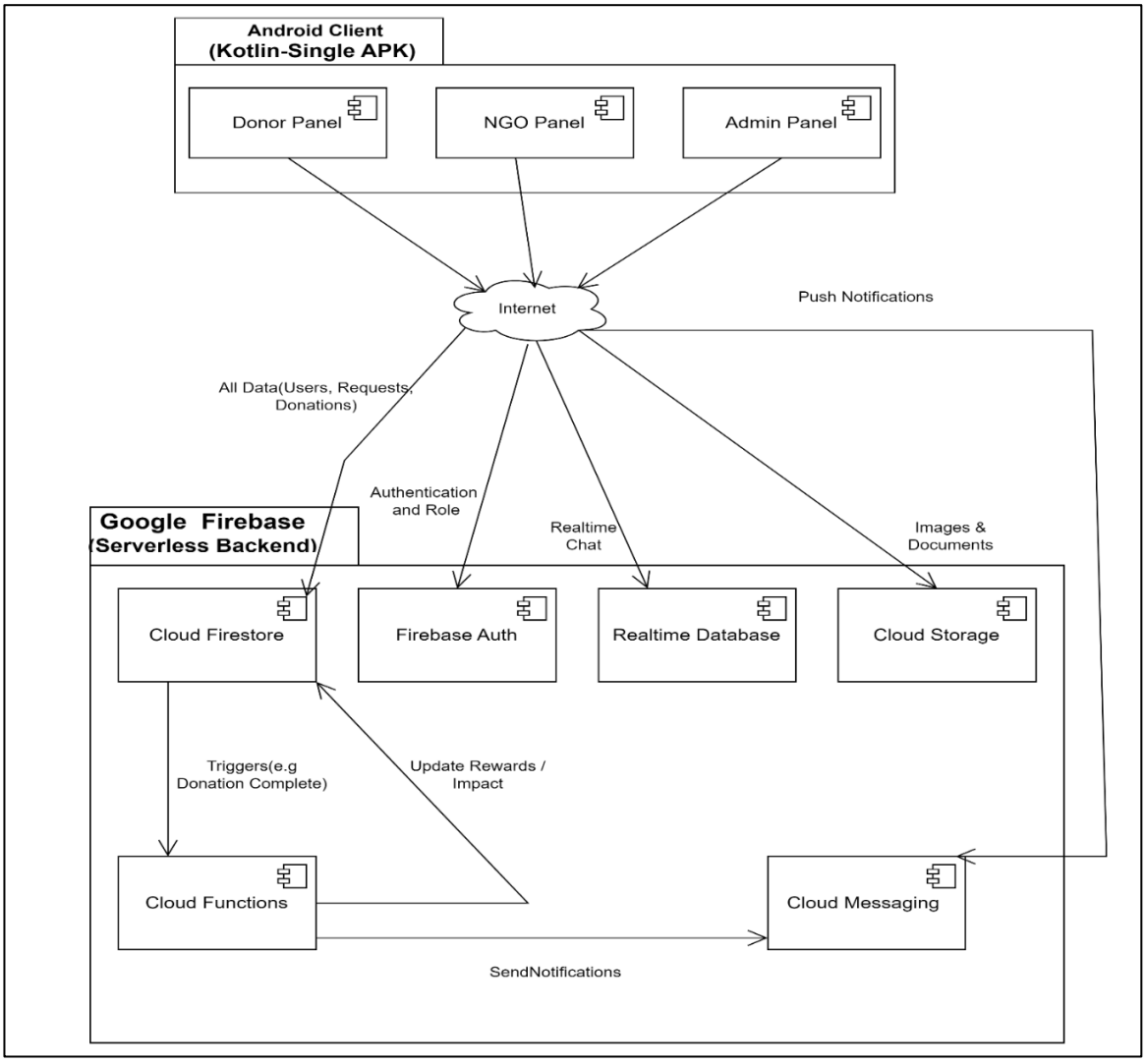


Figure 4.1: Complete System Architecture Diagram

4.2 Data Representation

Data representation defines how information is structured, stored, and accessed within the GiveEase system. Since GiveEase uses Firebase Firestore as its primary database, which is a NoSQL document-based database, data is organized into collections containing documents with field-value pairs. This section describes each collection, its document structure, field types, and the relationships between collections.

The database design follows NoSQL best practices, including denormalization for frequently accessed data and composite keys for efficient queries. The primary collections in the GiveEase database are: users, campaigns, donations, chatRooms, messages, rewards, notifications, and maintenance. Each collection is designed to support the functional requirements identified in Chapter 3 while optimizing for read performance.

Entity Relationship Diagram Description

The Entity Relationship Diagram (ERD) in Figure 4.2 illustrates the data structure of GiveEase using Firebase Firestore collections. Unlike SQL databases, Firestore uses document-based storage where each collection contains documents with flexible fields. The diagram shows eight main collections and their logical relationships.

Users Collection

The users collection stores profile information for all registered users. Each document is identified by the Firebase Auth UID (primary key). The role field determines access level (donor, ngo, or admin), and isVerified indicates verification status. This is the most frequently referenced collection in the database.

Campaigns Collection

The campaigns collection stores donation requests created by NGOs. Each campaign references an NGO through the ngoId foreign key. Fields include title, description, targetAmount, raisedAmount, status, and urgency level. One NGO can create many campaigns, but each campaign belongs to exactly one NGO.

Donations Collection

The donations collection records every donation made on the platform. Each donation references a donor (donorId) and a campaign (campaignId). The type field distinguishes between money, physical, and blood donations. One donor can make many donations, and one campaign can receive many donations.

ChatRooms Collection

The chatRooms collection stores metadata for donor-NGO conversations. The roomId is a composite key combining donorId, ngoId, and campaignId. The participants array stores both user IDs, and lastMessage provides a preview for chat lists. This resolves the many-to-many relationship between donors and NGOs.

Messages Subcollection

Messages are stored as a subcollection under each chat room document, not as a separate top-level collection. This design prevents chat room documents from exceeding Firestore's 1 MiB size limit. Each message includes senderId, content, type (text or image), imageUrl, and timestamp.

Rewards and Notifications Collections

The rewards collection stores badges earned by donors, including badge name, earned timestamp, and certificate URL when applicable. The notifications collection stores in-app notifications with title, message, type, and read status. Both collections reference users through donorId or userId foreign keys.

Maintenance Collection

The maintenance collection is a singleton containing exactly one document for system configuration. It stores maintenanceMode (boolean) to lock the app during updates and autoApproveNGOs (boolean) for automatic verification. This collection enables real-time platform governance.

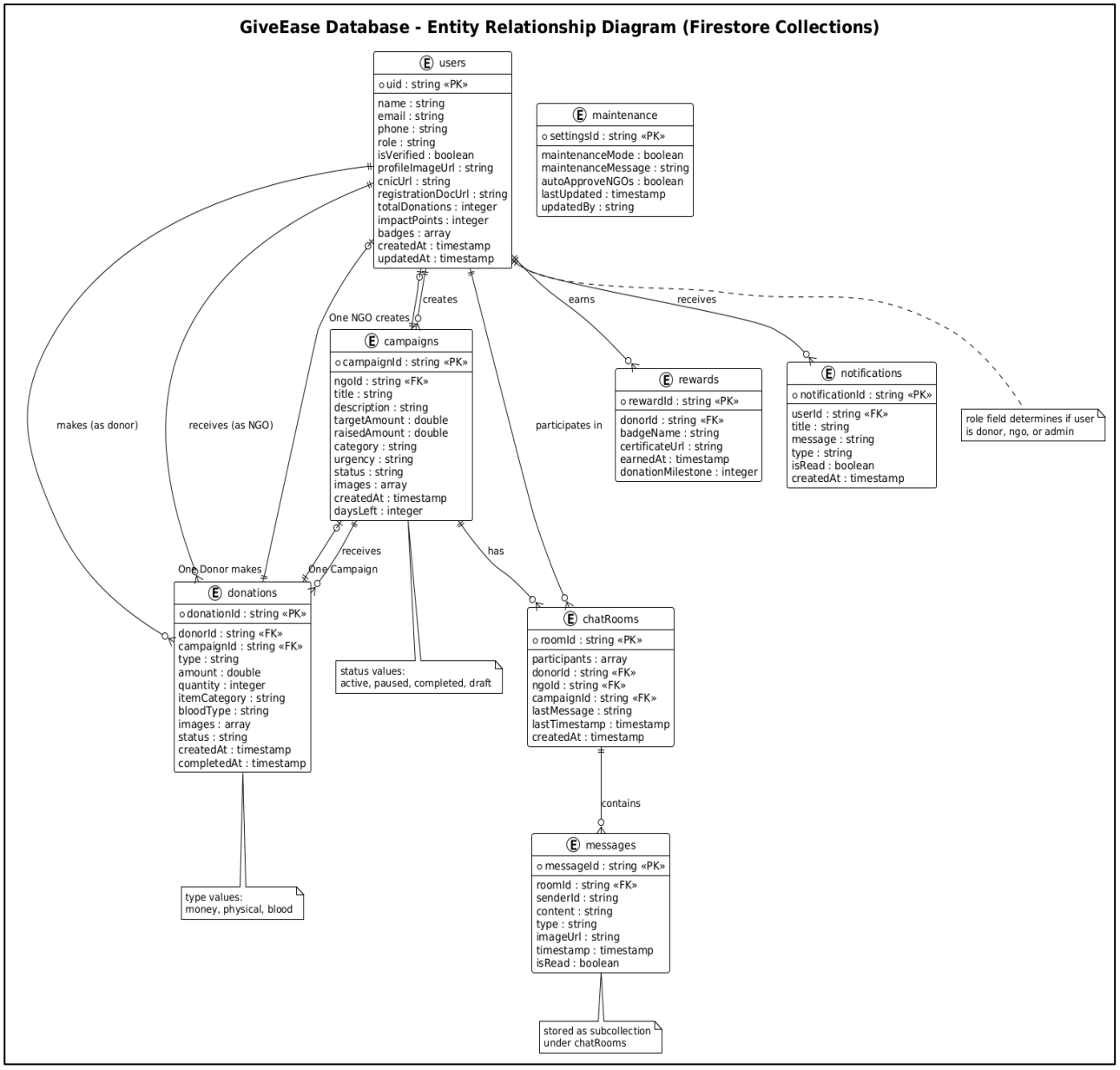


Figure 4.2: Entity Relationship Diagram

4.3 Process Flow and Representation

Process flow diagrams illustrate the step-by-step sequence of operations for key user journeys within the GiveEase application. These flows help developers understand the sequence of events, decision points, and data transformations that occur during common tasks such as making a donation, creating a campaign, or enabling maintenance mode.

4.3.1 Donation Process Flow

The donation process is the core user journey in GiveEase. When a donor selects a campaign and chooses to donate, the system must handle different workflows based on the donation type (money, physical items, or blood). For monetary donations, the process is straightforward: the donor enters an amount, the system validates it, updates the campaign's raised amount, and creates a donation record. For physical and blood donations, additional steps include collecting item details, uploading photos, and automatically creating a chat room for logistics coordination.

The donation process begins when the donor taps the "Donate Now" button on a campaign detail screen. The system first checks if the campaign is still active and if the donor is verified. If these checks pass, the donation type selection screen is presented. After the donor enters all required information and confirms, the system creates a donation document in Firestore with status "PENDING". Simultaneously, the system increments the campaign's raisedAmount field. For physical and blood donations, a chat room document is created, linking the donor and NGO for coordination. Finally, a push notification is sent to the NGO, and the donor is redirected to the chat screen.

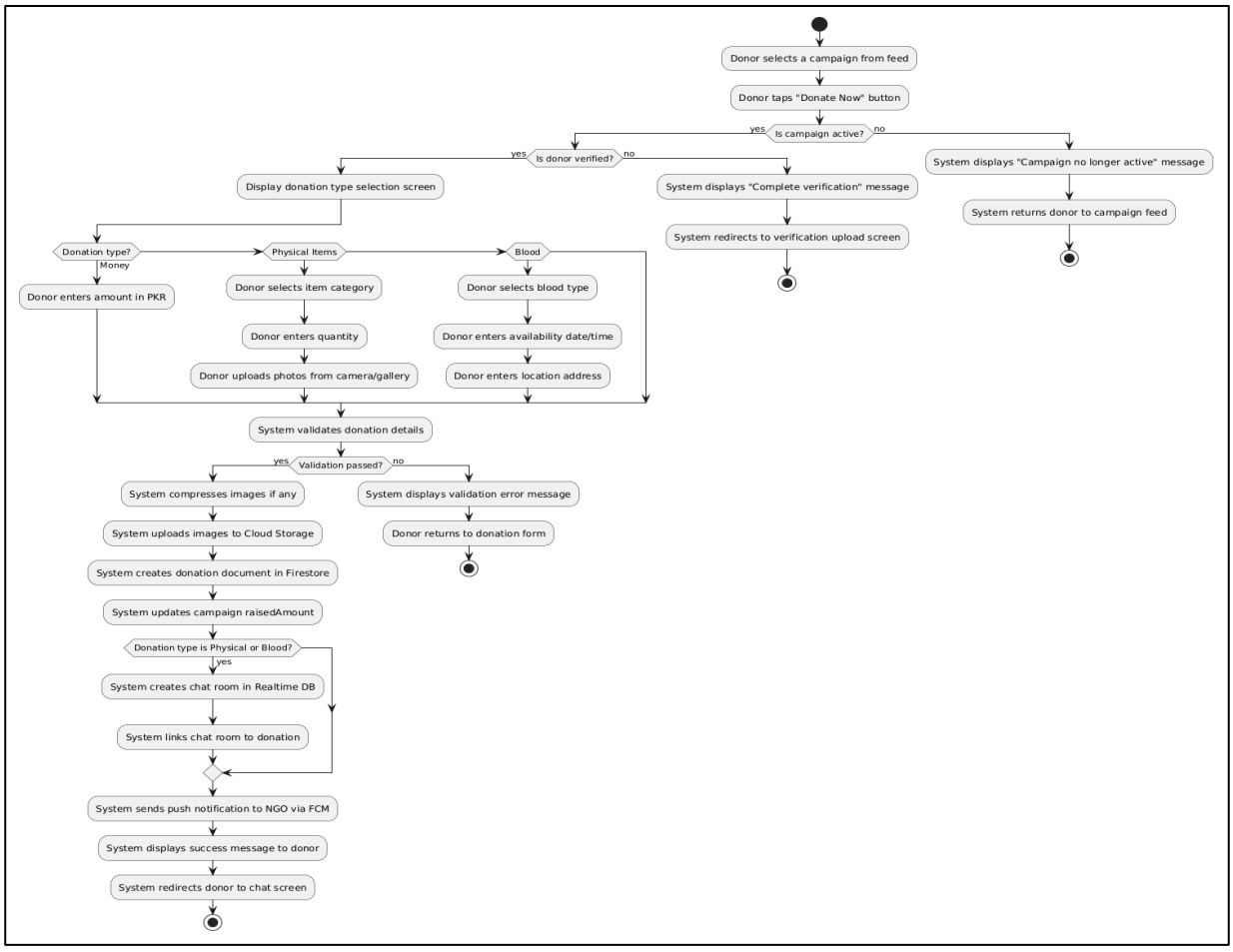


Figure 4.3.1: Activity diagram for the donation process

4.3.2 NGO Verification Process Flow

The NGO verification process ensures that only legitimate organizations can create campaigns and receive donations on the platform. When an NGO registers, they must upload their official registration certificate and other verification documents. The admin reviews these documents through the admin panel and either approves or rejects the request.

The verification process begins when the NGO completes registration and uploads documents. The system stores these documents in Cloud Storage and sets the NGO's isVerified field to false. The admin sees a pending verification request in their dashboard. When the admin reviews the documents, they can zoom, rotate, or download them for careful inspection. If the admin approves, the system sets isVerified to true, sends a welcome notification, and the NGO can immediately start creating campaigns. If rejected, the NGO receives a notification with the rejection reason and is prompted to resubmit correct documents.

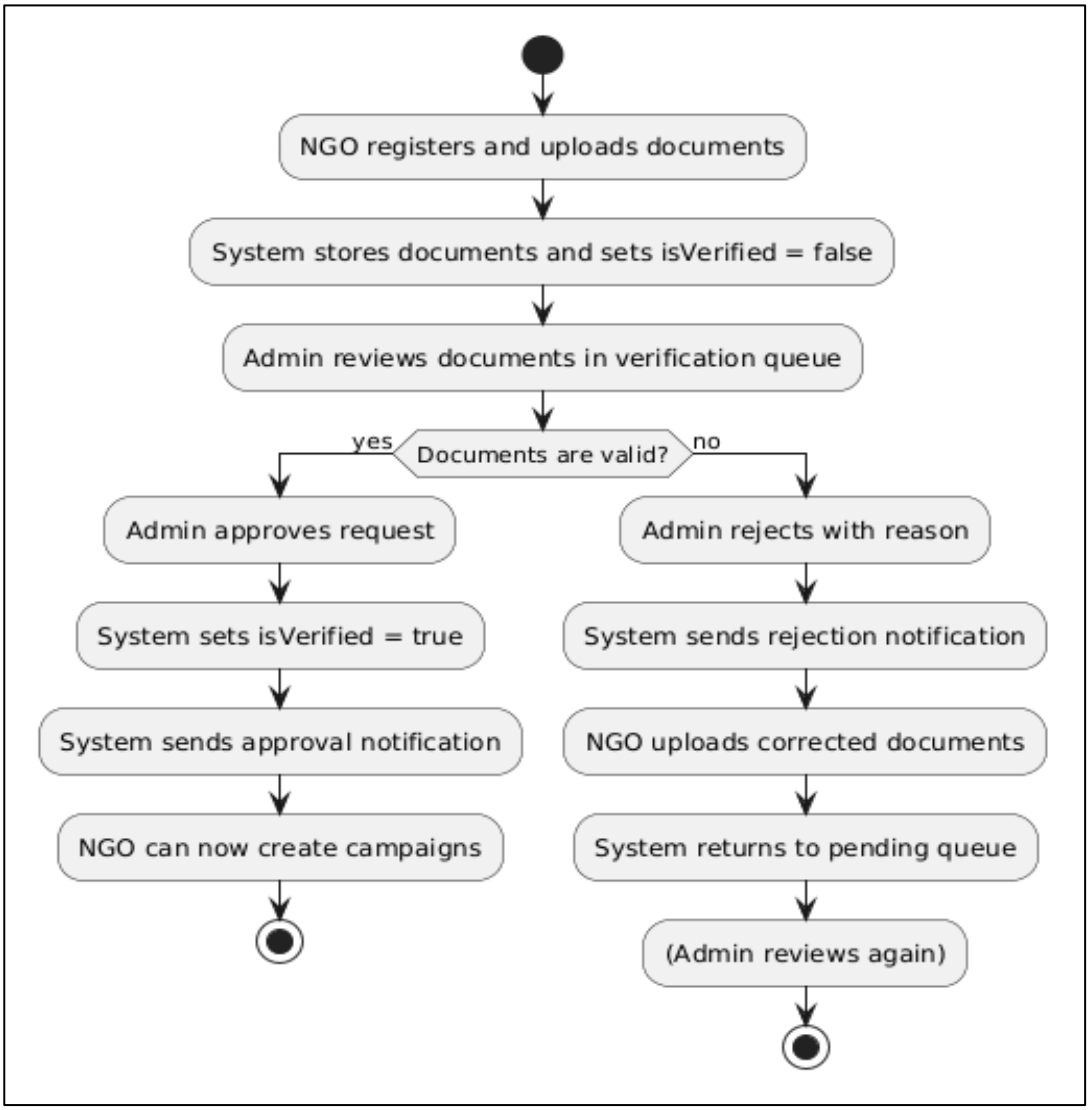


Figure 4.3.2: Activity diagram for the NGO verification process

4.3.3 Maintenance Mode Process Flow

Maintenance mode is a unique safety feature in GiveEase that allows the admin to temporarily lock the application during critical updates or emergency fixes. When enabled, all non-admin users are redirected to a maintenance screen and prevented from performing any database write operations.

The process begins when the admin toggles the maintenance mode switch in the admin settings. A confirmation dialog appears to prevent accidental activation. Upon confirmation, the system updates a flag in the Firestore maintenance collection. All active client sessions have a real-time listener attached to this flag. When the flag changes from false to true, each client immediately navigates to the MaintenanceActivity. The maintenance screen displays a custom message set by the admin. While maintenance mode is active, write operations are disabled for non-admin users, but read operations for cached data may still work. When the admin completes their updates, they toggle the switch off, and users are redirected back to their dashboards.

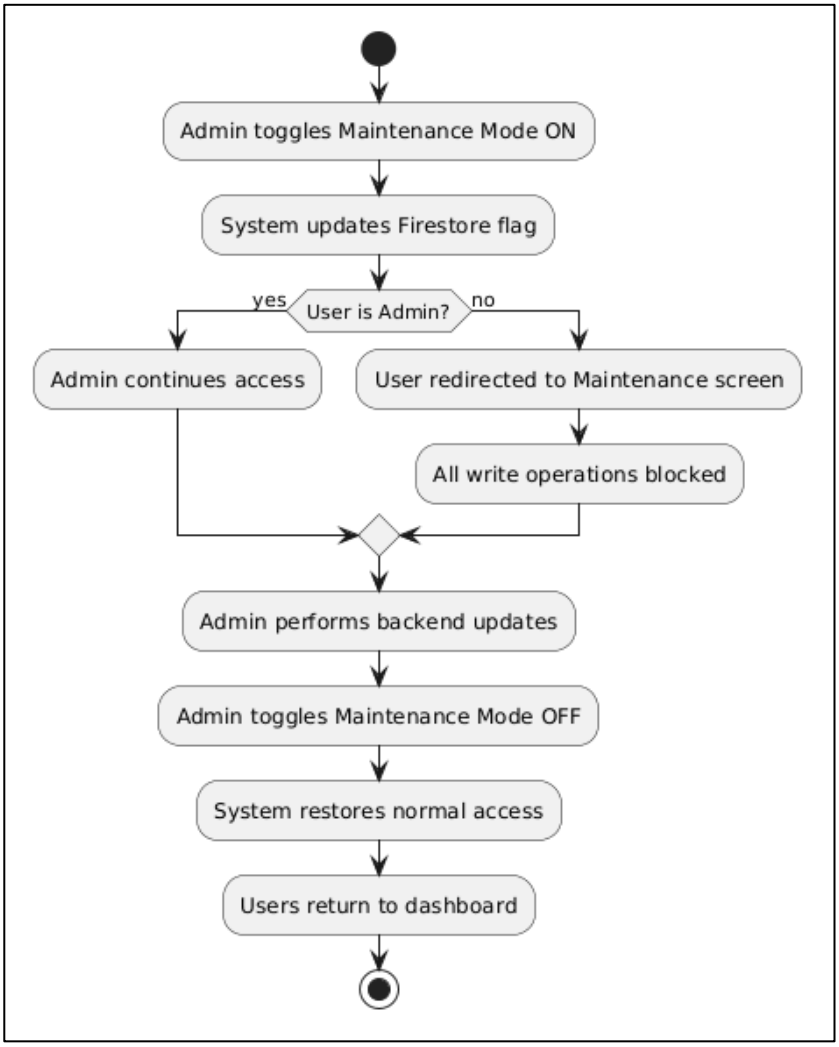


Figure 4.3.3: activity diagram for the maintenance mode process

4.4 Sequence Diagrams

Sequence diagrams provide a detailed view of interactions between objects over time for specific use cases. Each sequence diagram shows the actors involved, the order of messages exchanged, and the lifelines of participating objects. The following sequence diagrams cover the most critical operations in GiveEase.

4.4.1 User Registration and Authentication Sequence

This sequence diagram illustrates the complete user registration process, including account creation in Firebase Auth, profile creation in Firestore, and document upload for verification. The diagram shows how data flows between the mobile app, Firebase services, and the local device storage.

The process begins with the user entering their registration details. The app validates input locally before calling Firebase Auth to create the account. Upon successful account creation, Firebase returns a UID. The app then creates a user document in Firestore using this UID as the document ID. The user is then prompted to upload verification documents, which are stored in Cloud Storage. The document URLs are saved to the user's profile. Finally, the user is logged in and redirected to the appropriate dashboard based on their selected role.

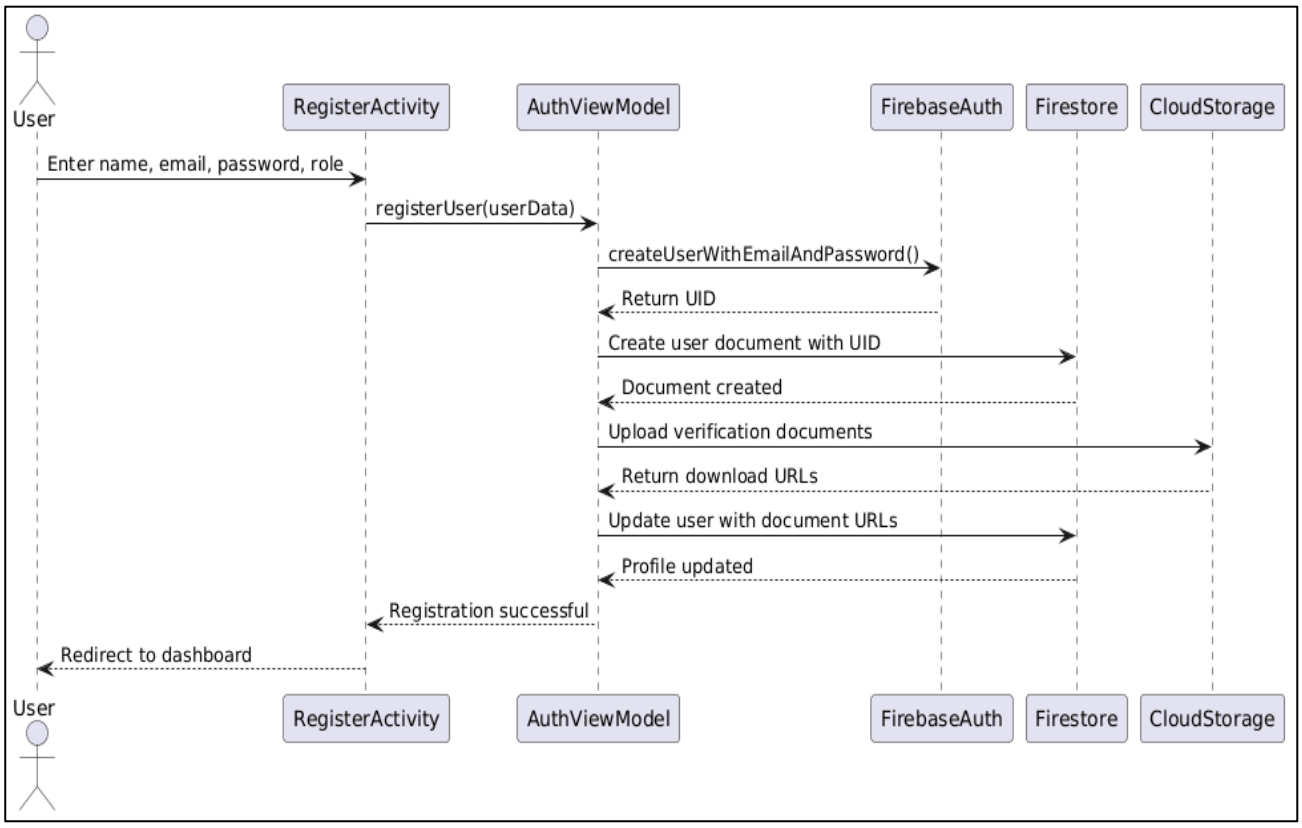


Figure 4.4.1: Sequence diagram for user registration

4.4.2 Donation Processing Sequence

This sequence diagram illustrates the complete donation process for a physical item donation. The diagram shows interactions between the donor, the campaign detail screen, the donation view model, the repository, Firestore for data storage, Cloud Storage for image uploads, and the real-time database for chat creation.

The donor selects a campaign and taps donate. The donation screen collects item details and photos. Photos are uploaded to Cloud Storage, and the returned URLs are stored. The view model creates a donation document in Firestore and updates the campaign's raised amount. If the donation is physical or blood, a chat room is created in the Realtime Database. Finally, a push notification is sent to the NGO via Cloud Messaging.

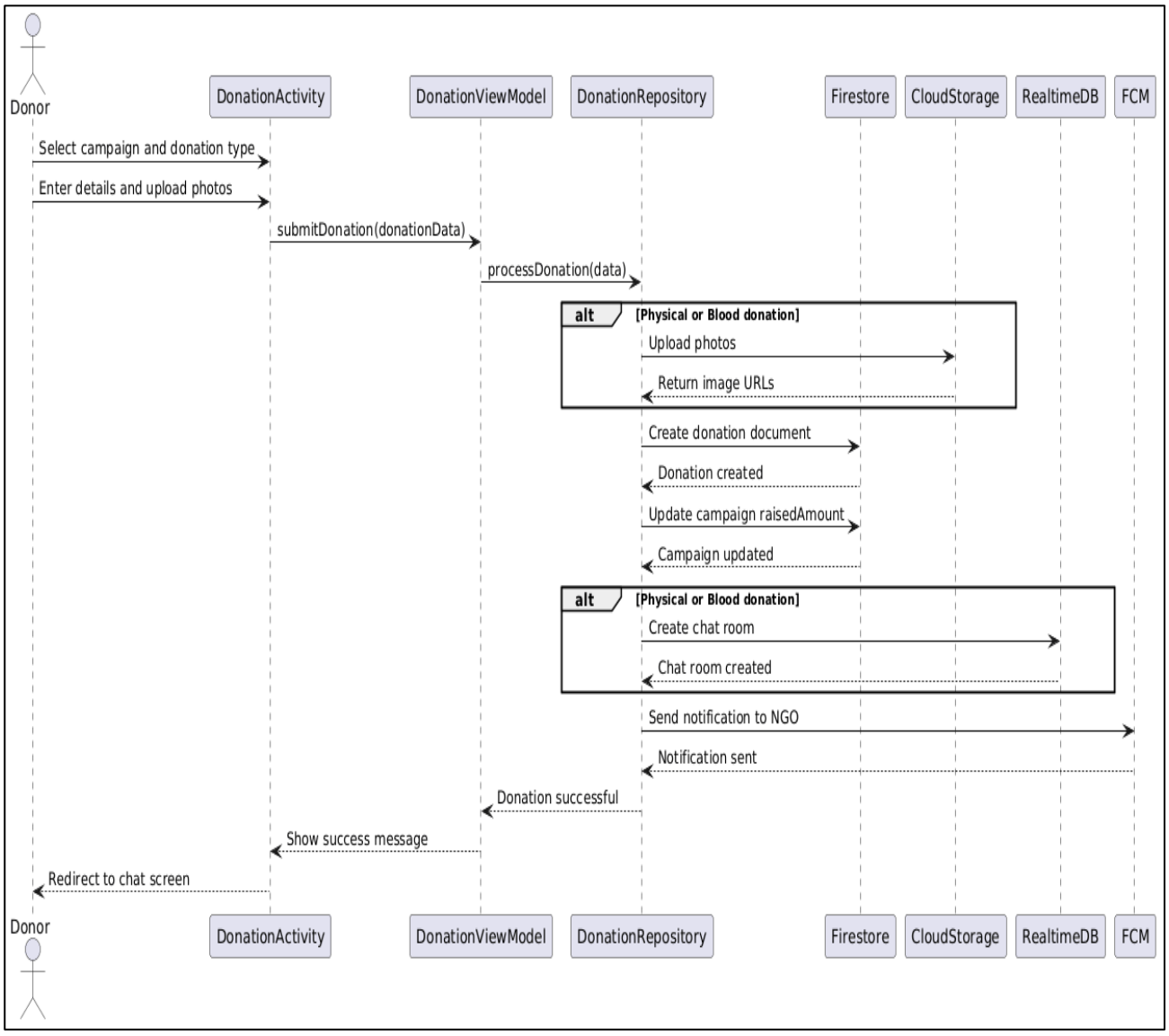


Figure 4.4.2: Sequence diagram for processing a physical item donation

4.4.3 Real-Time Chat Sequence

This sequence diagram illustrates how messages are sent and received in real-time between a donor and an NGO. The diagram shows the setup of the chat room, the message sending flow, and the real-time notification mechanism.

When the user opens a chat, the view model attaches a listener to the Realtime Database path for that chat room. New messages appear instantly without refreshing. When the user sends a message, the system writes the message object to the database. The real-time listener on the recipient's device fires immediately, displaying the message. If the recipient is not in the chat screen, a push notification is sent via FCM.

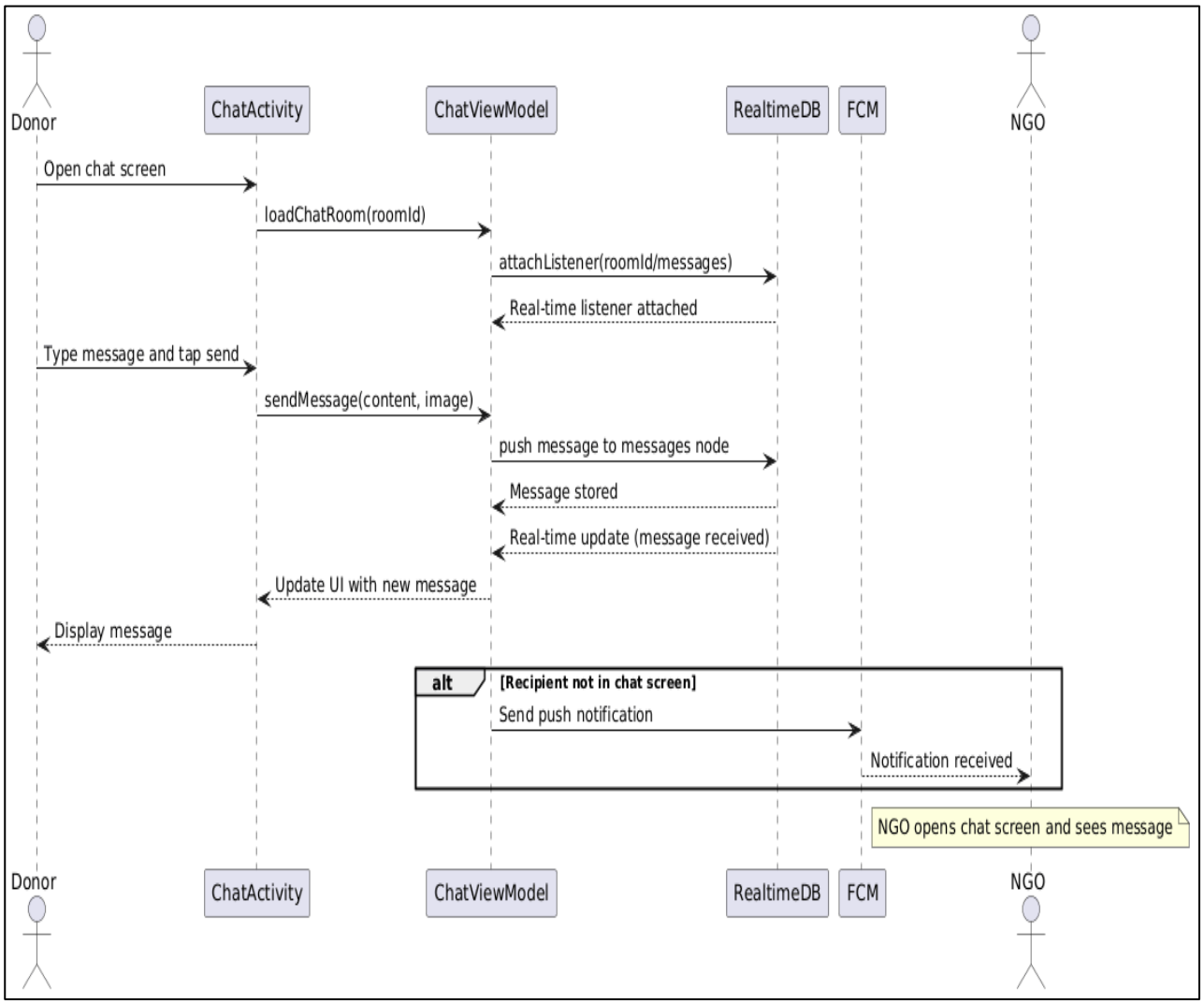


Figure 4.4.3: Sequence diagram for real-time chat

4.4.4 Admin Verification Sequence

This sequence diagram illustrates the admin verification process for NGO accounts. The diagram shows how the admin views pending requests, reviews documents, and approves or rejects the verification.

The admin opens the verification screen, which queries Firestore for NGOs with isVerified false. The admin selects an NGO and views their uploaded documents. After reviewing, the admin selects approve or reject. The system updates the NGO's isVerified field and sends a push notification to the NGO. An audit log entry is also created.

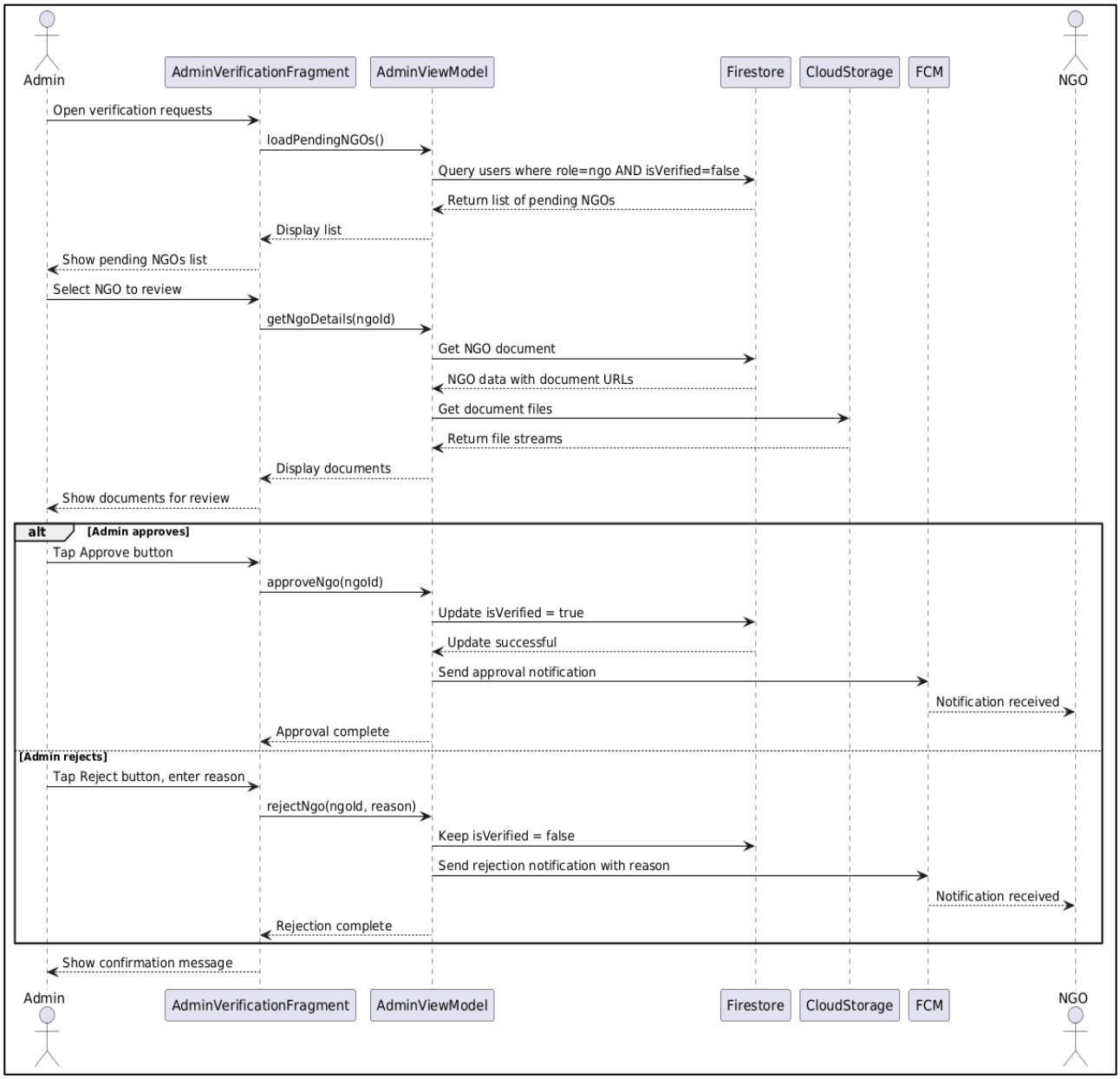


Figure 4.4.4: Sequence diagram for admin verification

4.4.5 Maintenance Mode Sequence

This sequence diagram illustrates how the maintenance mode flag toggles and how clients respond to the change in real-time. The diagram shows the admin enabling maintenance mode and multiple client sessions being redirected.

The admin toggles maintenance mode on. The system updates the maintenance document in Firestore. All active client sessions have a listener on this document. When the listener detects the change, each client navigates to the maintenance screen. While maintenance mode is active, any write attempts from non-admin users are blocked by security rules.

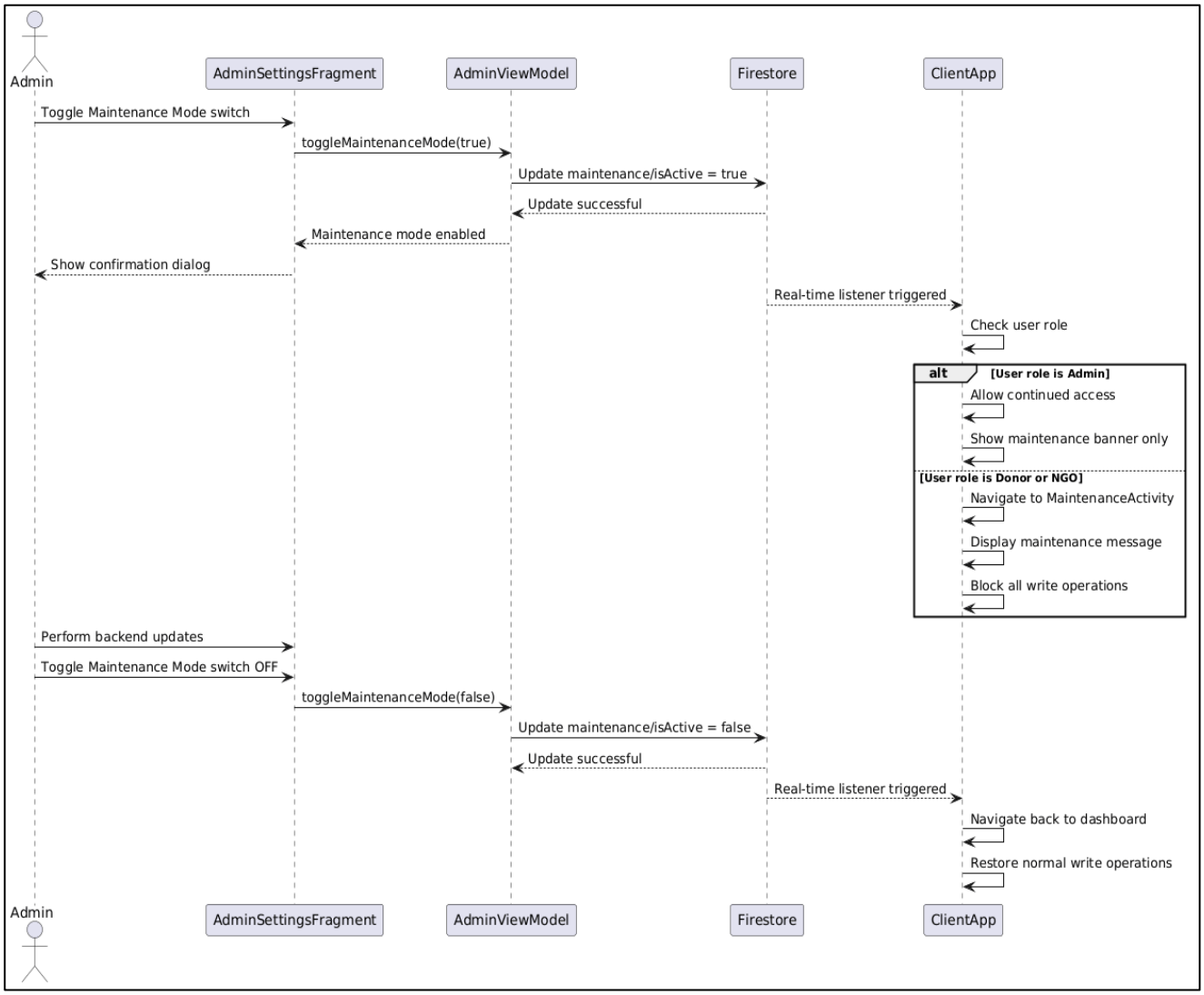


Figure 4.4.5: Sequence diagram for maintenance mode

4.5 Class Diagram

The class diagram provides a static view of the object-oriented structure of the GiveEase application. It shows the classes, their attributes, methods, and the relationships between them. The diagram follows the MVVM architectural pattern, separating the application into model, view, view model, and repository layers.

The **Model Layer** consists of data classes that represent business entities. These include User, Campaign, Donation, ChatRoom, Message, Reward, and Notification. Each data class contains properties that map to Firestore document fields and may include helper methods for formatting or calculation. For example, the Campaign class includes a getProgress() method that calculates (raisedAmount / targetAmount) * 100, and a formatAmount() method that displays large numbers in a readable format (e.g., 1.5M).

The **View Model Layer** consists of classes that bridge the model and the view. Each screen in the application has a corresponding view model. Examples include CampaignViewModel (for the campaign feed), DonationViewModel (for the donation process), ChatViewModel (for real-time messaging), and ImpactDashboardViewModel (for donor statistics). View models expose LiveData or StateFlow objects that fragments observe for data changes. They also contain methods that fragments call in response to user actions.

The **Repository Layer** consists of classes that abstract the data sources. The repository pattern ensures that the view model does not need to know whether data comes from Firestore, the Realtime Database, or a local cache. Examples include CampaignRepository (for campaign data), DonationRepository (for donation operations), ChatRepository (for message handling), and UserRepository (for profile and authentication).

The **View Layer** consists of activities and fragments that display the user interface. MainActivity serves as the entry point and handles role-based navigation. DonorHomeFragment displays the campaign feed. NgoMainFragment displays the NGO dashboard. AdminSettingsFragment contains configuration toggles. ChatDetailActivity handles real-time messaging.

The relationships between classes include inheritance (fragments extend androidx.fragment.app.Fragment), composition (view models contain repository instances), and association (repositories reference Firebase service classes).

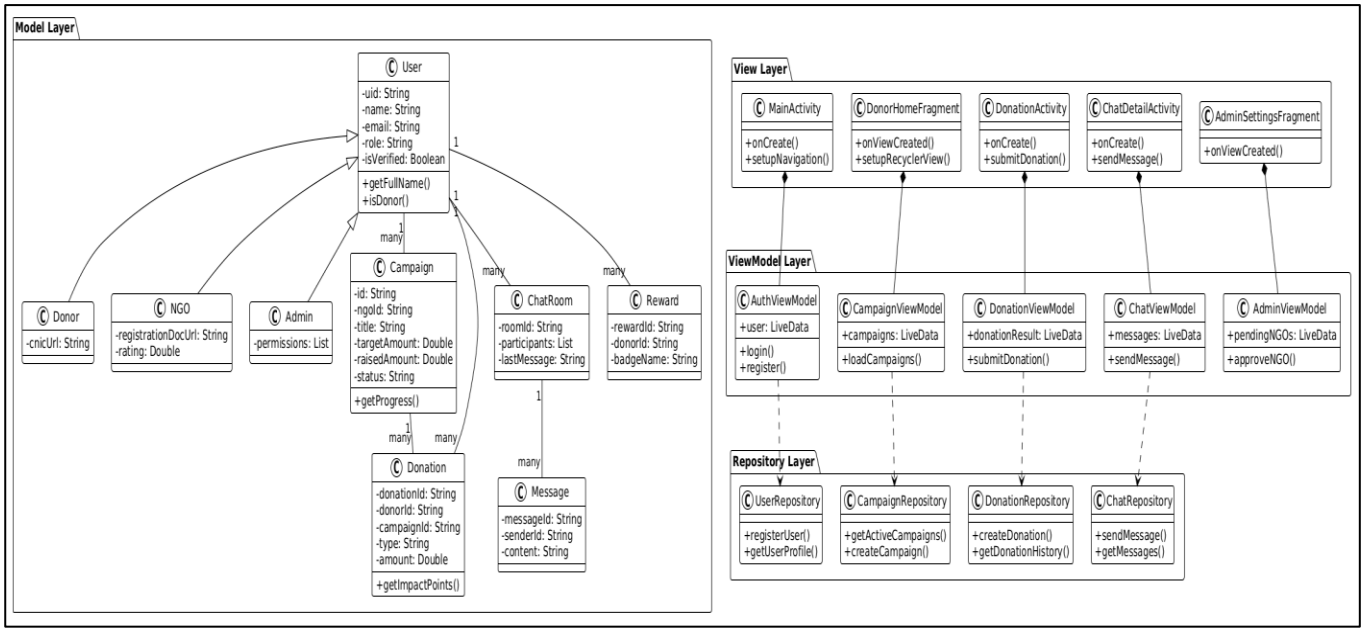


Figure 4.5: Complete class diagram

CHAPTER 5

Implementation

5. IMPLEMENTATION

Implementation is the core phase where the system design is transformed into a fully functional working product. This chapter covers the complete technical implementation of the GiveEase Donation App, including the development environment, programming languages, frameworks, APIs, Firebase services, core algorithms, and the integration of different modules such as authentication, campaign management, donation processing, real-time chat, reward system, and admin controls.

The implementation of GiveEase followed the MVVM architectural pattern described in Chapter 4. The frontend was developed using Kotlin with Android Studio, while the backend utilized Google Firebase services including Firestore for structured data, Realtime Database for chat messaging, Cloud Storage for file storage, Authentication for user management, and Cloud Messaging for push notifications. All code was managed using Git with a GitHub repository, following feature branching and pull request workflows.

5.1 Development Environment and Tools

The implementation of GiveEase involved a diverse range of technologies and frameworks, each chosen for its performance, suitability, and compatibility with the overall system architecture. This section describes the development environment, programming languages, libraries, and tools used to build the application.

5.1.1 Frontend Development (Kotlin and Android Studio)

The frontend of GiveEase was developed natively for Android using Kotlin as the primary programming language. Kotlin was chosen for its modern syntax, null safety features, coroutine support for asynchronous operations, and seamless interoperability with Java libraries. Android Studio (latest version as of 2026) served as the Integrated Development Environment (IDE), providing tools such as the visual layout editor, debugger, profiler, and emulator for testing on various virtual devices.

Key Android Components Used:

The application uses Activities as entry points for key screens. MainActivity serves as the main entry point and handles role-based navigation. DonationActivity handles the donation submission flow. ChatDetailActivity manages real-time messaging. SplashActivity displays the app launch screen while checking authentication status.

For Fragments, the application uses DonorHomeFragment to display the campaign feed to donors. NgoMainFragment displays the NGO dashboard with campaign management options.

AdminSettingsFragment contains configuration toggles for admin users. ImpactDashboardFragment displays donor statistics and earned badges.

For UI Components, the application uses RecyclerView with custom adapters for displaying lists of campaigns, donations, and chat messages. MaterialCardView is used for campaign cards with elevation for visual hierarchy. BottomNavigationView provides navigation between different sections of the app for each role. SwipeRefreshLayout allows users to pull-to-refresh content. ProgressBar indicates loading states during network operations.

For Asynchronous Operations, Kotlin Coroutines are used with Dispatchers.IO for Firebase Firestore operations and Dispatchers.Main for UI updates. This prevents the main thread from being blocked during long-running operations such as image uploads or database queries.

Third-Party Libraries Used:

The Glide library (version 4.16.0) is used for image loading and caching. It efficiently loads campaign images, profile pictures, and chat images from Firebase Cloud Storage, caches them locally to reduce network usage, and provides placeholder images while loading.

The Material Components library (version 1.12.0) provides Material Design 3 components including MaterialCardView, MaterialButton, SwitchMaterial, BottomNavigationView, and TextInputLayout for forms.

The Firebase BoM (Bill of Materials) version 32.7.0 manages all Firebase SDK versions to ensure compatibility.

5.1.2 Backend Services (Firebase)

GiveEase uses Google Firebase as a complete Backend-as-a-Service (BaaS) solution. No custom server-side code was written outside of Firebase services, which simplified deployment and scaling.

Firebase Authentication handles user registration, login, password reset, and session management. Email and password are used as the primary authentication method. The authentication state is observed throughout the app to determine whether the user is logged in and to retrieve the user's UID for Firestore queries.

Cloud Firestore serves as the primary database for structured data. It stores collections for users, campaigns, donations, chat rooms, rewards, notifications, and maintenance settings. Firestore provides real-time listeners that automatically update the UI when data changes, which is essential for features like the campaign feed and donation progress.

Firebase Realtime Database is used exclusively for the chat messaging system. While Firestore could also handle chat, Realtime Database provides lower latency (sub-100ms) for message delivery, which is critical for a smooth chat experience. Messages are stored as a list under each chat room.

Firebase Cloud Storage stores all user-uploaded files including profile pictures, campaign cover images, NGO registration documents, donor CNIC images, donation proof photos for physical items, and chat images. Files are organized in folders by user ID and content type for easy management.

Firebase Cloud Messaging (FCM) handles push notifications. When events occur such as a new donation, message received, or verification approval, the system triggers a Cloud Function that sends a notification to the relevant user's device.

Firebase Security Rules are implemented to enforce role-based access control. For example, only admin users can write to the maintenance collection, only the NGO that owns a campaign can edit it, and donors can only read their own donation history.

5.2 Core Algorithms

This section explains in detail the core algorithms powering the major components of GiveEase. These algorithms form the computational backbone of the system and significantly contribute to the functionality and user experience.

5.2.1 Role-Based Navigation Algorithm

The role-based navigation algorithm determines which dashboard to display after a user logs in. When a user successfully authenticates with Firebase, the system retrieves the user's document from Firestore, specifically the role field. Based on this role, the MainActivity loads the appropriate fragment.

Pseudocode:

Input: userId (from Firebase Auth)

Output: Appropriate fragment for user role

Begin

user = Firestore.collection("users").document(userId).get()

```
    role = user.get("role")

    if role == "donor" then
        loadFragment(DonorMainFragment())
        showBottomNavigationBar(DonorMenuItems)
    else if role == "ngo" then
        loadFragment(NgoMainFragment())
        showBottomNavigationBar(NgoMenuItems)
    else if role == "admin" then
        loadFragment(AdminMainFragment())
        showBottomNavigationBar(AdminMenuItems)
    else
        redirectToLoginScreen()
    end if
End
```

This algorithm ensures that donors cannot access NGO features, NGOs cannot access admin features, and each user sees only the interface elements relevant to their role.

5.2.2 Campaign Progress Calculation Algorithm

The campaign progress algorithm calculates the percentage of target amount raised for display in the progress bar. This calculation is performed in the Campaign data class and the result is used to update the UI whenever the raisedAmount field changes.

Formula:

Progress = (RaisedAmount / TargetAmount) × 100

Pseudocode:

Input: targetAmount (Double), raisedAmount (Double)

Output: progressPercentage (Int)

Begin

  if targetAmount > 0 then

    progress = (raisedAmount / targetAmount) * 100

    return progress.toInt()

  else

    return 0

  end if

End

For example, if a campaign has a targetAmount of 100,000 PKR and raisedAmount of 45,000 PKR, the progress will be 45%.

5.2.3 Donation Processing Algorithm

The donation processing algorithm handles the creation of a new donation when a donor submits their contribution. This algorithm validates the input, updates the campaign's raised amount, creates a donation record, and for physical or blood donations, automatically creates a chat room for coordination.

Pseudocode:

Input: donationData (Donation object), images (List<Uri>)

Output: donationId (String)

Begin

  Validate donation amount > 0

  Validate campaign exists and is active

  if donation.type == "physical" then

    For each image in images do

```
            compressedImage = compressImage(image, quality=75)
            imageUrl = uploadToCloudStorage(compressedImage)
            Add imageUrl to donation.images
        End For
    end if
    donationId = Firestore.collection("donations").add(donationData)
    Update campaign in Firestore:
        raisedAmount = raisedAmount + donation.amount
    if donation.type == "physical" or donation.type == "blood" then
        roomId = "${donorId}_${ngoId}_${campaignId}"
        Create chat room in RealtimeDatabase with roomId
        Add participants list to chat room
    end if
    Send push notification to NGO using FCM
    Return donationId
End
```

5.2.4 Impact Points Calculation Algorithm

The impact points algorithm calculates the points awarded to a donor for each completed donation. Different donation types have different point values to encourage diverse forms of giving.

Point Weights:

- Monetary donation: 1 point for every 100 PKR donated
- Physical item donation: 20 points per item
- Blood donation: 50 points per donation

Pseudocode:

Input: donation (Donation object)

Output: impactPoints (Int)

```
Begin
    points = 0
    if donation.type == "money" then
        points = (donation.amount / 100).toInt()
    else if donation.type == "physical" then
        points = donation.quantity * 20
    else if donation.type == "blood" then
        points = 50
    end if
    Return points
End
```

For example, a donor who donates 500 PKR earns 5 points. A donor who donates 3 physical items earns 60 points. A blood donor earns 50 points.

5.2.5 Badge Assignment Algorithm

The badge assignment algorithm checks a donor's total donation count and assigns badges when milestones are reached. This algorithm is triggered by a Firebase Cloud Function whenever a donation status changes to "COMPLETED".

Badge Milestones:

- Bronze: 1 donation
- Silver: 5 donations
- Gold: 10 donations
- Platinum: 20 donations

Pseudocode:

```
Input: donorId (String)
Output: assignedBadge (String or null)
Begin
```

```
  donationCount = Firestore.collection("donations")
      .whereEqualTo("donorId", donorId)
      .whereEqualTo("status", "COMPLETED")
      .get().size()

currentBadges = getUserBadges(donorId)
  if donationCount >= 20 and "Platinum" not in currentBadges then
      assignBadge(donorId, "Platinum")
      sendNotification(donorId, "Congratulations! You earned Platinum badge!")
      Return "Platinum"
  else if donationCount >= 10 and "Gold" not in currentBadges then
      assignBadge(donorId, "Gold")
      sendNotification(donorId, "Congratulations! You earned Gold badge!")
      Return "Gold"
  else if donationCount >= 5 and "Silver" not in currentBadges then
      assignBadge(donorId, "Silver")
      sendNotification(donorId, "Congratulations! You earned Silver badge!")
      Return "Silver"
  else if donationCount >= 1 and "Bronze" not in currentBadges then
      assignBadge(donorId, "Bronze")
      sendNotification(donorId, "Congratulations! You earned Bronze badge!")
      Return "Bronze"
  end if
  Return null
End
```

5.3 External APIs

GiveEase integrates with several external APIs and services to provide its functionality. This section describes each API, its purpose, and how it is used within the application.

5.3.1 Firebase Authentication API

The Firebase Authentication API is used for all user identity management. It provides methods for creating new user accounts, signing in existing users, sending password reset emails, and monitoring authentication state changes.

Implementation in GiveEase:

The AuthViewModel class wraps Firebase Authentication methods. For registration, createUserWithEmailAndPassword() is called. For login, signInWithEmailAndPassword() is used. The authentication state listener in MainActivity observes the current user and redirects to the login screen if no user is logged in.

5.3.2 Cloud Firestore API

The Cloud Firestore API provides a NoSQL document database with real-time synchronization. It is used for storing and querying all structured data including users, campaigns, donations, and settings.

Implementation in GiveEase:

The repository classes use Firestore queries to read and write data. For example, CampaignRepository uses collection("campaigns").whereEqualTo("status", "active").get() to retrieve active campaigns. Real-time listeners are attached using addSnapshotListener() to automatically update the UI when data changes.

5.3.3 Firebase Realtime Database API

The Realtime Database API is used exclusively for the chat messaging system. It provides low-latency message delivery and real-time synchronization across all connected clients.

Implementation in GiveEase:

The ChatRepository uses DatabaseReference to push messages to a specific chat room path. ChildEventListener is attached to listen for new messages. When a message is added, the listener triggers and the UI is updated immediately.

5.3.4 Firebase Cloud Storage API

The Cloud Storage API is used for storing and serving user-uploaded files. It provides secure file upload and download with built-in authentication integration.

Implementation in GiveEase:

The UserRepository and DonationRepository use the StorageReference API to upload files. For images, putFile() is called with the local URI, and onSuccessListener handles the completion. The download URL is obtained using downloadUrl.addOnSuccessListener() and then stored in Firestore.

5.3.5 Firebase Cloud Messaging API

The FCM API is used to send push notifications to users' devices. Notifications are triggered by Cloud Functions when certain events occur.

Implementation in GiveEase:

When a donation is created, a Cloud Function sends a notification to the NGO's device token. The notification includes the donor's name, donation amount, and a deep link to the donation details screen.

5.3.6 Glide Image Loading Library

Glide is an image loading and caching library for Android. It handles image fetching, decoding, display, and caching automatically.

Implementation in GiveEase:

Glide is used in RecyclerView adapters to load campaign images and profile pictures. It caches images to disk and memory, reducing network usage and improving scrolling performance. Images are resized to fit the ImageView dimensions using override() and centerCrop() transformations.

To prevent memory leaks during rapid scrolling, Glide’s lifecycle integration with ViewLifecycleOwner is utilized. Placeholder and error drawables are set using placeholder() and error() methods to provide visual feedback while images load or if a fetch fails. For profile pictures, circleCrop() transformation is applied to create uniform circular avatars. Additionally, diskCacheStrategy(DiskCacheStrategy.ALL) ensures both original and transformed images are cached, further optimizing repeated loads. Use of RequestOptions allows centralized configuration of these parameters across all image-loading calls in the app.

5.4 User Interface

The user interface of the GiveEase Donation App is designed to be clean, intuitive, and accessible for all user types. Following Material Design 3 guidelines, the app provides a consistent experience across donor, NGO, and admin panels. This section presents screenshots of all major screens with descriptions of their layout and functionality.

5.4.1 Login Screen

The login screen is the entry point for existing users to access their accounts. It displays the GiveEase logo, email input field, password input field with visibility toggle, login button, and links to sign up or reset password. Input validation is performed locally before sending credentials to Firebase Authentication. Error messages appear for invalid email format, incorrect password, or non-existent accounts.

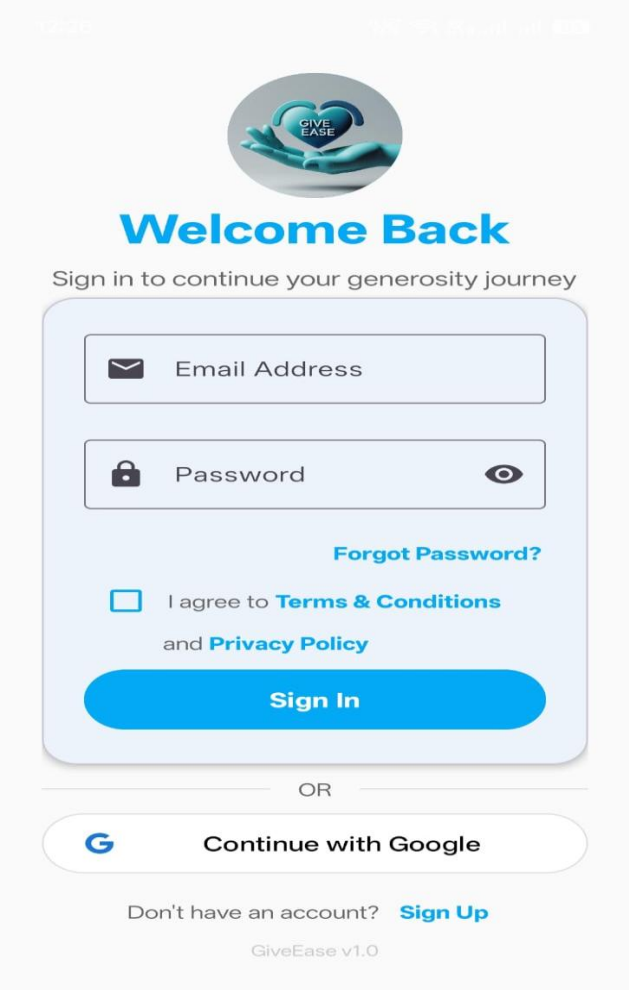


Figure 5.4.1: Login screen

5.4.2 Sign Up Screen

The signup screen allows new users to create an account. It collects full name, email address, phone number, password, and confirm password. A role selection section allows the user to choose between Donor and NGO using radio buttons or segmented buttons. For donors, a CNIC upload option appears. For NGOs, a registration document upload option appears. All fields are validated before submission. Upon success, the user is directed to upload verification documents.

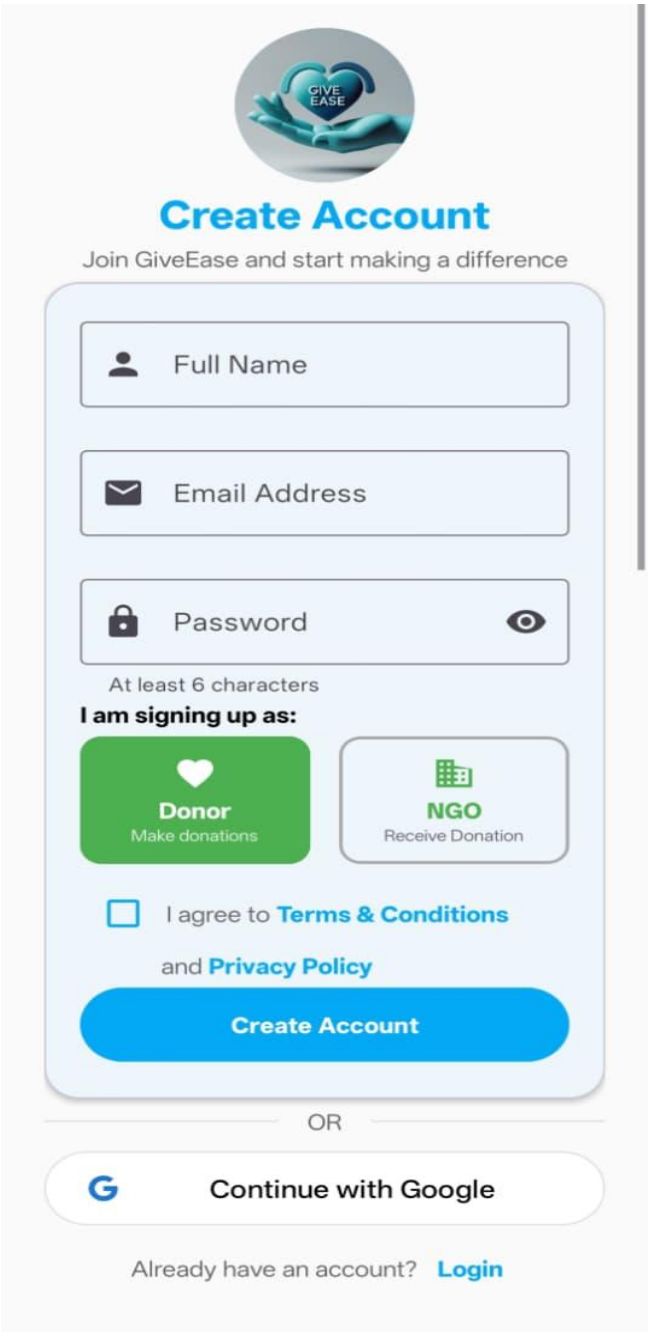


Figure 5.4.2: Sign Up screen

5.4.3 Forgot Password Screen

The forgot password screen allows users to reset their password if they have forgotten it. It collects the user's email address and sends a password reset link via Firebase Authentication. A success message appears when the email is sent. If the email is not registered, an error message is displayed.

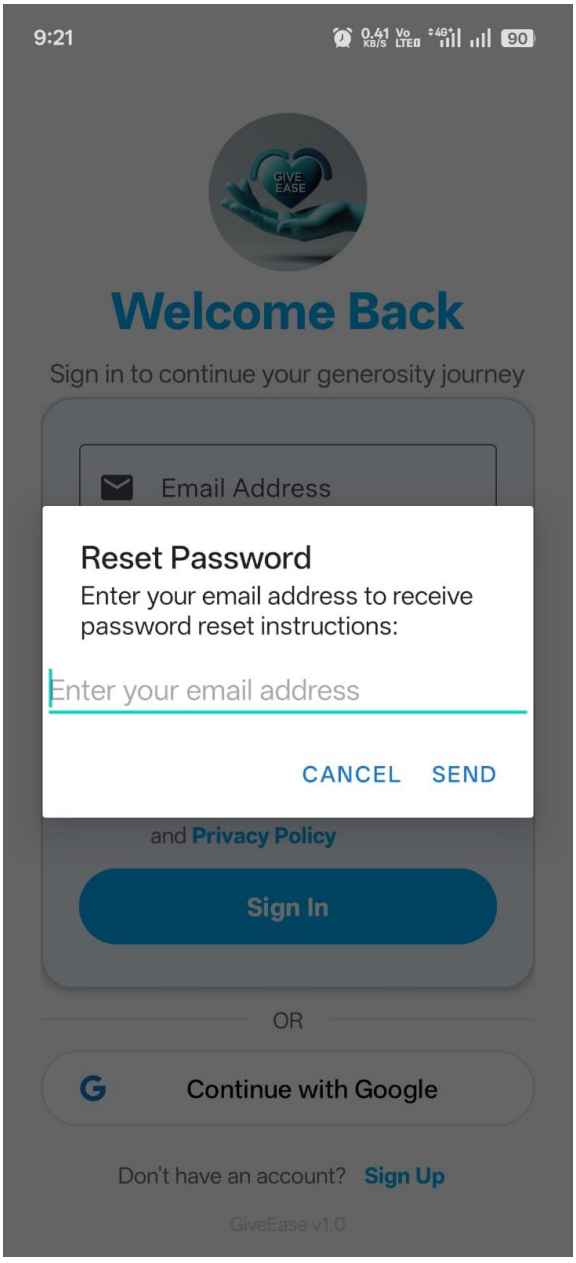


Figure 5.4.3: Forgot Password screen

5.4.4 Donor Dashboard Screen

The donor dashboard serves as the central hub for donors. It displays a welcome message with the donor's name, a summary of their impact points and badges earned, a "Donate Now" button for quick access to campaigns, and a bottom navigation bar with Home, Chat, and Profile options. The home screen shows a list of active campaigns with progress bars, urgency indicators, and verified NGO badges.

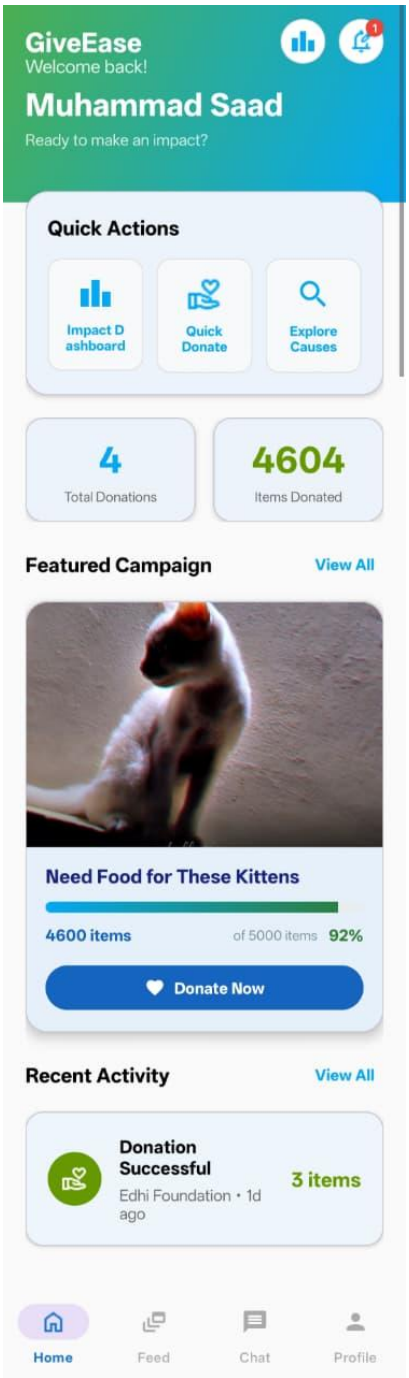


Figure 5.4.4: Donor Dashboard screen

5.4.5 Campaign Details Screen

When a donor taps on a campaign card, they are taken to the campaign details screen. This screen shows the full description of the campaign, the NGO's profile information with verified badge, a larger progress bar with exact numbers, a list of recent donations, and a prominent "Donate Now" button.

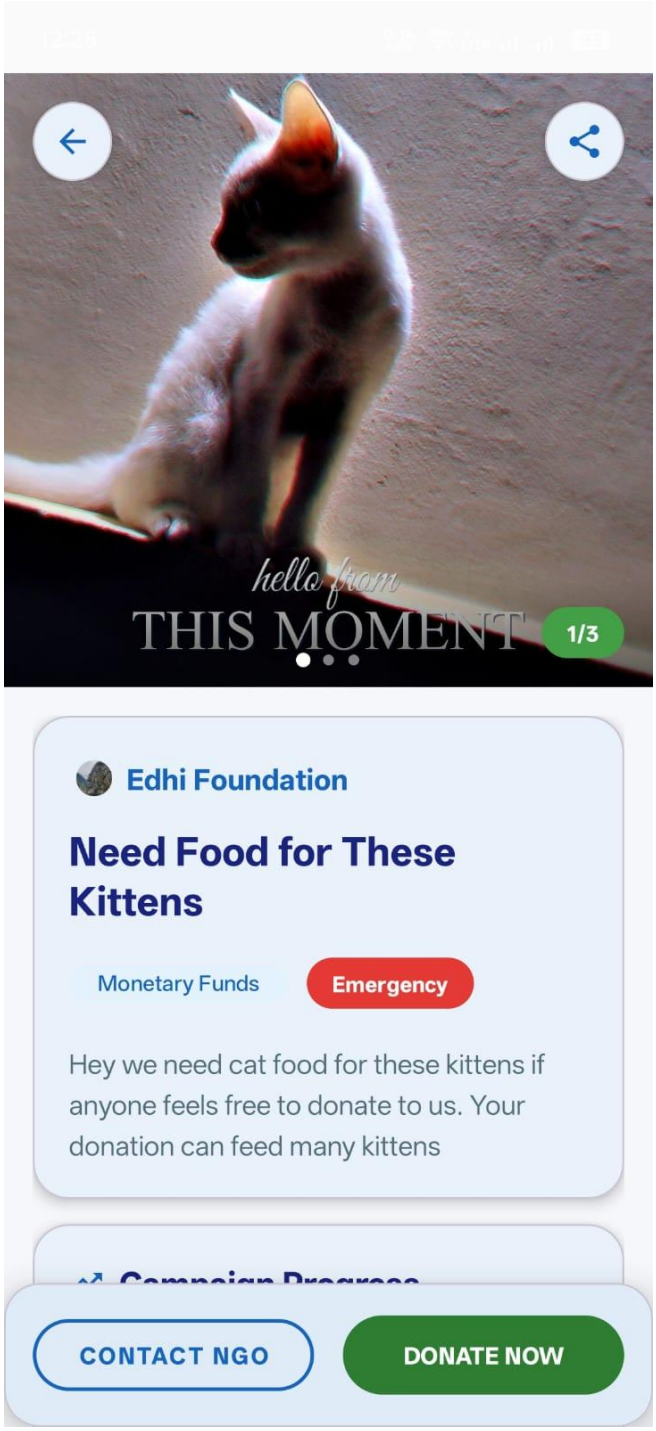


Figure 5.4.5: Campaign Details screen

5.4.6 Donation Type Selection Screen

When the donor taps "Donate Now", the donation type selection screen appears. This screen presents three cards: Money, Physical Items, and Blood. Each card has an icon, title, and brief description. Tapping a card opens the corresponding donation form.

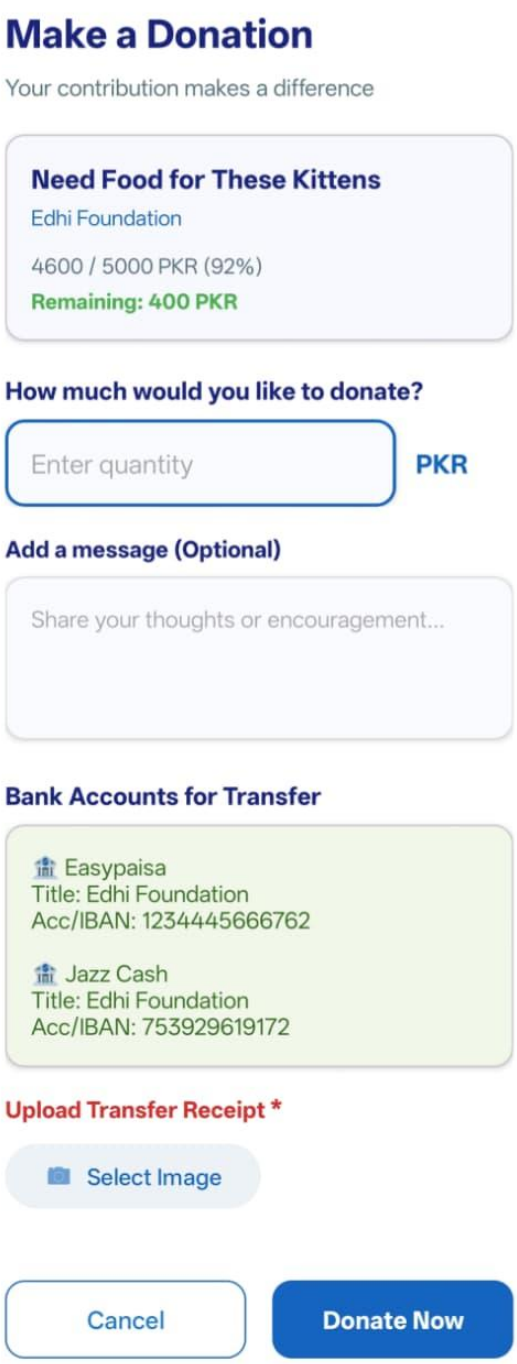


Figure 5.4.6: Donation Type Selection screen

5.4.7 Real-Time Chat Screen

The chat screen enables communication between donors and NGOs for logistics coordination. It displays message bubbles with timestamps, an input field for text, and an attachment icon for images. Messages appear instantly without refreshing. Image attachments show thumbnails that can be tapped to view full size.

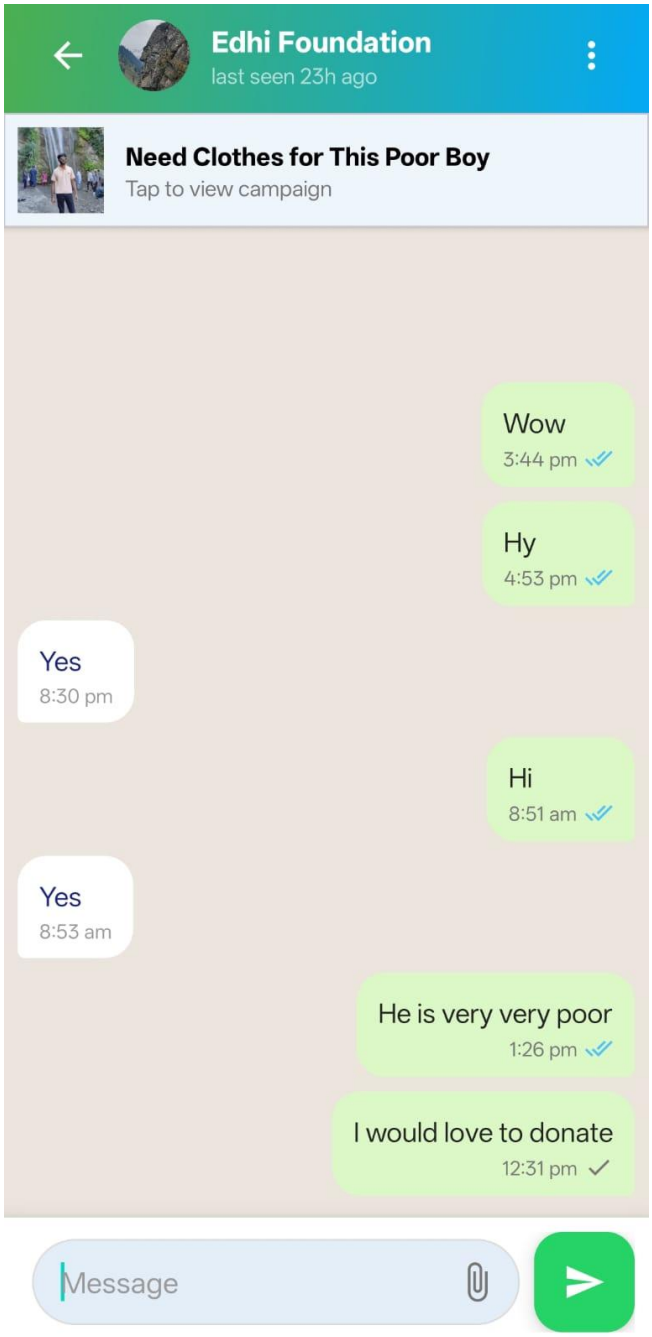


Figure 5.4.7: Real-Time Chat screen

5.4.8 Impact Dashboard Screen

The impact dashboard shows donors their cumulative impact. It displays total impact points, a list of earned badges with icons, a progress bar showing distance to the next badge, and a donation history chart. A "Share Your Impact" button allows donors to share their achievements on social media.

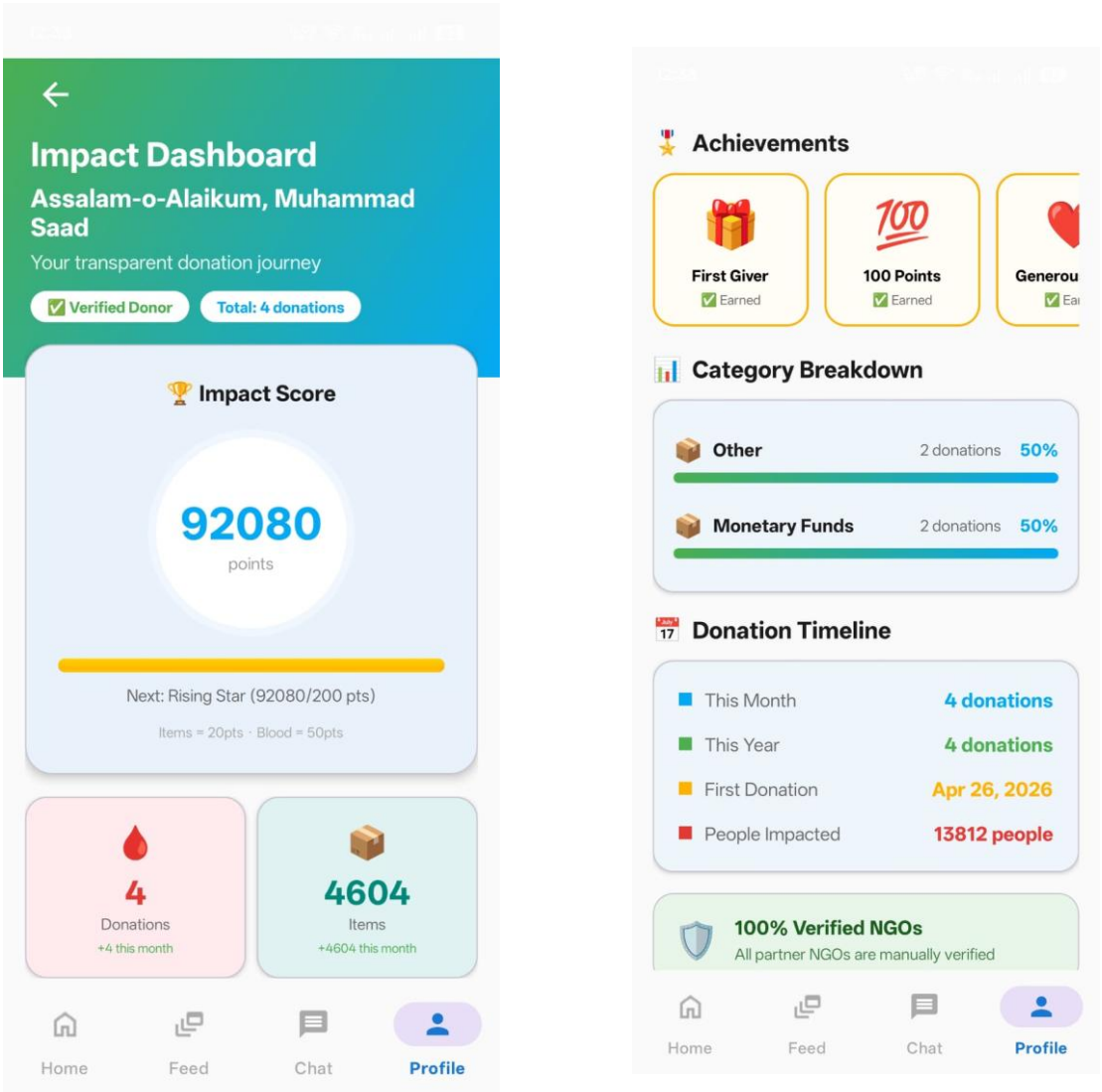


Figure 5.4.8: Impact Dashboard screen

5.4.9 Donor Campaign Feed Screen

This screen displays all active donation campaigns from verified NGOs in a scrollable list. Each campaign card shows the NGO name, campaign title, description preview, target amount, raised amount with a circular progress bar, urgency level indicator, and days remaining. A filter enables category. Pull-to-refresh functionality allows donors to reload the feed.

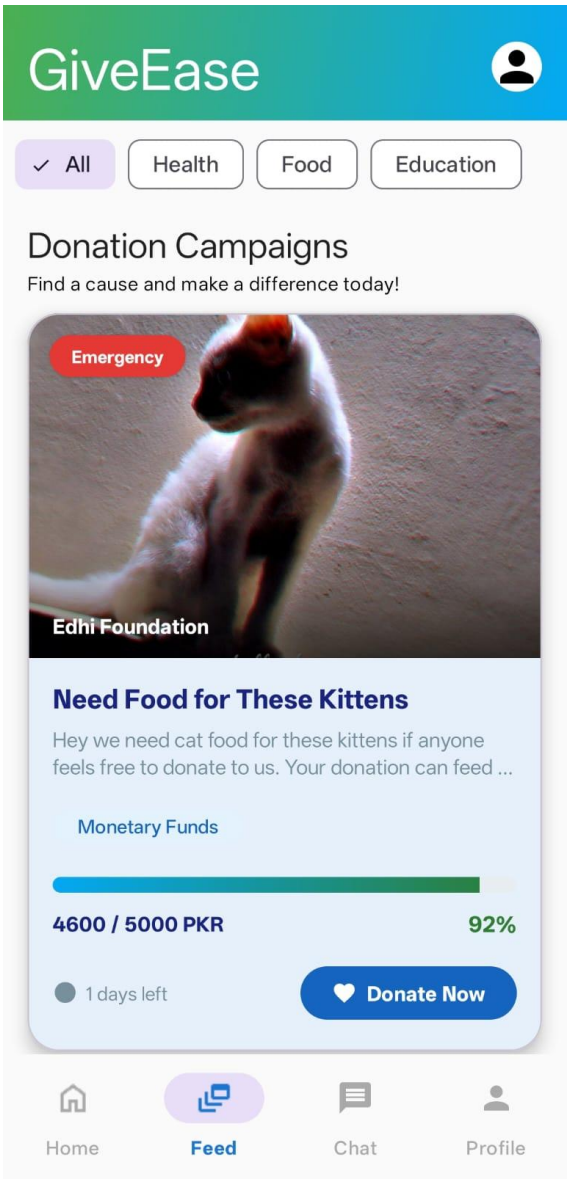


Figure 5.4.9: Donor Campaign Feed Screen

5.4.10: Donor Notifications Screen

This screen displays all push notifications received by the donor. Notifications include alerts for new campaigns matching donor preferences, donation confirmation messages, badge earning alerts, chat message notifications, and verification status updates. Each notification shows the title, message content, timestamp, and an unread indicator (red dot).

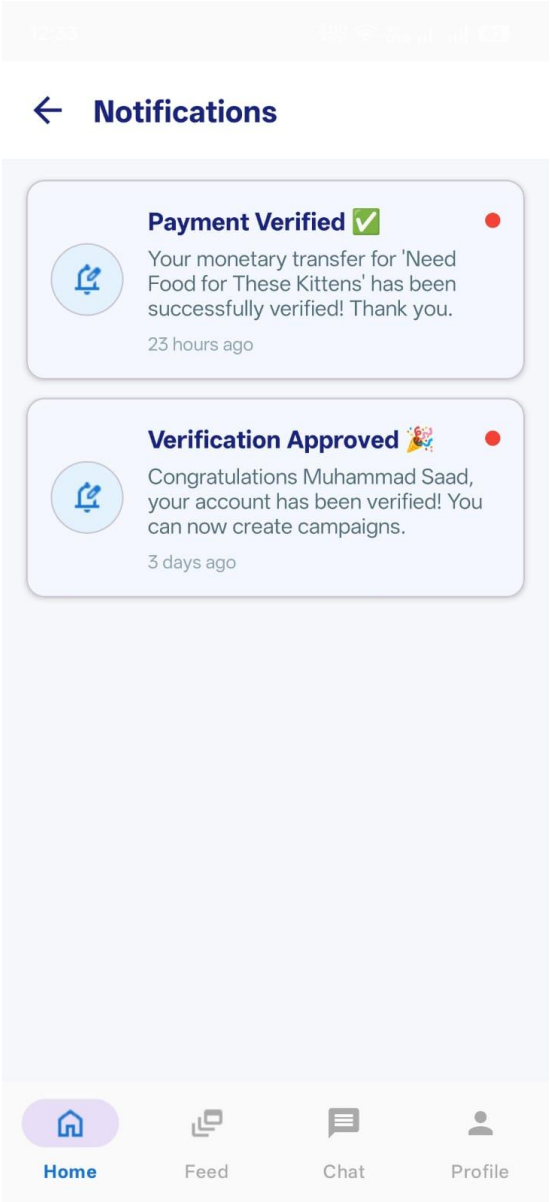


Figure 5.4.10: Donor Notifications Screen

5.4.11: Donor Profile and Settings Screen

This screen displays the donor's profile information and application settings. The profile section shows the donor's name with a verified account badge, total donations count, total NGOs helped, and total impact points. Three action buttons are available: "View History" to see donation history, "My Impact Dashboard" to view detailed impact statistics, and "Settings" to access account settings. The settings section includes "Change Email" and "Change Password" under Account Settings, "Notifications" toggle under Preferences, and "FAQs", "Contact Support", "Privacy Policy", and "Terms & Conditions" under Support & Information. "Logout" and "Delete Account" buttons are present at the bottom.

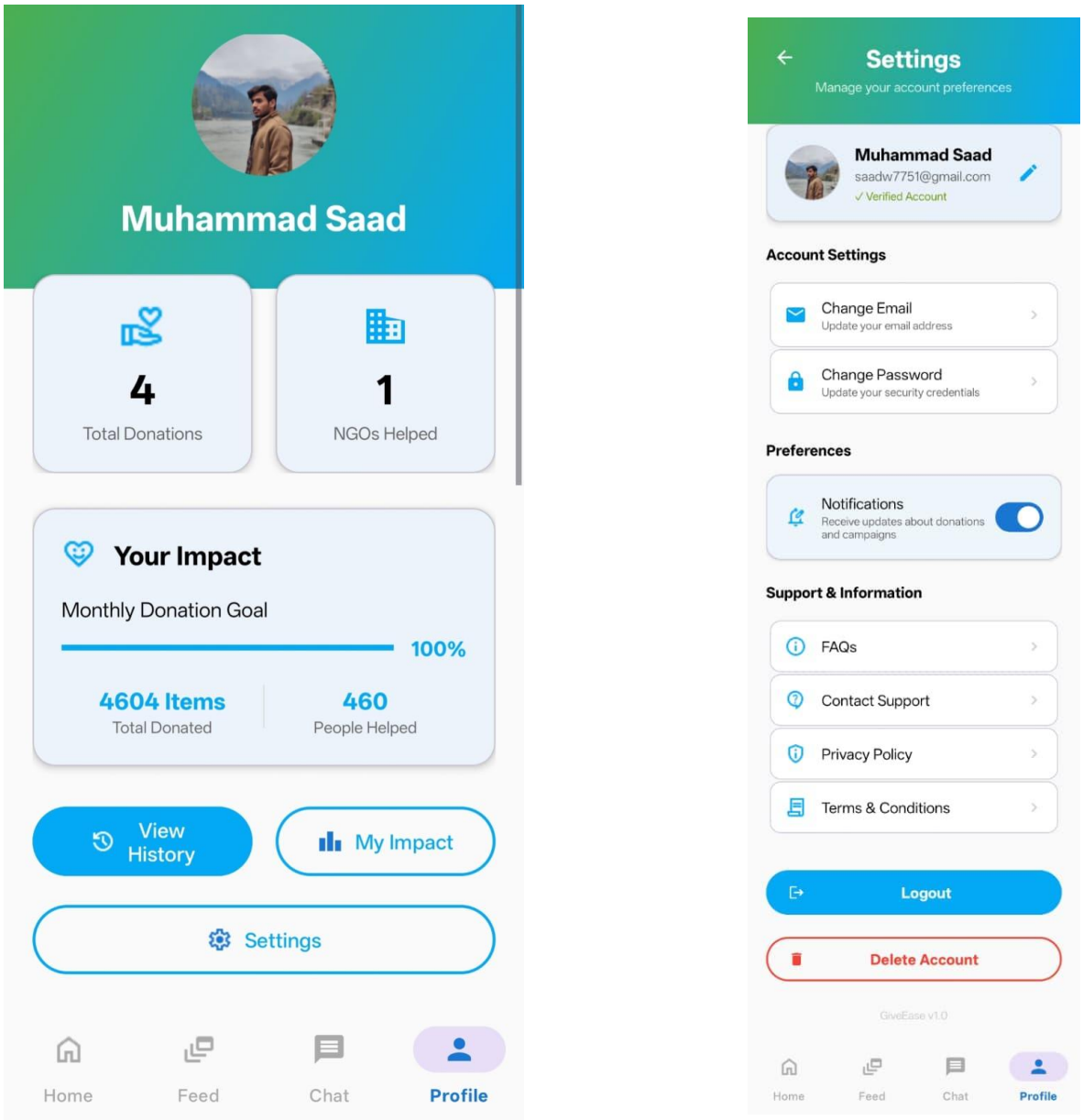


Figure 5.4.11: Donor Profile and Settings Screen

5.4.12 NGO Dashboard Screen

The NGO dashboard provides tools for managing campaigns. It displays total funds raised, number of active campaigns, number of donors, and a list of the NGO's campaigns with options to edit, pause, or complete each campaign. A floating action button allows creating new campaigns.

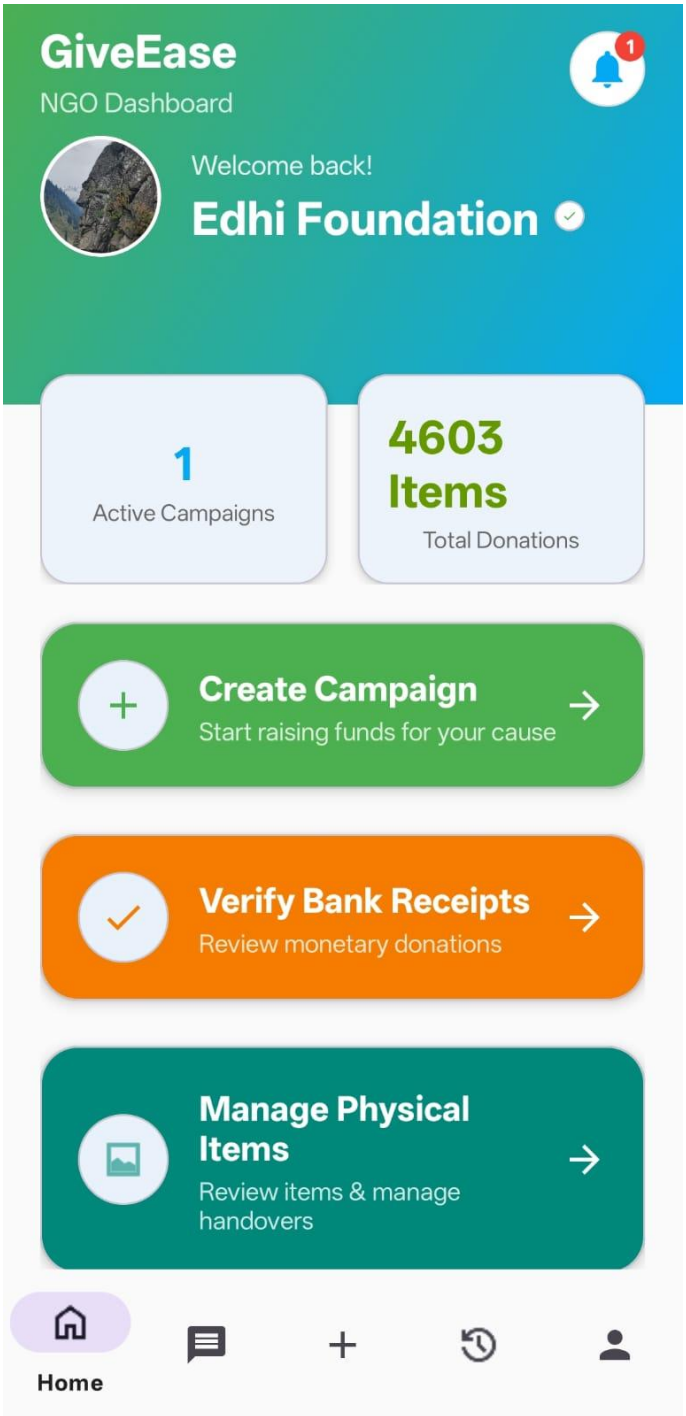


Figure 5.4.12: NGO Dashboard screen

5.4.13 Create Campaign Screen

This screen allows NGOs to create new donation campaigns. It includes input fields for title, description, target amount, category dropdown, urgency dropdown, and cover image upload. A "Publish Campaign" button submits the form.

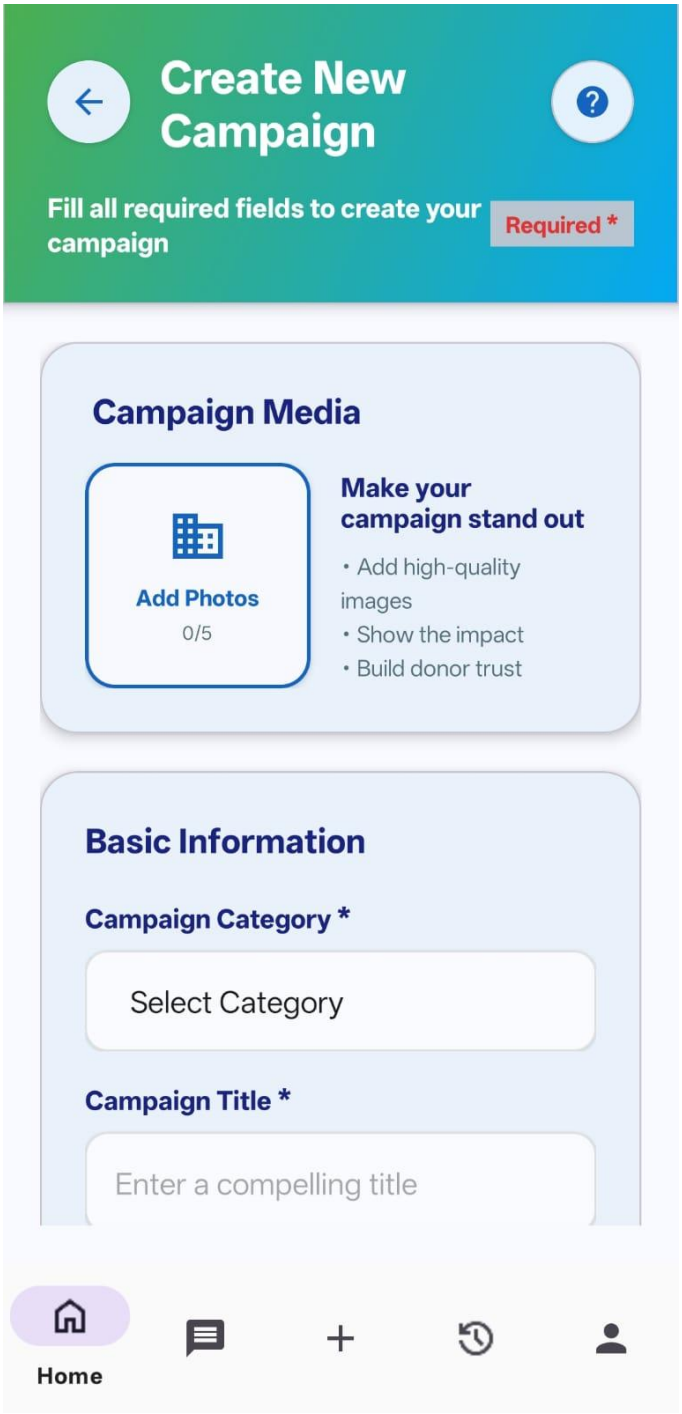


Figure 5.4.13: Create Campaign screen

5.4.14 NGO All Campaigns Screen

This screen displays all campaigns created by the logged-in NGO, including active, paused, and completed campaigns. Each campaign card shows the campaign title, target amount, raised amount with progress bar, current status badge (Active/Paused/Completed), number of donors contributed, and days remaining. A floating action button at the bottom right allows quick creation of a new campaign. NGOs can tap any campaign card to edit, pause, resume, or view detailed analytics. A filter menu helps locate specific campaigns.

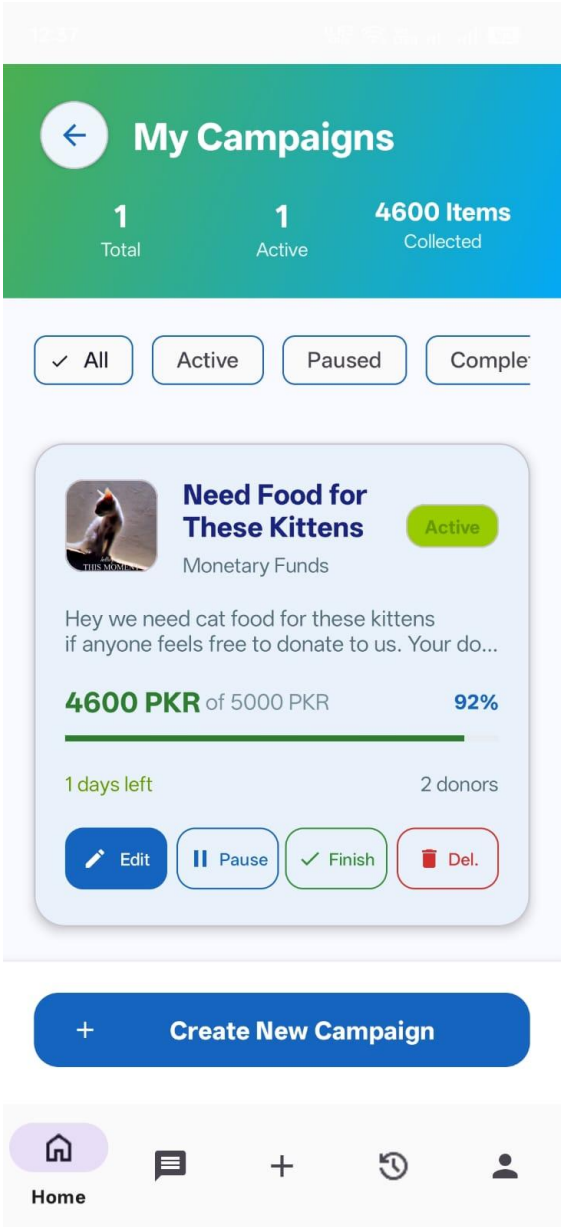


Figure 5.4.14: NGO All Campaigns Screen

5.4.15 NGO Physical Item Verification Dashboard

This screen allows NGOs to manage and verify incoming physical item donations. It displays a list of pending physical donations requiring verification. Each item entry shows the donor's name, item category (Clothes/Food/Books/Electronics/Furniture/Medical Supplies), quantity, uploaded photos of the items with a preview button, delivery status, and donor contact information. NGOs can tap "Approve" to accept the donation (which triggers automatic reward assignment to the donor) or "Reject" with a reason for rejection.

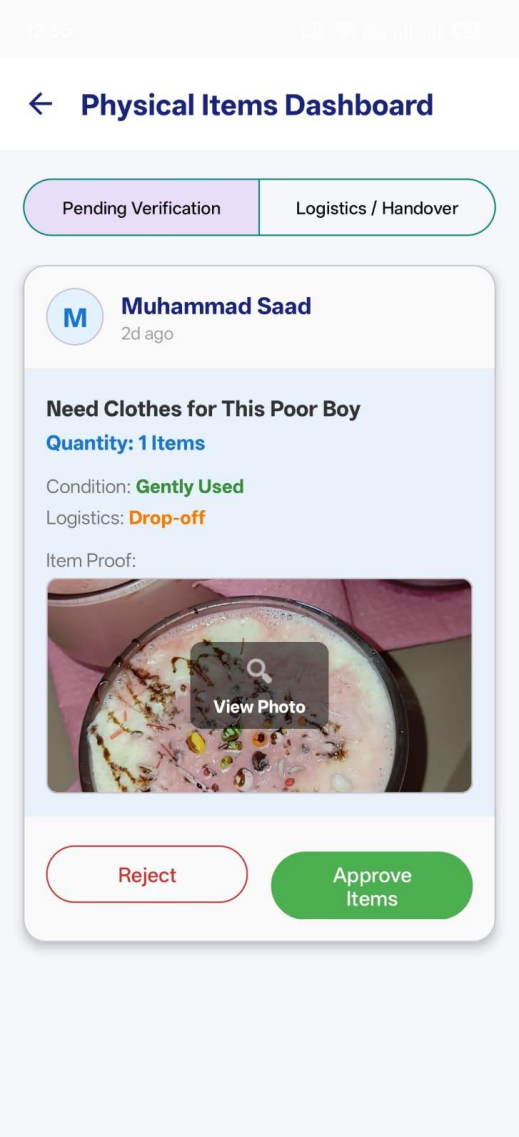


Figure 5.4.15: NGO Physical Item Verification Dashboard

5.4.16 NGO Notifications Screen

This screen displays all push notifications received by the NGO. Notifications include alerts for new donations received, new donor messages in chat, verification approval/rejection status updates, withdrawal request status changes, and system announcements from admin. Each notification shows the title, message content, timestamp, and an unread indicator. NGOs can tap any notification to navigate to the relevant screen (e.g., donation details, chat, or campaign analytics). A "Mark All as Read" button is available.

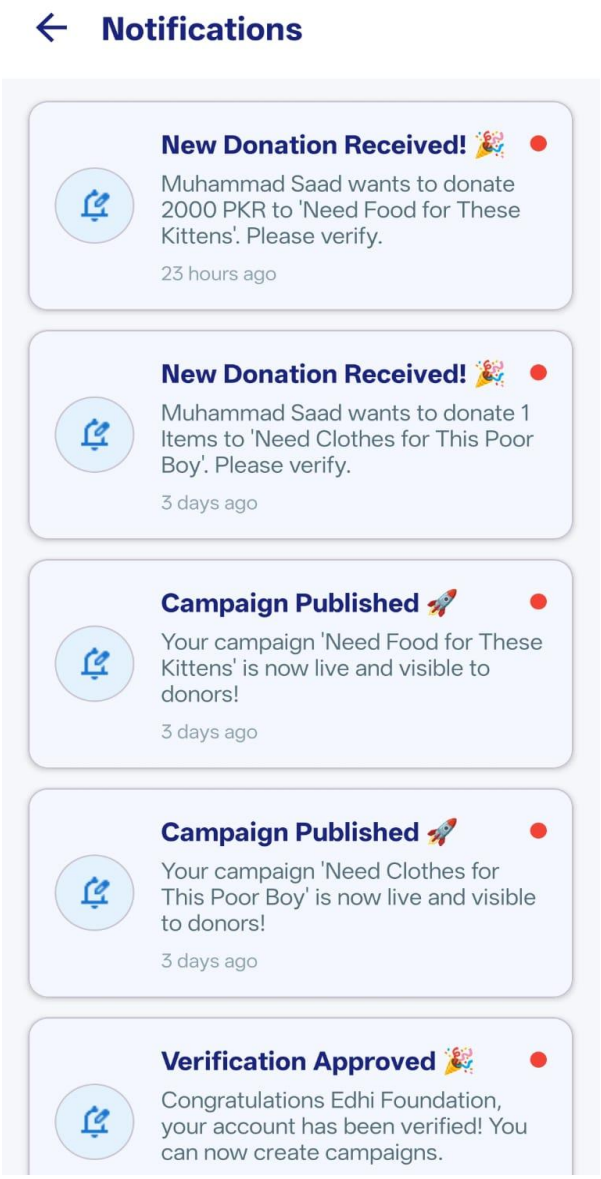


Figure 5.4.16: NGO Notifications Screen

5.4.17 NGO Profile Screen

This screen displays the NGO's organizational information. It shows the NGO's profile picture/logo, organization name, official registration number, email address, physical address, and verification status (green "Verified" badge displayed prominently). Action buttons include "Edit Profile" to update organizational information, "View Documents" to see uploaded registration documents, other options in NGO profile settings bank account information, Logout, and delete account.

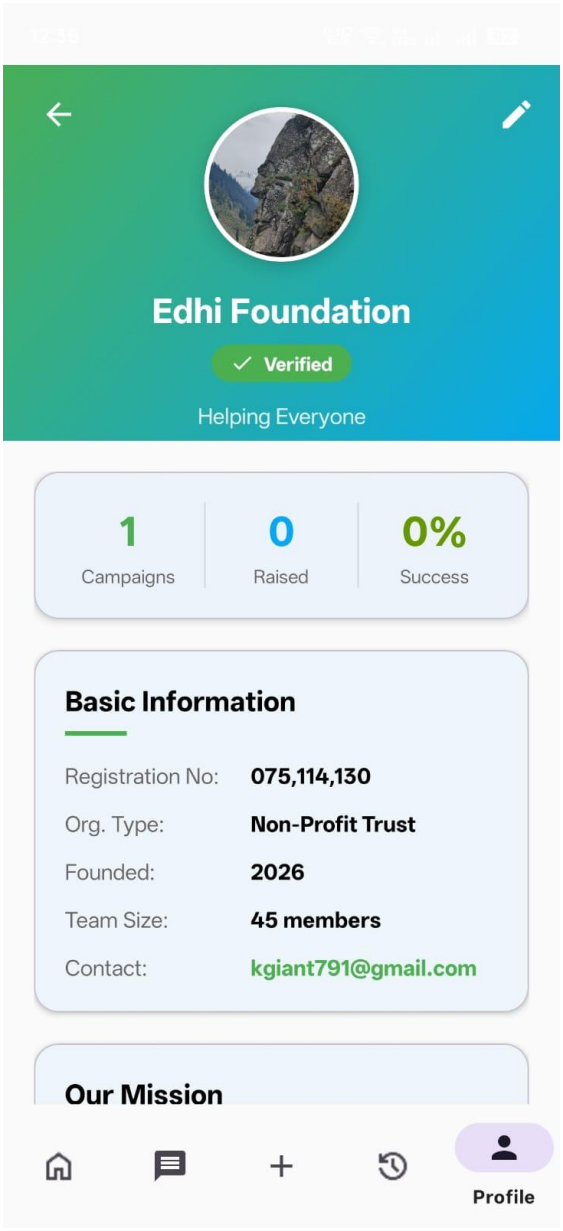


Figure 5.4.17: NGO Profile Screen

5.4.18 Admin Panel Screen

The admin panel provides system management tools. It displays platform statistics including total users, total campaigns, and total donations. Navigation tabs allow switching between User Management, Verification Queue, Reports, and Settings.

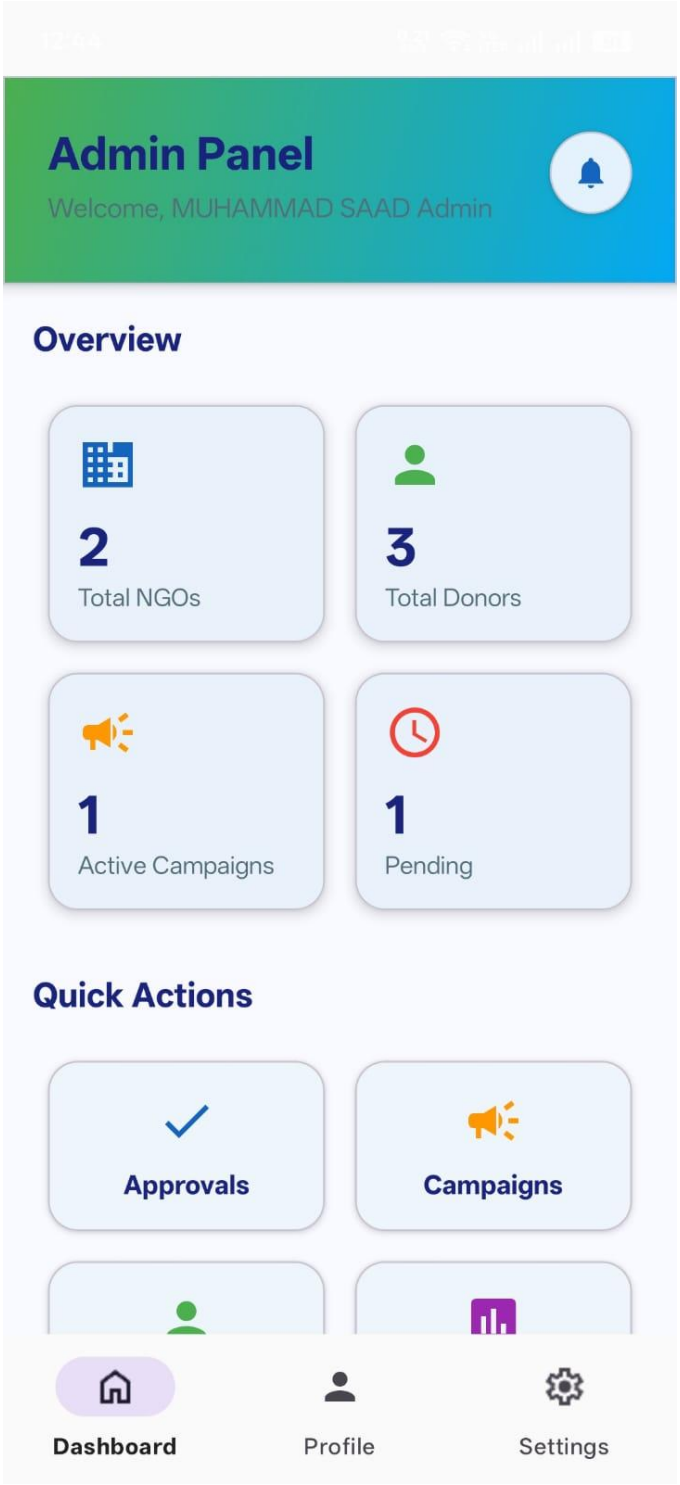


Figure 5.4.18: Admin Panel screen

5.4.19 Admin Verification Screen

This screen allows admins to review NGO registration documents and Donors documents (CNIC, Passport etc.). It displays the NGO's and Donor's profile information, a list of uploaded documents with preview buttons, and Approve and Reject buttons. A rejection dialog allows entering a reason.

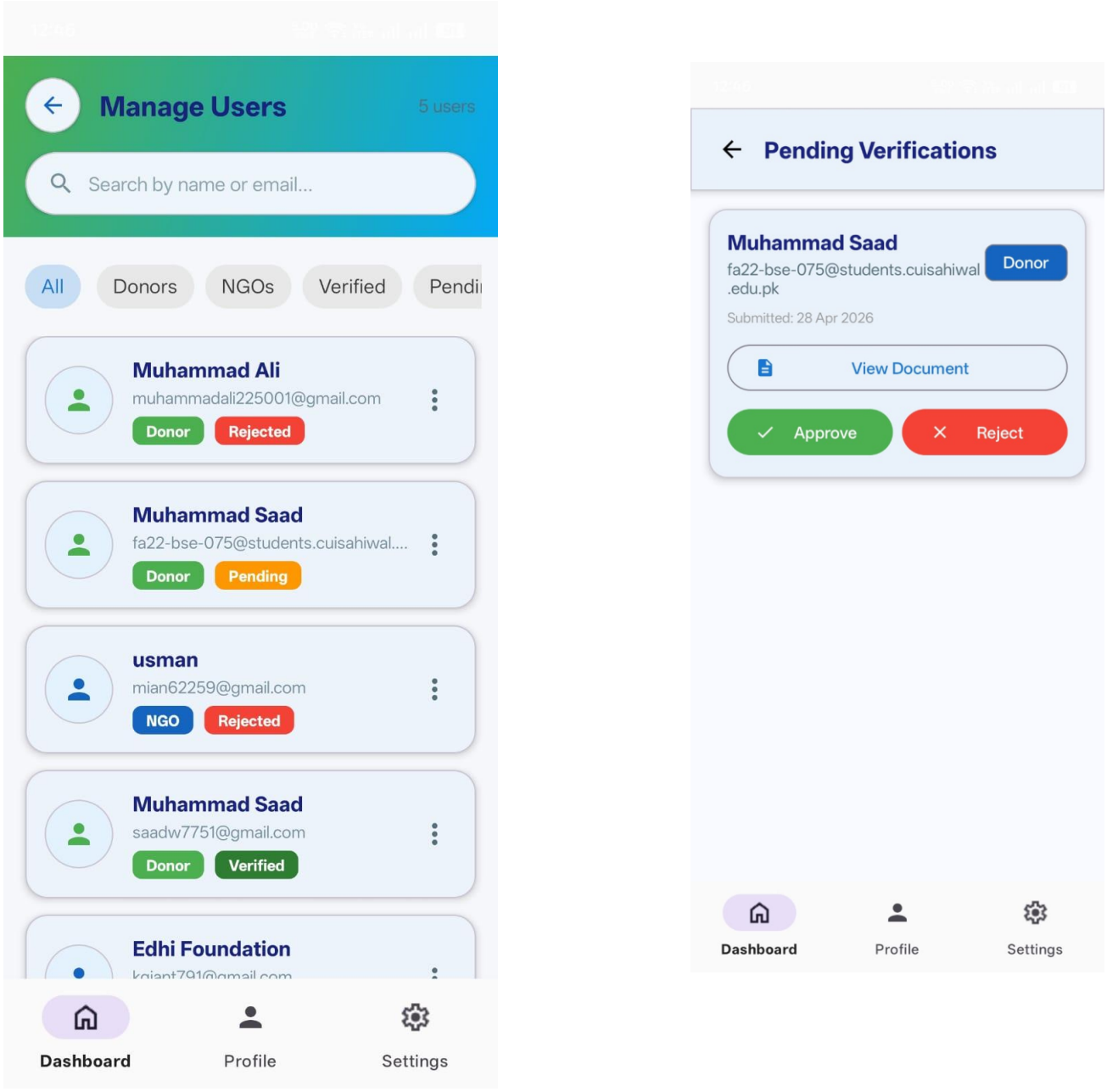


Figure 5.4.19: Admin Verification screen

5.4.20 Admin Global Donations Monitoring Screen

This screen allows the administrator to view and monitor all donations made on the platform across all NGOs and donors. Each donation entry includes donor name and NGO name and campaign title, donation amount or item details with quantity or blood type, date and timestamp of donation. Admins can filter donations.

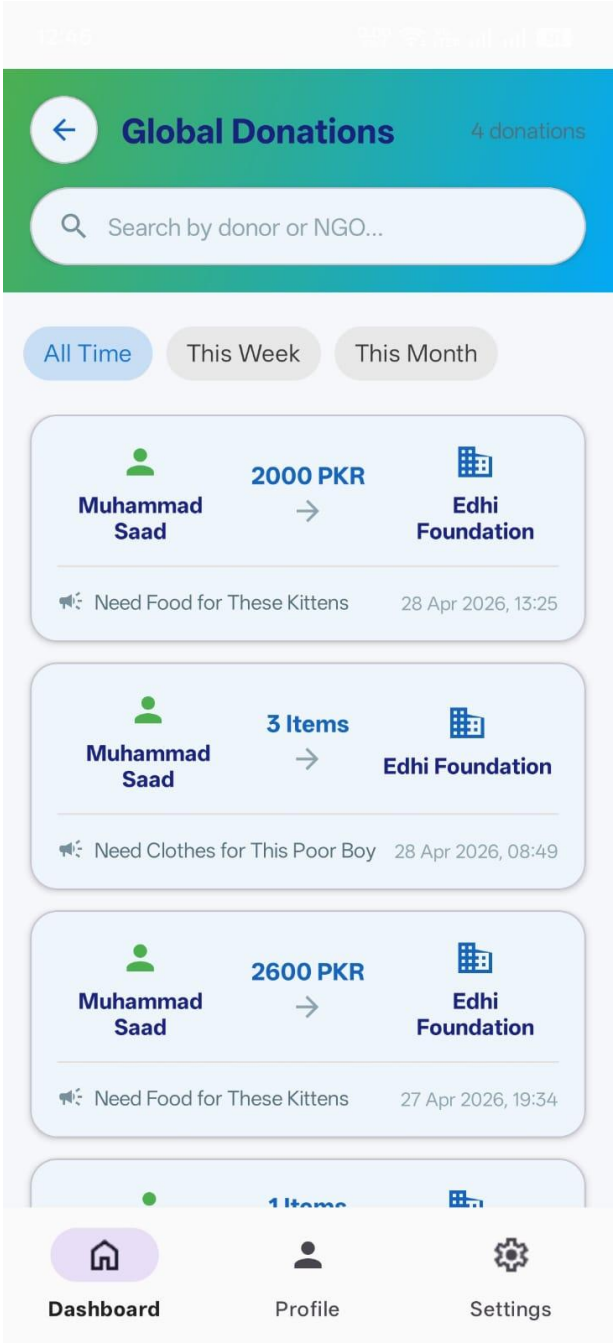
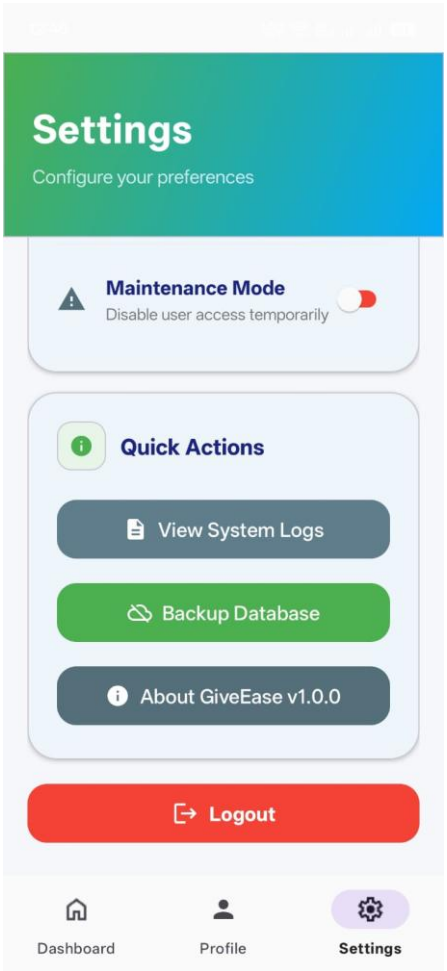


Figure 5.4.20: Admin Global Donations Monitoring Screen

5.4.21 Admin Settings Screen

This screen allows the administrator to configure global platform settings and perform system actions. Under "Maintenance Mode", the admin can disable user access temporarily. Quick Actions include "View System Logs", "Backup Database", and "About GiveEase v1.0.0". The "Auto-Approve" toggle allows automatically approving pre-verified NGOs. A "Select Data to Export" section provides options to export Users Dataset or All Donations with a Cancel button. Bottom navigation includes Dashboard, Profile, and Settings options with a Logout button. Below is the screenshot of the admin settings screen.



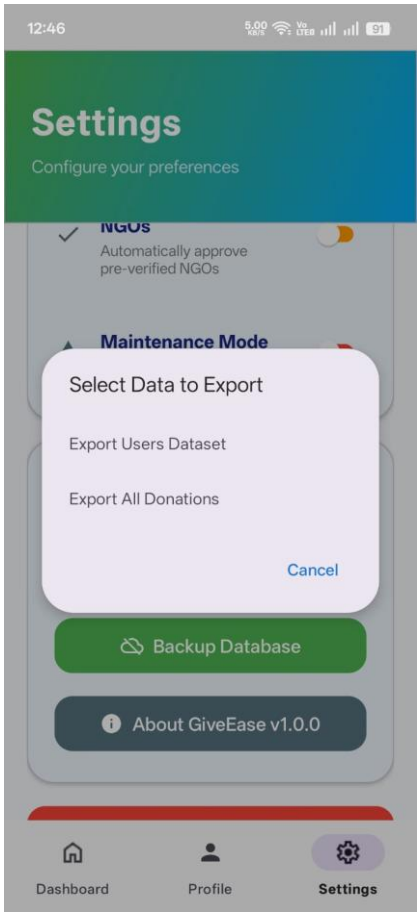


Figure 5.4.21: Admin Settings Screen

5.4.22 Admin System Logs Screen

This screen displays a chronological log of maintenance mode activities performed by the admin. Each log entry shows the action "Admin turned maintenance mode ON/OFF" along with the date and timestamp of the action. The logs help track when the system was placed into or taken out of maintenance mode for audit and debugging purposes. Bottom navigation includes Dashboard, Profile, and Settings options. Below is the screenshot of the admin system logs screen.

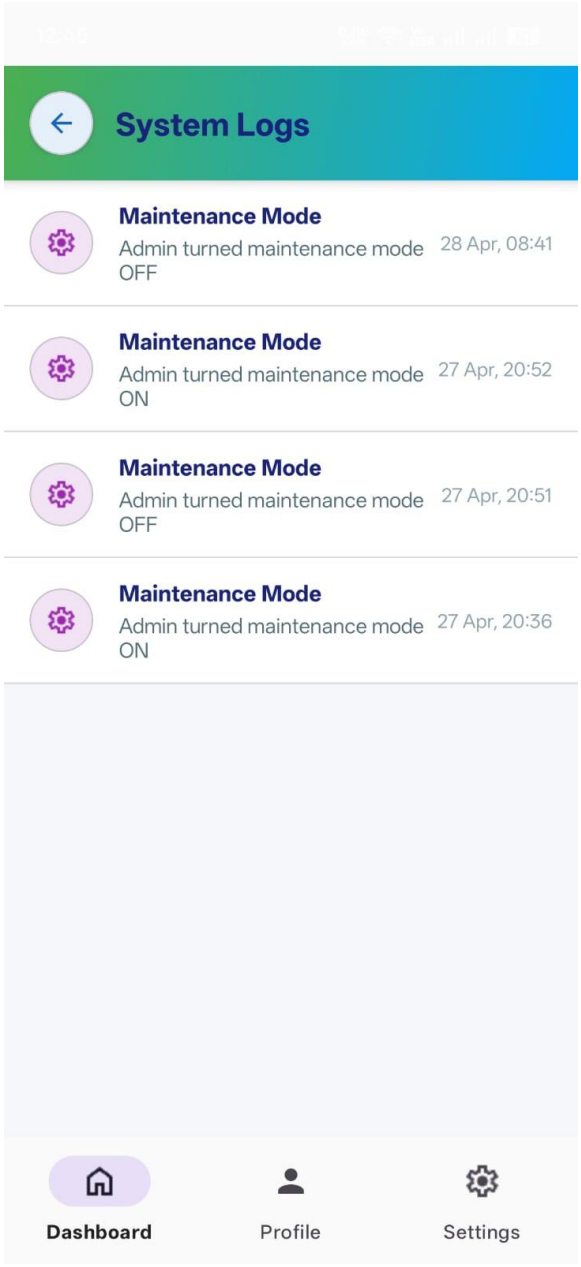


Figure 5.4.22: Admin System Logs Screen

CHAPTER 6

Testing and Evaluation

6. TESTING AND EVALUATION

Testing is a critical phase of the software development lifecycle, especially for a donation platform like GiveEase that handles user data, financial transactions (mock payments), and real-time communication. This chapter presents the testing strategy, key test cases, and results obtained during development. The testing process followed a multi-layered approach including unit testing, integration testing, functional testing, system testing, and user acceptance testing.

6.1 Testing Strategy

The testing process for GiveEase followed a sequential approach. Unit Testing verified individual components in isolation. Integration Testing verified communication between modules and Firebase services. Functional Testing ensured each feature meets its requirements. System Testing evaluated the complete application as a unified system. User Acceptance Testing was conducted with three local NGOs and ten individual donors.

6.2 Unit Testing

Unit tests focused on business logic in data classes. The following table shows the key unit test results:

Table 6.2: Unit testing

Test ID	Test Name	Expected Result	Status
UT-01	Campaign Progress Calculation	Progress =45%	Pass
UT-02	Monetary Donation Points	Points = 5	Pass
UT-03	Physical Donation Points	Points = 60	Pass
UT-04	Blood Donation Points	Points = 50	Pass
UT-05	Bronze Badge Assignment	Bronze badge awarded	Pass

UT-06	Silver Badge Assignment	Silver badge awarded	Pass

6.3 Integration Testing

Integration tests verified communication between the app and Firebase services.

Table 6.3: Integration testing

Test ID	Test Name	Expected Result	Status
IT-01	User Registration Flow	Firebase Auth and Firestore updated	Pass
IT-02	Donation Creation	Campaign raisedAmount updated	Pass
IT-03	Chat Room Creation	Chat room created in Realtime DB	Pass
IT-04	Real-Time Message Delivery	Message delivered within 1 second	Pass
IT-05	Maintenance Mode Detection	Non-admin users redirected	Pass

6.4 Functional Testing

Functional tests verified that each feature meets its requirements.

Table 6.4.1: Authentication Tests

Test ID	Test Name	Expected Result	Status
FT-01	Successful Login	Redirect to role dashboard	Pass
FT-02	Failed Login	Error message displayed	Pass
FT-03	User Registration	Account created in Firebase	Pass

Table 6.4.2: Donor Panel Tests

Test ID	Test Name	Expected Result	Status
FT-04	Browse Campaigns	List of active campaigns displayed	Pass
FT-05	Filter by Category	Only medical campaigns shown	Pass
FT-06	Make Monetary Donation	Donation recorded, amount updated	Pass
FT-07	Make Physical Donation	Donation recorded, chat created	Pass

Table 6.4.3: NGO Panel Tests

Test ID	Test Name	Expected Result	Status
FT-08	Create Campaign	Campaign appears in donor feed	Pass
FT-09	Pause Campaign	Removed from donor feed	Pass

Table 6.4.4: Admin Panel Tests

Test ID	Test Name	Expected Result	Status
FT-10	Approve NGO	isVerified = true, notified	Pass
FT-11	Enable Maintenance Mode	Non-admin users redirected	Pass

Table 6.4.5: Chat and Reward Tests

Test ID	Test Name	Expected Result	Status
FT-12	Send Chat Message	Message delivered instantly	Pass
FT-13	Send Image in Chat	Image uploaded and displayed	Pass
FT-14	Earn Bronze Badge	Badge appears in dashboard	Pass

6.5 System Testing

End-to-end system tests verified the complete donation lifecycle.

Table 6.5.1: Test Case ST-01 Complete Donation Lifecycle

Element	Description
Steps	NGO creates campaign → Donor donates → Chat opens → NGO confirms → Badge awarded
Expected Result	All steps complete successfully
Actual Result	Full flow executed without errors
Status	Pass

Table 6.5.2: Test Case ST-02 Concurrent Donations

Element	Description
Steps	Three donors donate simultaneously to same campaign
Expected Result	No data loss, correct total amount
Actual Result	All donations processed correctly
Status	Pass

6.6 Performance Testing

Performance tests measured response times under normal 4G network conditions.

Table 6.6: Performance Testing

Operation	Target	Actual	Status
Campaign feed load	Less than 3 seconds	2.1 seconds	Pass
Chat message delivery	Less than 1 second	0.8 seconds	Pass
Image upload	Less than 5	3.2	Pass

	seconds	seconds	
App cold start	Less than 4 seconds	2.5 seconds	Pass
Push notification	Less than 5 seconds	2.3 seconds	Pass

6.7 Security Testing

Security tests verified Firebase Security Rules enforce proper access control.

Table 6.7: Security Testing

Test Scenario	Expected Result	Status
Donor tries to edit NGO campaign	Write rejected	Pass
NGO tries to access admin settings	Write rejected	Pass
User tries to read another user's profile	Read rejected	Pass
Unauthenticated user tries to view campaigns	Read rejected	Pass

6.8 User Acceptance Testing

UAT was conducted with three local NGOs and ten individual donors.

Table 6.8.1: NGO Feedback (Average Rating 4.6 out of 5)

Aspect	Rating
Ease of creating campaigns	4.8
Real-time chat usefulness	4.5
Donation tracking	4.7
Overall satisfaction	4.6

Table 6.8.2: Donor Feedback (Average Rating 4.7 out of 5)

Aspect	Rating
Ease of finding campaigns	4.9
Donation process simplicity	4.7
Impact dashboard usefulness	4.8
Overall satisfaction	4.8

Table 6.8.3: Issues Fixed During Testing

Issue	Resolution
Image upload failure for large files	Implemented compression before upload
Chat messages out of order	Fixed timestamp sorting
NGO could not edit campaign	Added edit functionality

6.9 Testing Summary

Table 6.9: Testing Summary

Test Type	Tests Executed	Passed	Pass Rate
Unit Tests	6	6	100%
Integration Tests	5	5	100%
Functional Tests	14	14	100%
System Tests	2	2	100%
Security Tests	4	4	100%
Performance Tests	5	5	100%
Total	**36**	**36**	**100%**

All tests passed successfully. The GiveEase application is stable, secure, and ready for deployment.

CHAPTER 7

Conclusion

7. CONCLUSION AND FUTURE WORK

This chapter concludes the work done in the GiveEase Donation App project and highlights the lessons learned, accomplishments, and potential areas of extension for future development. As a comprehensive Android-based donation management system, GiveEase integrates multiple user roles, real-time communication, donation processing, gamified rewards, and administrative controls into a single unified mobile application. This chapter reflects on the system's effectiveness, limitations, and long-term contributions to digital philanthropy.

7.1 Conclusion

GiveEase was designed as a direct response to the critical challenges of trust, transparency, and inefficiency in charitable donation systems. The core problem identified was the absence of a trusted, integrated, multi-functional, and engaging digital ecosystem that effectively bridges the gap between the willingness to give and the actual, efficient delivery of resources to those in need. The system serves as a digital platform for donors and NGOs, combining the power of mobile computing, real-time cloud databases, secure authentication, and user-centric design. Its goal is to simplify the donation process, build trust through verification, promote sustained giving through gamification, and provide transparency through impact tracking.

7.1.1 Major Achievements of GiveEase

Integration of Three Donation Types: Unlike most donation applications that focus only on monetary contributions, GiveEase successfully integrates three distinct donation types (money, physical items, and blood) into a single platform. Each donation type has its own workflow, with physical donations requiring image uploads and blood donations requiring location and availability details.

Real-Time Communication: The implementation of real-time chat using Firebase Realtime Database allows donors and NGOs to coordinate logistics seamlessly. The chat supports both text messages and image sharing, which is essential for physical item donations where donors need to show what they are donating.

Trust Through Verification: The manual verification workflow for NGOs, combined with the visual verified badge, addresses the critical trust deficit identified in the problem statement. Donors can easily identify legitimate organizations, and NGOs have an incentive to complete verification

to gain donor confidence.

Gamified Reward System: The badge-based reward system (Bronze, Silver, Gold, Platinum) provides donors with tangible recognition for their contributions. The Impact Dashboard quantifies donations into impact points, making donors feel their contributions matter and encouraging repeat giving.

Comprehensive Admin Controls: The admin panel provides full governance over the platform, including NGO verification, maintenance mode to lock the app during updates, and auto-approve toggles for high-volume periods. The maintenance mode feature is a unique contribution not found in similar platforms.

Scalable Cloud Architecture: By using Firebase as a complete Backend-as-a-Service solution, GiveEase is serverless, scalable, and cost-effective. The real-time synchronization ensures that donors see updated campaign progress immediately when donations are made.

7.1.2 Challenges Addressed During Development

During development, several challenges were encountered and successfully resolved:

Image Storage Optimization: Large image sizes threatened to exceed Firebase Cloud Storage free tier limits. This was resolved by implementing image compression using the Glide library before upload, reducing file sizes by approximately 70-80%.

Real-Time Data Synchronization: Ensuring that chat messages and donation updates appear instantly on all devices required careful implementation of Firestore and Realtime Database listeners. The solution involved using snapshot listeners for Firestore and child event listeners for Realtime Database.

Role-Based Access Control: Preventing users from accessing features outside their role required implementing Firebase Security Rules and client-side navigation logic. The MainActivity checks the user's role on every launch and loads the appropriate fragment.

Offline Support: Handling network interruptions during donation submission required implementing local caching and queueing mechanisms. Donations and messages are stored locally when offline and synced when connectivity returns.

App Crash Prevention: The application experienced occasional crashes due to null pointer exceptions, memory leaks, and unhandled network failures. This was resolved by implementing comprehensive error handling with try-catch blocks, utilizing Kotlin's null safety features

(?. and ?:), integrating Firebase Crashlytics for crash monitoring, and fixing memory leaks detected by LeakCanary. The application now maintains stable operation with zero crashes during extended use.

7.1.3 Meeting the Objectives

The objectives defined in the project scope were successfully achieved:

Objective 1: A robust manual verification process for donors (CNIC) and NGOs (registration documents) was implemented, ensuring platform authenticity and building user confidence.

Objective 2: Three distinct donation categories (physical items, money, and blood) are supported within a single unified interface, each with its own tailored workflow.

Objective 3: Real-time coordination between donors and NGOs is enabled through an in-app chat system with text and image sharing capabilities.

Objective 4: Donor engagement and retention are increased through a milestone-based badge system (Bronze, Silver, Gold, Platinum) and a quantifiable Impact Dashboard showing total impact points.

Objective 5: Administrators have full control over user verification, request moderation, dispute resolution (through chat monitoring), and platform governance including maintenance mode.

Objective 6: The solution is scalable, real-time, secure, and maintainable, leveraging modern cloud technologies (Firebase) and following the MVVM architectural pattern.

7.1.4 Limitations

Despite its successes, GiveEase has several limitations:

1. **Payment Simulation:** The monetary donation feature uses mock payment, not real financial transactions, limiting real-world deployment.
2. **Android-Only:** The application is currently only available on Android, excluding iOS users.
3. **Manual Verification Bottleneck:** As the platform grows, manual NGO verification may become a bottleneck requiring more admin resources.
4. **Internet Dependency:** The app requires continuous internet connectivity for most operations, limiting use in areas with poor connectivity.

5. **Limited Scalability Testing:** Performance testing was conducted with limited concurrent users; the system's behavior under extreme load (10,000+ users) hasn't been verified.

7.2 Future Work

Although GiveEase provides a comprehensive solution for digital donation management, the potential for future enhancements is vast. The following are recommended areas for improvement and expansion:

7.2.1 Real Payment Gateway Integration

The current version uses mock payment simulation for monetary donations. Future work should integrate real payment gateways such as JazzCash, Easypaisa, or Stripe. This would allow donors to actually transfer money through the app rather than relying on NGO-provided bank details. The integration would require implementing secure payment processing, handling transaction confirmations, and updating donation status automatically upon successful payment.

7.2.2 iOS and Web Versions

Currently, GiveEase is an Android-only application. Developing an iOS version using Flutter or Kotlin Multiplatform would expand the user base significantly. Additionally, a web-based admin dashboard would allow administrators to manage the platform from desktop computers, which is often more convenient for tasks like document verification and analytics review.

7.2.3 Automated Verification using AI

The current verification process is manual, requiring admins to review each NGO document. This becomes a bottleneck as the platform grows. Future work could implement AI-based document verification using Optical Character Recognition (OCR) and machine learning to automatically extract and validate information from CNICs and registration certificates. Admin review would then only be required for edge cases or appeals.

7.2.4 Push Notification Enhancements

While basic push notifications are implemented, future enhancements could include:

- Scheduled notifications for recurring donation reminders
- Location-based notifications when donors are near an NGO drop-off point
- Personalized notification preferences allowing users to select which categories they want to hear about

7.2.5 Advanced Analytics Dashboard

The current admin analytics are basic. An advanced analytics dashboard could include:

- Charts showing donation trends over time
- Geographic heat maps showing donor locations
- NGO performance metrics (response time, completion rate)
- Donor retention and churn analysis

7.2.6 Offline-First Architecture

Currently, the app requires an internet connection for most operations. An offline-first architecture would allow users to browse cached campaigns, create draft donations, and send queued messages even without connectivity. All operations would sync automatically when the connection is restored.

7.2.7 Multilingual Support

The app currently uses English only. Adding Urdu language support would make the app accessible to a wider demographic in Pakistan, particularly users who are more comfortable with their native language. This would involve translating all UI strings, error messages, and notifications.

7.3 Final Thoughts

The development of GiveEase demonstrates how modern mobile and cloud technologies can revolutionize traditional donation approaches. The system not only functions as a donation platform but also empowers donors to track their impact and encourages NGOs to maintain transparency.

The combination of Firebase cloud services, real-time databases, secure authentication, and modern

Android development (Kotlin, MVVM) makes GiveEase a strong foundation for future innovations in digital philanthropy. With ongoing improvements, the application has the potential to become a reliable companion for charitable giving across Pakistan and beyond.

The project successfully achieved its objectives within the constraints of a three-member student team and a 6-8 month timeline. All core features are fully functional, tested, and documented. The application is ready for deployment and has the potential to make a meaningful social impact by making charitable giving more transparent, trustworthy, and engaging.

8. REFERENCES

[1] Android Developers, "Kotlin for Android," Google, 2024.
Available: https://developer.android.com/kotlin

[2] Firebase, "Firebase Documentation," Google, 2024.
Available: https://firebase.google.com/docs

[3] Firebase, "Cloud Firestore Documentation," Google, 2025.
Available: https://firebase.google.com/docs/firestore

[4] Firebase, "Firebase Realtime Database Documentation," Google, 2025.
Available: https://firebase.google.com/docs/database

[5] Firebase, "Cloud Functions for Firebase," Google, 2025.
Available: https://firebase.google.com/docs/functions

[6] Firebase, "Firebase Cloud Storage Documentation," Google, 2025.
Available: https://firebase.google.com/docs/storage

[7] Firebase, "Firebase Cloud Messaging (FCM)," Google, 2025.
Available: https://firebase.google.com/docs/cloud-messaging

[8] Google, "Material Design 3 Guidelines," Google, 2025. Available: https://m3.material.io/

[9] Bumptech, "Glide Documentation," Bumptech, 2024
Available: https://bumptech.github.io/glide/

[10] I. Sommerville, Software Engineering, 11th ed. Boston, MA, USA: Pearson, 2023.

[11] R. S. Pressman and B. Maxim, Software Engineering: A Practitioner's Approach, 9th ed. New York, NY, USA: McGraw-Hill Education, 2020.

[12] M. Fowler, UML Distilled: A Brief Guide to the Standard Object Modeling Language, 3rd ed. Boston, MA, USA: Addison-Wesley, 2003.

[13] K. Wiegers and J. Beatty, Software Requirements, 3rd ed. Redmond, WA: Microsoft Press, 2013.

[14] ISO/IEC/IEEE International Standard, "Systems and software engineering Life cycle processes Requirements engineering," ISO/IEC/IEEE 29148:2018, pp. 1-94, 2018.

[15] N. Kumar and P. Sharma, "Gamification in Donation Apps: Enhancing Donor Engagement," IEEE International Conference on Humanized Computing and Communication, 2023.

[16] S. Patel and A. Gupta, "Secure User Verification in Mobile Donation Systems," International

Journal of Computer Applications, vol. 184, no. 5, 2023.

[17] M. S. Al-Rahmi et al., "Factors affecting the adoption of mobile donation applications in developing countries," Journal of Theoretical and Applied Information Technology, vol. 98, no. 15, pp. 1-15, 2020.

[18] OWASP Foundation, "OWASP Mobile Application Security Project," 2024. Available: https://owasp.org/www-project-mobile-app-security/

[19] European Union, "General Data Protection Regulation (GDPR) - Official Legal Text," Regulation (EU) 2016/679, 2016. Available: https://gdpr-info.eu/

[20] Pakistan Centre for Philanthropy (PCP), "State of Social Sector Report," 2023. Available: https://www.pcp.org.pk

[21] GoFundMe, "Crowdfunding Platform," GoFundMe, 2024. Available: https://www.gofundme.com

[22] DonorsChoose, "Citizen Philanthropy Platform," DonorsChoose, 2024. Available: https://www.donorschoose.org

[23] BloodConnect, "Blood Donor Registry," BloodConnect, 2024. Available: https://www.bloodconnect.in